

Decentralized Identification (DxID) – Technical Specification & Whitepaper

Introduction

Decentralized Identification (DxID) is a privacy-preserving digital identity framework that uses cutting-edge zero-knowledge cryptography to enable secure and verifiable identities without sacrificing user privacy. It leverages **zk-STARKs** (Zero-Knowledge Scalable Transparent Arguments of Knowledge) as the core proof mechanism to achieve trustless verification of identity attributes at scale. With DxID, individuals, organizations, and even IoT devices can establish decentralized identifiers (DIDs) and verifiable credentials that can be **proven** without disclosing sensitive underlying data. This technical document serves as a comprehensive specification, combining a whitepaper's depth, a system architecture description, and a developer-focused README. It is intended for developers, researchers, and stakeholders to understand and implement the DxID system.

Motivation: Today's digital identity solutions often rely on centralized authorities and vulnerable credential stores. Users are forced to overshare personal information to prove simple claims (such as age or citizenship), leading to privacy leaks and large-scale data breaches. Furthermore, emerging threats like deepfakes and misinformation erode trust in digital content, calling for robust methods to authenticate information provenance. DxID addresses these challenges by putting users in control of their identity data and employing **zero-knowledge proofs (ZKPs)** to confirm identity assertions without revealing private information [chain.link](#). The use of zk-STARKs provides both **scalability** and **post-quantum security**, future-proofing the identity system against quantum adversaries [threeprotocol.ai](#) [threeprotocol.ai](#).

Key Features of DxID:

- **Self-Sovereign Identity:** Users create and control their own decentralized identifiers without reliance on a central authority. Each identity is anchored in a blockchain-based registry, but personal data stays off-chain under user control.
- **Selective Disclosure:** Utilizing ZKPs, particularly zk-STARK proofs, users can prove claims about their attributes (age, qualifications, rights, etc.) without exposing the actual data [info.suresync.nl](#). This ensures compliance with principles like data minimization and privacy by design.
- **No Trusted Setup & Post-Quantum Security:** Unlike some earlier ZK systems, zk-STARKs require no trusted setup ceremony [threeprotocol.ai](#). They rely on collision-resistant hash functions and information-theoretic security, making DxID

resilient against quantum computing threatsthreeprotocol.ai.

- **High Scalability:** The architecture is designed to handle high volumes of identity proofs and verifications. zk-STARKs can support large transaction throughput due to efficient proof generation and transparent verification, suitable for Internet-scale identity usagethreeprotocol.ai.
- **Content Authentication:** DxID includes a content signing protocol where any digital content (documents, images, videos, etc.) can be signed by an identity and accompanied by a proof of authenticity. This helps establish content provenance, which is crucial for combating misinformation. Every piece of content can be verified as untampered and genuinely originating from the claimed identity in real-time.
- **Interoperability:** The system aligns with emerging standards for decentralized identity (W3C Decentralized Identifiers and Verifiable Credentials). It is designed to integrate with or complement existing DID methods (like Sovrin, uPort, Veres One) while offering enhanced privacy and security features.
- **Smart Contract Integration:** DxID provides on-chain smart contract interfaces for managing identity records and verifying zk-STARK proofs of identity attributes on-chain. This enables decentralized applications (dApps) to easily consume verified identity information, for example in DeFi compliance or supply chain audits, without ever seeing the raw private data.

This document is organized as follows: **Section 2** covers the cryptographic foundations (zero-knowledge proofs, STARKs, and the mathematical background) that underpin DxID. **Section 3** details the system architecture, including how identities are created, how attributes and credentials are managed off-chain, and how the content authentication protocol works (with diagrams). **Section 4** explores full use cases across industries – from financial KYC to healthcare and education credentials, supply chain traceability, and misinformation prevention – illustrating how DxID is applied in each scenario. **Section 5** dives into the zk-STARK implementation details: how proofs are constructed and verified in DxID. **Section 6** describes the blockchain integration: the smart contract design, on-chain DID registry, and metadata storage. **Section 7** compares DxID to existing decentralized identity systems (Sovrin, uPort, Veres One, etc.) to highlight differences in trust model, privacy, and security. **Section 8** discusses the security benefits of DxID, with a focus on cryptographic strength and post-quantum resilience. **Section 9** examines integration potential with the Internet of Things (IoT) and artificial intelligence (AI), and how cross-border identities can be facilitated. **Section 10** outlines the vision for resisting misinformation by using DxID for content authenticity. **Section 11** is a developer-oriented README section, providing instructions on installing, building, and running a reference implementation (CLI and dApp) of DxID. Finally, **Section 12** covers the licensing and a list of references is provided in **Section 13**.

By combining theoretical foundations with practical engineering details, this document aims to serve as a definitive reference for the DxID project. **Developers** will find detailed specifications and example code to implement or integrate with DxID. **Security researchers** can review the cryptography and threat model. **Decision-makers and investors** can understand the technology's value proposition, backed by references to state-of-the-art research and industry standards.

Cryptographic Foundations

Decentralized Identification (DxID) builds upon advanced cryptographic techniques to ensure that identity information can be verified without revealing sensitive data. This section provides an overview of the cryptographic concepts underpinning DxID, notably **Zero-Knowledge Proofs (ZKPs)** and **zk-STARKs**, along with the mathematical background that makes these proofs possible. A solid understanding of these foundations is crucial for appreciating the security and privacy guarantees of the system.

Zero-Knowledge Proofs (ZKPs)

A *zero-knowledge proof* is a method by which one party (the *prover*) can convince another party (the *verifier*) that a certain statement is true, without revealing **why** it's true or any additional information beyond the validity of the statement. Formally, ZKPs satisfy three properties: **completeness** (if the statement is true and both parties follow the protocol, the verifier will be convinced), **soundness** (if the statement is false, no cheating prover can convince the verifier otherwise, except with negligible probability), and **zero-knowledge** (the verifier learns nothing other than the fact that the statement is true). This ingenious concept was first introduced in the 1980s and has since become a pillar of modern cryptography.

In the context of digital identity, ZKPs allow a user to prove possession of a credential or a certain attribute **without disclosing the credential itself or any other attributes**. For example, a user can prove "I am over 18 years old" to a service, without revealing their exact birthdate or any other personal data. The service only learns the truth of the statement (over 18) and nothing else. ZKPs thereby enable **selective disclosure** and fine-grained privacy control, which are core to the self-sovereign identity philosophy.

There are different constructions of ZKPs, including interactive proofs (where the prover and verifier engage in a back-and-forth communication) and non-interactive proofs (where a single proof string suffices). Non-interactive proofs often leverage the Fiat-Shamir heuristic to remove interaction. In blockchain and decentralized systems, non-interactive zero-knowledge proofs are typically required (since on-chain or asynchronous environments cannot do interactive handshakes easily).

Two prominent families of ZKPs that have gained traction in recent years are zk-SNARKs and zk-STARKs. Both are non-interactive succinct proofs, meaning the proof can be verified much faster than re-executing the original statement and the proof size is much smaller than the

witness (secret data) or computation itself[chain.link](#). They allow one party to prove knowledge of a secret or the correctness of a computation to anyone, publicly, without revealing the secret or the computation's details.

- **zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge):**
zk-SNARKs were introduced around 2012 and were famously applied in the cryptocurrency Zcash to enable private transactions[chain.link](#). A zk-SNARK proof attests that “I know a witness for this statement” with high efficiency. However, classic zk-SNARK systems (such as Groth16) often require a **trusted setup** – a special ceremony to generate common reference parameters. If the secret “toxic waste” from that ceremony is not destroyed, it could be used to forge proofs undetectably[chain.linkchain.link](#). This reliance on a trusted setup is seen as a potential vulnerability and a hurdle for decentralization[chain.link](#). Additionally, most SNARK constructions rely on elliptic curve cryptography and pairings, which are not post-quantum secure. Despite these drawbacks, SNARKs produce very small proofs (often just a few hundred bytes) and have fast verification times, making them practical for on-chain verification in today's environments.
- **zk-STARKs (Zero-Knowledge Scalable Transparent Arguments of Knowledge):**
zk-STARKs are a newer generation of ZKPs, introduced in a 2018 paper by Eli Ben-Sasson et al.[chain.link](#). The acronym emphasizes key properties: *Scalable* (supporting proofs of very large statements efficiently), *Transparent* (no trusted setup needed), and *Arguments of Knowledge* (computationally sound proofs of knowledge). STARKs use only basic cryptographic primitives (hash functions and information-theoretic constraints) rather than number-theoretic assumptions. They employ techniques from polynomial commitment schemes and error-correcting codes to prove statements about computations. We will delve deeper into STARKs shortly, as they form the crux of DxID's trust model.

Why Zero-Knowledge for Identity? Traditional digital identity verification involves sharing data – for example, sending a copy of your ID or certificate to a verifier, who then checks its validity. This model inherently leaks information and often requires trusting the verifier to handle your data carefully. With ZKPs, a verifier doesn't need to see or store your personal data at all; they only see a proof that some conditions are satisfied. As a result, ZKPs **dramatically reduce the exposure of personal data**, mitigating the risk of data breaches and surveillance. In the DxID system, zero-knowledge proofs are used for: proving control of an identity, proving attributes about that identity (like “I have a valid driver's license” or “My nationality is X”), and proving statements about content (like “This document was signed by an accredited journalist”). All of these can be done without revealing the underlying private information or credential content.

To illustrate, consider that in DxID, a university could issue a cryptographic credential to a student that includes their degree and graduation year. Later, the student can prove to an employer “I have a degree in Computer Science from University Y” by generating a ZKP from that credential. The proof would convince the employer of the truth of the statement, but the

employer would not see the actual certificate, transcripts, or even know exactly when the student graduated – only that the proof is valid. This approach enforces **data minimization**, as highlighted by prior self-sovereign identity systems like Sovrin, where claims are stored privately and only proofs are shared info.suresync.nl.

zk-STARKs: Scalable Transparent Arguments of Knowledge

Definition and Properties: zk-STARK stands for *Zero-Knowledge Scalable Transparent Argument of Knowledge*. STARKs are a particular type of zero-knowledge proof system focused on scalability and transparency. As mentioned, they were introduced as an alternative to SNARKs to eliminate the need for a trusted setup and to leverage only symmetric cryptography [chain.link](#). A STARK proof demonstrates that a certain computation was carried out correctly (and optionally that the prover knows some secret data that produced a public output), with the following hallmarks:

- **Transparent (No Trusted Setup):** STARKs do not require any initial ceremony or secret trapdoor. The public parameters used in proof generation and verification (like the choice of prime fields, hash functions, etc.) are generic and contain no hidden structure. This removes a significant security concern present in SNARK systems threeprotocol.ai. In STARKs, the verifier's trust is only in well-known hash functions and classical randomness – there is no need to trust that someone destroyed secret numbers after a setup. This property is extremely valuable for an identity system, as it means the entire scheme can be bootstrapped without having to trust any privileged party at inception.
- **Scalable and Fast to Generate Proofs:** STARKs are designed to handle large computations efficiently. The proving time grows quasi-linearly with the size of the computation trace (often noted as T for number of steps) – roughly on the order of $T * \text{polylog}(T)$ medium.com. This means that generating a proof for a computation with millions of steps is feasible. Verification time for STARKs is significantly sub-linear in the computation size, typically polylogarithmic in T medium.com. In simpler terms, verifying a STARK proof is exponentially faster than naively re-executing the original computation. This scalability is crucial if we want to support many identity verifications in parallel or very complex attribute proofs (for instance, proving a machine learning model's output related to your data without revealing the data).
- **Succinct (Relatively Short Proofs):** STARK proofs are larger than SNARK proofs, but still much smaller than the raw data they attest to. A STARK proof might be on the order of tens to hundreds of kilobytes, depending on the statement complexity and security parameters. While this is larger than a SNARK proof (which might be under 1 KB), it is still compact enough to transmit and verify quickly. The proof size and verification complexity scale roughly polylogarithmically with the computation size, which for practical purposes means even huge statements yield proofs that a verifier can download and check in milliseconds to seconds.

- **Post-Quantum Security:** One of the standout advantages of STARKs is their resistance to quantum attacks. STARKs rely on cryptographic hash functions (like SHA-256, Keccak, etc.) and collision-resistant hashing, as well as information-theoretic soundness via Reed-Solomon code based proofs. These primitives are believed to be secure against quantum adversaries (a quantum computer offers at most a quadratic speedup against brute-force hashing via Grover's algorithm, which can be mitigated by increasing hash output size). They do not rely on the hardness of problems like discrete logarithm or factoring (which could be broken by Shor's algorithm on a sufficiently powerful quantum computer). Therefore, STARKs are considered **plausibly post-quantum secure**medium.com. In contrast, many common digital signature algorithms and SNARK systems could be broken if large-scale quantum computers become a reality. For an identity system that aims to stand the test of time (and protect identities potentially for a lifetime or beyond), post-quantum security is a critical consideration. DxID's use of STARKs helps "future-proof" the integrity of identity proofsthreeprotocol.ai.

Mathematical Background of STARKs: At a high level, a STARK works by converting a computational statement into a **polynomial verification problem**. The general process (which we later detail in implementation) is:

1. **Arithmetization:** The computation that needs to be proven (for example, "hash x with SHA-256 five times and get result y", or "given these credentials, they satisfy property P") is translated into an equivalent representation in terms of algebraic constraints, often a set of polynomial equations over a finite field. In STARKs, this usually takes the form of an **AIR (Algebraic Intermediate Representation)** or a set of **trace** and **constraint** polynomials. The program is executed to produce a **trace** of its state at each step, and then constraints (in the form of polynomial relations) ensure that trace is valid for the program logic.
2. **Commitment to Polynomials (Merkle Tree):** The prover computes these polynomial evaluations (the execution trace, which might be a large table of values) and then commits to them using a Merkle tree. Essentially, it places all the evaluation data into a Merkle tree and publishes the Merkle root to the verifier. This root serves as a cryptographic commitment to the entire computation trace – any tampering with the trace would change the root, so it's binding.
3. **Low Degree Proof (FRI protocol):** The prover needs to convince the verifier that the polynomials and constraints are satisfied and that certain functions are of low degree (a necessary condition if the computation was carried out correctly). STARKs employ an **interactive oracle proof** called FRI (Fast Reed-Solomon Interactive Oracle Proof of proximity) to prove that a given committed polynomial is of degree less than a certain bound. The prover and verifier engage in a protocol where the verifier challenges the prover to reveal certain random positions (using public random coins that both can compute via Fiat-Shamir) in the committed trace and related polynomials. Because the challenges are random and the polynomials should be low-degree if the computation is

valid, the prover cannot cheat on too many positions without detection. FRI ensures with high probability that the prover's data is consistent with a correct, low-degree polynomial (meaning the computation followed the rules). All interactions are made non-interactive via Fiat-Shamir, resulting in a single proof that includes the necessary Merkle branches and polynomial evaluations.

4. **Zero-Knowledge via Random Masking:** By default, a STARK as described is an argument of knowledge and can prove a computation's correctness. To achieve **zero-knowledge**, the prover must ensure that no sensitive information can be derived from the proof. This is typically done by randomizing certain trace values that are not meant to be revealed, or by adding random noise (blinding) to the trace in the last steps of the computation so that the specific values are concealed dev.risczero.com dev.risczero.com. The protocol is designed such that these randomizations do not affect the validity of the computation but hide the actual witness. In practice, if the statement to prove is something like "I know a secret that hashes to value H", the secret itself is never revealed; only commitments to it (in hash form) are part of the trace, and the trace is padded with randomness to avoid leaking information through multiple proofs. We ensure that DxID's STARK proofs implement full zero-knowledge, so that proving an identity attribute does not leak the attribute itself (aside from what is intentionally disclosed).
5. **Verification:** The verifier uses the proof to check a few things: (a) that the Merkle roots match and that all opened Merkle branches in the proof are consistent (ensuring data integrity of what prover committed), (b) that the challenged polynomial evaluations are consistent with the claimed low-degree (this is the core of verifying FRI steps), and (c) that all constraint equations hold for the sampled steps. If all checks pass, the verifier is convinced the computation was carried out correctly on some secret inputs the prover possesses, without needing to see those inputs. Verification is highly efficient – it involves hashing a number of times (for Merkle proofs and Fiat-Shamir challenges) and evaluating some polynomial formulas on the sampled points, typically on the order of a few thousand basic operations even for huge original computations. In blockchain terms, verifying a STARK could be more computationally intensive than verifying a SNARK, but it is still far, far less work than doing the original computation or brute-force checking the property.

To ground this in a simple example, consider proving a hash chain: "Given a secret value x , I applied SHA-256 ten times and the final output is y ." A STARK prover will create a trace of length 10 (or 11 including the initial state) with each step being the hash output at that round. The constraints ensure that each step's output = SHA-256 of previous step's output. The prover commits to all intermediate hash values via a Merkle tree. Then the FRI protocol is used to ensure that all steps followed the SHA-256 transition constraints. The final step's output is public (y), and initial step's input x can be kept hidden because the trace can be padded or combined with random dummy computations to not leak x – or x can be a private input whose knowledge

is being proven. The result is a proof that the prover knows an x such that after 10 hashes it equals y , without revealing x . This is analogous to how in an identity setting, we might prove “I have a credential that, when processed appropriately, yields a claim that I’m over 18” without showing the actual credential.

Comparison to SNARKs: It’s useful to summarize the differences between zk-SNARKs and zk-STARKs in the context of DxID’s requirements:

- *Trusted Setup:* SNARKs (like Groth16) require a setup; STARKs do not threeprotocol.ai. For an open identity system, avoiding a trusted setup removes a potential single point of failure. DxID opts for STARKs partly for this reason – we do not want the validity of identity proofs to rest on the honesty of a one-time ceremony. Everyone using DxID can verify proofs without having to trust anything beyond standard cryptography.
- *Performance:* SNARK proofs are extremely small and fast to verify (often under 10ms verification). STARK proofs are larger and take longer to verify (tens of ms to a few seconds depending on parameters). However, STARKs can handle more complex computations without exploding in prover time, and they parallelize well. In identity use cases, most proofs (like proving knowledge of a credential or an attribute comparison) are not extremely large computations, so either SNARKs or STARKs would be efficient enough. But if we consider future scenarios like proving machine learning model inferences or large database lookups under ZK for identity purposes, STARKs offer more headroom for scale. The trade-off is that on-chain verification of STARKs could be more costly in terms of gas compared to SNARKs due to larger proof size chain.link. DxID mitigates this by doing most verifications off-chain or in rollup-style batch verification where possible, and by leveraging optimized verifier contracts or validity rollups for on-chain cases.
- *Transparency & Auditability:* Because STARKs are transparent and based on simple assumptions, it’s easier for a wide community to audit and trust the system. SNARKs, while scientifically sound, introduce that external assumption of “someone destroyed the toxic waste”. By using STARKs, DxID aligns with a maximally trustless approach – any developer can independently check the proof system without needing to participate in or trust an external setup. As one source puts it, “STARKs can be easier to audit than SNARKs” since no external parameters are needed chain.link.
- *Post-Quantum:* As noted, SNARKs that use elliptic curves are not quantum-safe, whereas STARKs are. Given that identities may need to remain secure and fraud-proof for decades, this is a forward-looking choice. **Quantum resistance** is listed as a key feature of zk-STARKs threeprotocol.ai. In the words of a Three Protocol blockchain article, zk-STARKs use standard hash functions and are “more resilient to future advancements in quantum computing” threeprotocol.ai. For DxID, this means the identity proofs should stand firm even if attackers eventually possess quantum capabilities – an important assurance for governments or long-term archival credentials.

In summary, zk-STARKs provide the cryptographic backbone for DxID, enabling **provable integrity** of identity claims with **zero knowledge** and at a **scale** appropriate for a decentralized ecosystem. The next sections will show how these cryptographic tools are applied in the system architecture – how an identity is created, how credentials are issued and proved, and how content is signed and verified.

Before moving on, it's worth highlighting an insightful perspective from the original STARK research, which encapsulates why such proof systems are vital for society:

“Human dignity demands that personal information, like medical and forensic data, be hidden from the public. But veils of secrecy designed to preserve privacy may also be abused to cover up lies and deceit by institutions entrusted with data, unjustly harming citizens and eroding trust in central institutions. Zero-knowledge proof systems are an ingenious cryptographic solution to this tension between the ideals of personal privacy and institutional integrity.”[chain.link](#)

This quote underlines the very balance that DxID aims to achieve – empowering individual privacy while enabling verifiable integrity of claims and content. With the cryptographic foundations laid out, we proceed to the system architecture that makes use of these tools.

System Architecture

DxID's architecture is designed to realize a **self-sovereign, decentralized identity system** that incorporates zk-STARK proofs for privacy and integrity. The architecture covers the entire lifecycle of an identity: from creation and registration, through credential issuance and storage, to presentation and verification of identity proofs. It also includes a specialized protocol for content authentication, where digital content is signed and verified via decentralized identities.

We will break down the system architecture into the following sub-sections:

- **Identity Creation and DID Management:** How a new decentralized identity (DxID) is created, represented, and anchored on a blockchain.
- **Identity Attributes and Verifiable Credentials:** How identity attributes (like name, age, qualifications) are bound to the identity via credentials, and how these are kept off-chain for privacy.
- **Off-Chain Data Storage and Privacy:** The mechanisms by which sensitive identity data and credentials are stored off the blockchain, and how privacy is ensured.
- **Verification Process and ZK Proof Generation:** The workflow for proving an identity attribute or claim to a verifier, including zk-STARK proof generation and verification steps.

- **Content Authentication Protocol:** A detailed look at how DxID is used to sign digital content (such as documents or media) and how verifiers can authenticate the content's origin and integrity using the identity system.
- **System Components and Diagram:** A summary of the key components (user agent/wallet, issuer, verifier, blockchain, etc.) and their interactions, accompanied by a diagram illustrating the data flows in DxID.

Identity Creation and DID Management

Each participant in DxID (individual, organization, device, etc.) will create a **Decentralized Identifier (DID)** to serve as their identity's reference. This DID is a globally unique identifier that does not depend on any central issuing authority. In DxID, we follow the W3C DID standard format, which looks like:

vbnet

CopyEdit

`did:dxid:<unique-string>`

For example, an identity could be `did:dxid:123456789abcdef`. The `<unique-string>` could be, for instance, a self-generated public key fingerprint or a UUID; in our implementation we often derive it from a public key or a hash of an initial public key, ensuring it's unique and not guessable.

Key Generation: The user (or entity) generates a **public/private key pair** which will be used to control the DID. We support modern digital signature algorithms (initially ECDSA or EdDSA for practicality, and later to be augmented with post-quantum signature algorithms for long-term security). The private key remains with the user (in their wallet or secure enclave) and the public key (or a cryptographic digest of it) forms the basis of the DID document's verification method. For example, if using a public key directly as the identifier, a DID might internally incorporate a hash of the public key. In Three Protocol's approach, they mention users generate digital identities pseudonymized through SHA-256 hashes of their public keys threeprotocol.ai – we can adopt a similar approach so that the on-chain identifier is actually a hash, adding a layer of pseudonymity. Regardless of format, the DID is tied to a public key which is how others can verify signatures from this identity.

DID Document: In line with W3C standards, each DID can be resolved to a **DID Document**, which contains metadata about the identity: public keys (verification methods), authentication methods, and service endpoints if any. In DxID, the DID Document will typically include:

- The **public key(s)** for the identity (for example, an Ed25519 public key used for signing).
- The supported **verification methods** (e.g., indicating the type of signature or encryption this identity can do).
- Optionally, a reference to a **revocation registry** or list for credentials (if the identity has issued credentials that could be revoked).
- **Service endpoints** (optional): URLs or decentralized addresses where more information or services related to this DID can be found (for instance, an endpoint for obtaining the identity's public profile or for connecting via DIDComm). In DxDID, one interesting "service" could be a pointer to the identity's content-signing feed or storage for certified content, although content is typically published elsewhere with just signatures tied to the DID.

Blockchain Anchoring: The DID and a minimal DID Document (or a hash of it) are registered on a blockchain via a smart contract or ledger entry. For example, Ethereum can be used with a DxDID Registry smart contract that maps the DID to the DID Document data (or to an IPFS CID where the full DID Document is stored, to save space). In permissioned identity ledgers like Sovrin, the ledger stores DIDs and public keys; similarly, in DxDID we have a *DxDID Ledger/Registry*. This could be implemented on a general-purpose chain (like a smart contract on Ethereum, Polygon, etc.) or on a purpose-built identity ledger. In any case, the ledger provides **global resolution** of the DID: anyone who sees "did:dxdid:123...abc" can query the DxDID Registry and obtain the corresponding public key and metadata needed to interact with that identity. The design ensures that **anyone can participate** in the DID system (similar to how Veres One is a public ledger open to all w3c-ccg.github.io, DxDID's registry is public and permissionless).

Once registered, the identity is considered **self-sovereign** – only the identity owner (who holds the private key) can control it (e.g., update its associated public keys for rotation, or add service endpoints), and no central authority can arbitrarily revoke or alter the identity. Even the registration itself is usually either user-driven or done through a decentralized mechanism (in Sovrin, writing to the ledger required trustee nodes to agree; in DxDID, if it's on a public chain, it's open like uPort's Ethereum usage info.suresync.nl).

Example: Alice wants to create a DxDID. She uses the DxDID CLI or dApp to generate a new key pair. Suppose a hash of her public key is $H(\text{pubAlice}) = 0x\text{ABCD1234} \dots$. The system registers `did:dxdid:abcd1234...` on the blockchain by publishing a DID Document (or its reference) linking that DID to Alice's public key. Now Alice can prove control of that DID by signing a challenge with her private key. Others can resolve the DID to get her public key and verify such signatures. The registration might look like a transaction `registerDID(identifier, publicKey, docURI)` on the DxDIDRegistry contract. Once confirmed, Alice's identity is live.

Decentralization Consideration: Because DxID aims to be ledger-agnostic, it could integrate with different blockchains. For instance, if using **Ethereum** or another EVM chain, we have a smart contract. If using **Hyperledger Indy** style network, DIDs are written via transactions to that network. If using **Veres One** ledger, the DID method specification w3c-ccg.github.io is followed. The method “dxid” would be documented so that standard DID resolvers could fetch the DID Document. The main point is that the **verifiable data registry** (to use W3C’s term) for DxID is a blockchain ensuring global uniqueness and tamper-resistance of DIDs. This prevents impersonation (nobody can claim the same DID as Alice once it’s registered) and provides an immutable audit trail of identity updates.

Key Rotation and Recovery: The architecture includes provisions for key management. Over time, an identity might need to rotate its keys (e.g., if a key is compromised or simply to follow cryptographic agility best practices). The DID Document can be updated on-chain to reflect a new public key (possibly requiring the old and new key to cosign the update if the old key is still available, or using a social recovery mechanism if not). This ensures the identity remains persistent even if keys change. For example, `did:dxid:abcd1234...` might initially map to key1, and later be updated to key2. Verifiers would check the ledger for the latest valid key for that DID. All updates are signed by the identity’s keys (or a designated guardian key) and recorded on-chain.

Privacy of DID Creation: Note that simply creating a DID does not reveal any personal information – it’s just an identifier and a public key. DxID uses pseudonymous identifiers (random or hashed strings). There is no requirement to put one’s real name, etc., on chain. This means creating an identity does not equate to revealing your identity. Users can have multiple DIDs for different contexts to avoid correlation (DxID encourages pairwise unique DIDs or at least use-case specific DIDs, akin to how Sovrin and others allow an agent to use unique DIDs per relationship info.suresync.nl). For example, Alice might have one DID for financial interactions and another DID for social media, to prevent those contexts from being trivially linked via a common identifier. The system supports that by allowing unlimited DIDs per entity, though managing them is up to the user’s wallet software.

Identity Attributes and Verifiable Credentials

An identity by itself (just a DID and keys) is like an empty passport – it provides a way to authenticate the holder, but it doesn’t contain attributes. For DxID to be useful, we need to associate **identity attributes** or claims with the DID. This is done through **Verifiable Credentials (VCs)**. A Verifiable Credential is a tamper-evident credential, digitally signed by an **issuer**, that asserts certain information about a **subject** (the identity holder). For example, a government could issue a credential to Alice’s DID asserting her age and citizenship; a university could issue a degree credential; or Alice herself could issue a credential about her preferences to share selectively.

In DxID, credentials serve as the containers of personal data and claims, but crucially, the actual credentials are held by the user off-chain (often in their wallet app) and are not published globally. What is published (when needed) are proofs derived from those credentials.

Issuance of Credentials: A credential is issued by an authority (issuer) to the holder's DID. The process is:

- The issuer (identified by their own DID and having its own known public key) creates a credential data structure (for instance, following the W3C Verifiable Credential JSON-LD or JWT format) containing:
 - Credential ID (unique)
 - Issuer DID
 - Holder DID (subject)
 - The claims (e.g., “birthDate: 1990-01-01” or “education: BSc in Computer Science”)
 - Issuance date, expiration date (if any)
 - Any other metadata (credential type, schema, etc.)
- The issuer signs this credential with their private key, producing a digital signature. This signature ensures the credential cannot be altered without detection and that verifiers can confirm it indeed came from that issuer.
- The credential is then given to the holder (e.g., as a JSON file or stored in their identity wallet app). The holder might store encrypted backups of it in cloud or device, but it is under their control.
- Optionally, the issuer might record a **hash** of the credential on the blockchain or a reference for revocation checking. For example, they might register the credential's ID on a public revocation registry smart contract which can be toggled if the credential is later revoked (like how Hyperledger Indy/Sovrin handle revocation registry entries). This allows verifiers to check that a credential hasn't been revoked. However, even this can be done in a privacy-preserving way (only storing a hash or non-identifying index).

The important point is that **full credentials (with personal details) are not posted on-chain**. Only pointers or cryptographic commitments might be. This approach aligns with best practice from other SSI systems: Sovrin/Indy keeps claims off-ledger (on the user's device)info.suresync.nl; uPort also did not publish claim contents, only attestations on chaininfo.suresync.nl. DxID follows suit to achieve **data minimization** – the ledger is only for identifiers and proofs of integrity, not for personal data.

Selective Disclosure and ZK Proofs from Credentials: Traditional verifiable credentials can be presented to verifiers as a whole, but that would reveal everything in them. Instead, DxID employs zero-knowledge proofs to allow selective disclosure of credential data:

- We utilize cryptographic credential schemes that support selective disclosure (such as CL-signatures used in IBM Identity Mixer/Hyperledger Indy's AnonCreds, or BBS+ signatures which allow selectively revealing some attributes and not others). However, here we go further and leverage STARKs to prove statements about credentials.
- For each credential, the holder can generate a proof that a certain attribute in it satisfies a condition. For example, from a credential with birthDate, the holder can prove "birthYear $\leq$ 2005" to show they are at least 18 in 2023, without revealing the exact birthdate. This might be done by internally computing the birth year and comparing it to 2005 in a small arithmetic circuit, then proving knowledge of a signature on that birthdate that meets the condition, all encapsulated in a STARK proof.
- The credential's signature by the issuer can be verified within the ZK proof as well. This is a critical piece: the STARK can be used to show "I have a credential signed by Issuer X, and in that credential attribute Y has value Z" without revealing Z. The STARK program would take as inputs the credential data (encrypted or committed) and the issuer's public key, verify the issuer's signature (which can be done in circuits, though verifying modern signatures like EdDSA in a STARK might be heavy, we can optimize or do SNARK inside STARK if needed), and then check the attribute condition. The output of the program might simply be a boolean valid/invalid which is always "valid = true" for a well-formed credential meeting the condition. The STARK proof then convinces the verifier this is the case. This effectively provides the function of something like Idemix/AnonCreds ZK proofs info.suresync.nl but using our unified STARK approach.
- The result for the verifier is that they get a succinct proof, which when verified tells them that *some credential by a trusted issuer exists and contains the information needed*. The verifier might also require a proof of issuer's signature validity (which can be part of the ZK proof) or they might separately check a disclosed issuer signature if the holder is willing to disclose that much (for instance, sometimes revealing the issuer and credential type is okay, but not the specific attribute values).

Linking Identities and Credentials: In the credential, the subject is the holder's DID. However, when doing ZK proofs, we have to ensure that the prover is indeed the subject of the credential. If the credential is bound to a specific DID, then the prover should prove knowledge of the secret key of that DID as well (i.e., authenticate as the DID owner). In practice, this might mean the credential contains not just "Holder DID: did:dxid:abcd1234..." but also perhaps a public key or a commitment that ties it to the holder's key. If using CL-signatures or BBS+, often they support proving ownership by generating a proof of possession. In STARK context, we can simply combine a proof of identity control (like a signature with the user's DID private key on a

nonce) with the proof of credential content, inside a single proof or as two linked proofs. Typically:

- The verifier sends a challenge nonce to the holder's DID.
- The holder signs it with their DID private key to prove control (this could be a normal signature outside the ZK proof, as a standard authentication).
- Separately, the holder provides the ZK proof about the credential's content.
- Because the credential itself is about that DID, the verifier can match that the DID from the signature equals the DID in the credential (the DID might be revealed in plain if necessary, or the proof might carry that equivalently).
- Thus, the verifier knows the person proving the credential is indeed the rightful owner (and not someone who somehow obtained a copy of a credential file but isn't the subject).

Revocation: A complete system must handle credential revocation (e.g., an issuer revokes a license). The architecture includes a revocation registry smart contract where issuers can publish revocations (likely as a list of credential IDs or hash values). When a user generates a ZK proof from a credential, they should incorporate evidence that the credential is not revoked. This could be done by having the proof check a revocation status. However, checking a dynamic on-chain revocation list inside a ZK proof is non-trivial (it would mean the proof somehow had access to fresh blockchain data). A simpler approach is:

- The verifier, upon receiving a proof and the credential's identifier (which might be revealed in hashed form), can query the revocation registry off-chain. If the credential is revoked, they reject regardless of the proof.
- If we want to keep the credential ID hidden for privacy, it's more complex. Possibly a merkle tree of valid credentials with a non-revocation ZK proof could be done (Indy anoncreds have a mechanism for proving a credential isn't revoked by proving presence in a non-revoked accumulator). For now, we assume either credential IDs aren't super private (they might not directly identify the person), or the system will implement an accumulator-based revocation check that can be done in zero-knowledge. Such details might be addressed in a later version of DxiD.

Diagram – Credential Flow: The roles here are:

- *Issuer:* (e.g., Government, University, Company) – issues and signs credentials.

- *Holder*: (user controlling a DxD) – stores credentials and later presents proofs.
- *Verifier*: (service or entity needing to verify a claim) – checks proofs and possibly consults issuer's public key or revocation info.

We can illustrate a simple credential issuance and use case in a diagram:

Figure: High-level architecture of decentralized identity with issuers, holder (user), and verifier. The issuer signs a verifiable credential for the user's DID (step 1). The credential is stored in the user's wallet off-chain (step 2). When needed, the user generates a zero-knowledge proof of a claim from the credential (step 3) and presents it to a verifier, along with authentication of their DID (step 4). The verifier checks the proof and the issuer's signature or issuer's public key from the blockchain (step 5), without the user revealing the underlying personal data.info.suresync.nl/currency.medium.com

(The diagram above shows the key interactions: an Issuer provides a credential to a Holder (user), which the user stores privately. Later, the user uses a ZK Proof to convince a Verifier of something about that credential. Note that there is no direct contact between Issuer and Verifier during presentation, preserving privacy, as the verifier trusts the credential because it's signed by the issuer and proven by the holder.)[currency.medium.com](https://info.suresync.nl/currency.medium.com)

Off-Chain Data Storage and Privacy

DxD adheres to the principle that **personal data stays with the user**, not on the public ledger. All sensitive identity information and documents are stored off-chain, under the user's control, and only cryptographic commitments or proofs derived from them are shared. This sub-section describes how and where the data is stored, and how privacy is maintained even when interacting with verifiers or the blockchain.

User Wallet (Identity Agent): Each user will typically have an identity wallet application (on a smartphone, computer, or hardware wallet) which manages their DIDs, keys, and credentials. This wallet stores:

- The user's private keys for each DID (hopefully in secure storage, possibly hardware secure module or OS keystore).
- The user's collection of verifiable credentials (which can be encrypted at rest).
- Metadata like who issued each credential, when it expires, etc.
- A log of consents given or proofs shared (so the user can audit where they have used their data).

The wallet software can also act as an **agent** to automate interactions: for example, responding to a verifier's request by assembling the needed ZK proof. Sovrin's architecture introduced the concept of cloud agents and edge agents to assist the user info.suresync.nl. DxID can support similar patterns – e.g., a personal cloud agent that holds an encrypted backup of credentials and can respond to proof requests when the user approves via their device. But fundamentally, control remains with the user.

Encrypted Storage and Cloud Backup: The credentials and any identity documents in the wallet are encrypted by the user's own keys. For backup or portability, a user may store an encrypted backup on a cloud server or distributed storage (like IPFS or Storj, etc.), but only they hold the decryption key. Even if these encrypted blobs are stored in a decentralized storage network, it does not compromise privacy. For instance, a diploma credential could be stored as an encrypted JSON on IPFS; the IPFS content ID could even be put in the DID Document as a service endpoint (if the user chooses) to enable portability, but since it's encrypted, no one can read it without the key.

Peer-to-Peer Exchange: When an issuer issues a credential, it might deliver it to the user directly peer-to-peer (e.g., via an HTTPS API call, or via a DIDComm secure message). No public repository is needed. Similarly, when a user needs to present a credential or proof to a verifier, they communicate directly (e.g., present the proof via a web interface or a secure connection). The blockchain is only used for things that need global consensus – like verifying a DID's current public key or checking an issuer's public key and revocations. All actual data exchange can be off-chain and encrypted.

****Minimizing Correlation**

Introduction

Decentralized Identification (DxID) is a privacy-preserving digital identity framework that uses cutting-edge zero-knowledge cryptography to enable secure and verifiable identities without sacrificing user privacy. It leverages **zk-STARKs** (Zero-Knowledge Scalable Transparent Arguments of Knowledge) as the core proof mechanism to achieve trustless verification of identity attributes at scale. With DxID, individuals, organizations, and even IoT devices can establish decentralized identifiers (DIDs) and verifiable credentials that can be **proven** without disclosing sensitive underlying data. This technical document serves as a comprehensive specification, combining a whitepaper's depth, a system architecture description, and a developer-focused README. It is intended for developers, researchers, and stakeholders to understand and implement the DxID system.

Motivation: Today's digital identity solutions often rely on centralized authorities and vulnerable credential stores. Users are forced to overshare personal information to prove simple claims (such as age or citizenship), leading to privacy leaks and large-scale data breaches. Furthermore, emerging threats like deepfakes and misinformation erode trust in digital content, calling for robust methods to authenticate information provenance. DxID addresses these

challenges by putting users in control of their identity data and employing **zero-knowledge proofs (ZKPs)** to confirm identity assertions without revealing private information [chain.link](#)]. The use of zk-STARKs provides both **scalability** and **post-quantum security**, future-proofing the identity system against quantum adversaries [threeprotocol.ai](#) [threeprotocol.ai](#)].

Key Features of DxID:

- **Self-Sovereign Identity:** Users create and control their own decentralized identifiers without reliance on a central authority. Each identity is anchored in a blockchain-based registry, but personal data stays off-chain under user control.
- **Selective Disclosure:** Utilizing ZKPs, particularly zk-STARK proofs, users can prove claims about their attributes (age, qualifications, rights, etc.) without exposing the actual data [info.suresync.nl](#)]. This ensures compliance with principles like data minimization and privacy by design.
- **No Trusted Setup & Post-Quantum Security:** Unlike some earlier ZK systems, zk-STARKs require no trusted setup ceremony [threeprotocol.ai](#)]. They rely on collision-resistant hash functions and transparent public randomness, making DxID resilient against future quantum computing threats [threeprotocol.ai](#)].
- **High Scalability:** The architecture is designed to handle high volumes of identity proofs and verifications. zk-STARKs can support large transaction throughput due to their efficient proof generation and transparent verification, suitable for Internet-scale identity usage [threeprotocol.ai](#) [threeprotocol.ai](#)].
- **Content Authentication:** DxID includes a content signing protocol where any digital content (documents, images, videos, etc.) can be signed by an identity and accompanied by a proof of authenticity. This helps establish content provenance, which is crucial for combating misinformation. Every piece of content can be verified as untampered and genuinely originating from the claimed identity in real-time.
- **Interoperability:** The system aligns with emerging standards for decentralized identity (W3C Decentralized Identifiers and Verifiable Credentials). It is designed to integrate with or complement existing DID methods (like Sovrin, uPort, Veres One) while offering enhanced privacy and security features.
- **Smart Contract Integration:** DxID provides on-chain smart contract interfaces for managing identity records and verifying zk-STARK proofs of identity attributes on-chain. This enables decentralized applications (dApps) to easily consume verified identity information – for example, in DeFi compliance or supply chain audits – without ever seeing the raw private data.

This document is organized as follows: **Section 1** (Introduction) outlines the vision and features of DxID. **Section 2** covers the cryptographic foundations (zero-knowledge proofs, STARKs, and the mathematical background) that underpin DxID. **Section 3** details the system architecture, including how identities are created, how attributes and credentials are managed off-chain, and how the content authentication protocol works (with diagrams). **Section 4** explores full use cases across industries – from financial KYC to healthcare and education credentials, supply chain traceability, and misinformation prevention – illustrating how DxID is applied in each scenario. **Section 5** dives into the zk-STARK implementation details: how proofs are constructed and verified in DxID. **Section 6** describes the blockchain integration: the smart contract design, on-chain DID registry, and metadata storage. **Section 7** compares DxID to existing decentralized identity systems (Sovrin, uPort, Veres One, etc.) to highlight differences in trust model, privacy, and security. **Section 8** discusses the security benefits of DxID, with a focus on cryptographic strength and post-quantum resilience. **Section 9** examines integration potential with the Internet of Things (IoT) and artificial intelligence (AI), and how cross-border identities can be facilitated. **Section 10** outlines the vision for resisting misinformation by using DxID for content authenticity. **Section 11** is a developer-oriented README section, providing instructions on installing, building, and running a reference implementation (CLI and dApp) of DxID. Finally, **Section 12** covers licensing, and **Section 13** provides a list of references.

By combining theoretical foundations with practical engineering details, this document aims to serve as a definitive reference for the DxID project. **Developers** will find detailed specifications and example code to implement or integrate with DxID. **Security researchers** can review the cryptography and threat model. **Decision-makers and investors** can understand the technology's value proposition, backed by references to state-of-the-art research and industry standards.

Cryptographic Foundations

Decentralized Identification (DxID) builds upon advanced cryptographic techniques to ensure that identity information can be verified without revealing sensitive data. This section provides an overview of the cryptographic concepts underpinning DxID, notably **Zero-Knowledge Proofs (ZKPs)** and **zk-STARKs**, along with the mathematical background that makes these proofs possible. A solid understanding of these foundations is crucial for appreciating the security and privacy guarantees of the system.

Zero-Knowledge Proofs (ZKPs)

A *zero-knowledge proof* is a method by which one party (the *prover*) can convince another party (the *verifier*) that a certain statement is true, without revealing **why** it's true or any additional information beyond the validity of the statement. Formally, ZKPs satisfy three properties: **completeness** (if the statement is true and both parties follow the protocol, the verifier will be convinced), **soundness** (if the statement is false, no cheating prover can convince the verifier otherwise, except with negligible probability), and **zero-knowledge** (the verifier learns nothing

other than the fact that the statement is true). This ingenious concept was first introduced in the 1980s and has since become a pillar of modern cryptography.

In the context of digital identity, ZKPs allow a user to prove possession of a credential or a certain attribute **without disclosing the credential itself or any other attributes**. For example, a user can prove “I am over 18 years old” to a service, without revealing their exact birthdate or any other personal data. The service only learns the truth of the statement (that the user is an adult) and nothing else. ZKPs thereby enable **selective disclosure** and fine-grained privacy control, which are core to the self-sovereign identity philosophy.

There are different constructions of ZKPs, including interactive proofs (where the prover and verifier engage in a back-and-forth communication) and non-interactive proofs (where a single proof artifact suffices). Non-interactive proofs often leverage the Fiat-Shamir heuristic to remove interaction. In blockchain and decentralized systems, non-interactive zero-knowledge proofs are typically required, since on-chain or asynchronous environments cannot easily support interactive challenge-response rounds.

Two prominent families of ZKPs that have gained traction in recent years are **zk-SNARKs** and **zk-STARKs**. Both are non-interactive succinct proofs, meaning the proof can be verified much faster than re-executing the original computation, and the proof size is much smaller than the original data or computation being proved[\[chain.link\]](#). They allow one party to prove to another that a statement is true (e.g., “I possess a valid credential with property X”) without revealing the underlying secrets.

- **zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge):** zk-SNARKs were introduced around 2012 and were famously applied in the cryptocurrency Zcash to enable private transaction[\[chain.link\]](#). A zk-SNARK proof attests that “*I know a secret witness that satisfies this public statement*” with very high efficiency. However, classic zk-SNARK systems (such as Groth16) often require a **trusted setup** – a special ceremony to generate common reference parameters. If the secret “toxic waste” from that ceremony is not destroyed, it could be used to forge proofs undetectable[\[chain.linkchain.link\]](#). This reliance on a trusted setup is seen as a potential vulnerability and a hurdle for decentralization[\[chain.link\]](#). Additionally, most SNARK constructions rely on elliptic curve cryptography and pairings, which are not post-quantum secure. Despite these drawbacks, SNARKs produce very small proofs (often just a few hundred bytes) and have extremely fast verification times, making them practical for on-chain verification in today’s environments.
- **zk-STARKs (Zero-Knowledge Scalable Transparent Arguments of Knowledge):** zk-STARKs are a newer generation of ZKPs, introduced in a 2018 paper by Eli Ben-Sasson et al[\[chain.link\]](#). The acronym emphasizes key properties: *Scalable* (supporting proofs of very large computations efficiently), *Transparent* (no trusted setup needed), and *Argument of Knowledge* (a proof of knowledge that is sound under computational assumptions). STARKs use only basic cryptographic primitives (hash functions and information-theoretic security) rather than heavy number-theoretic

assumptions. They employ techniques from polynomial commitments and error-correcting codes to prove statements about computations in a way that is highly scalable. We will delve deeper into STARKs shortly, as they form the crux of DxID's trust model.

Why Zero-Knowledge for Identity? Traditional digital identity verification involves sharing data – for example, sending a copy of your ID or certificate to a verifier, who then checks its validity. This model inherently leaks information and often requires trusting the verifier to handle your data carefully. With ZKPs, a verifier doesn't need to see or store your personal data at all; they only see a proof that certain conditions are satisfied. As a result, ZKPs **dramatically reduce the exposure of personal data**, mitigating the risk of data breaches and surveillance. In the DxID system, zero-knowledge proofs are used for proving control of an identity (authentication) and proving attributes or claims about that identity (authorization) without revealing the actual attributes.

For example, consider that in DxID, a university could issue a cryptographic credential to a student that includes their degree and graduation year. Later, the student can prove to an employer “*I have a Bachelor's degree in Computer Science from University Y*” by generating a ZKP from that credential. The proof would convince the employer of the truth of the statement, but the employer would not see the actual certificate, transcripts, or even know exactly when the student graduated – only that a trusted issuer (University Y) vouches for the degree. This approach enforces **data minimization**, as practiced by other self-sovereign identity systems. For instance, Sovrin (Hyperledger Indy) uses zero-knowledge proof technology to enable selective disclosure of claims, keeping the actual claims off the blockchain on the user's device info.suresync.nl]. DxID adopts a similar philosophy, generalizing it with zk-STARKs for even greater flexibility in what can be proved.

zk-STARKs: Scalable Transparent Arguments of Knowledge

Definition and Properties: zk-STARK stands for *Zero-Knowledge Scalable Transparent Argument of Knowledge*. STARKs are a type of zero-knowledge proof system focused on scalability and transparency. As mentioned, they were introduced as an alternative to SNARKs to eliminate the need for a trusted setup and to leverage only symmetric cryptography [chain.link](#)]. A STARK proof demonstrates that a certain computation or statement is valid, with the following hallmarks:

- **Transparent (No Trusted Setup):** STARKs do not require a trusted setup ceremony or secret trapdoor. The public parameters used in proof generation and verification are untainted by secret randomness – typically they consist of fixed values like a prime field definition and public hash functions. This enhances security because there is no “master key” that could compromise the system threeprotocol.ai]. In contrast, zk-SNARK systems often have to trust that some initial parameters were generated honestly and then destroyed. By being *transparent*, STARKs align perfectly with decentralized identity's trust assumptions: users and verifiers do not have to trust any central party or setup

phase in order to trust the proofs.

- **Scalable and Fast to Generate Proofs:** STARKs are designed to handle large computations efficiently. The proving time scales quasi-linearly with the size of the computation trace, and verifying a STARK proof is much faster than performing the original computation. Concretely, if a computation takes T steps, producing a STARK proof might take on the order of $T \cdot \text{polylog}(T)$ operations, and verification might take on the order of $\text{polylog}(T)$ operationmedium.com]. In other words, generating proofs is only modestly more expensive than the computation itself, and verifying proofs is exponentially faster than re-running the computation. Proof sizes and verification times grow sub-linearly with T . This property is crucial for DxID because it means even complex identity checks (or multiple batched verifications) can be proven and verified quickly, enabling the system to scale to many users and claims. High throughput identity verification is possible – for example, validating thousands of credentials in a batch via one STARK proof – which is attractive for scenarios like large-scale credential audits or KYC compliance checks that need to be both fast and private.
- **Succinct Proofs:** STARK proofs, while larger than typical SNARK proofs, are still much smaller than the data they prove and can be transmitted efficiently. A STARK proof might be tens to a few hundred kilobytes, depending on parameters and statement complexity. This is considered *succinct* for most real-world uses because, for instance, it allows proving the correctness of megabytes of data or computation with only a few hundred KB of proof. Verification of a STARK proof involves checking a small number of hash computations and polynomial evaluations, making it feasible on-chain in many cases (though more expensive than a SNARK verification). The trade-off is that SNARKs have tiny proofs (e.g., <1 KB) and very fast verification, while STARKs have larger proofs and slightly slower verification. However, STARKs avoid the downsides of SNARKs (trusted setup, heavy crypto assumptions). In practice, DxID leverages STARKs for the most privacy-critical operations and is designed to accommodate the proof sizes (for example, by doing verification off-chain or in layer-2 environments if necessary). According to one comparison, verifying STARKs can be more gas-intensive than SNARKs due to larger proof size, but they offer easier auditability and no trust ceremonychain.link].
- **Post-Quantum Security:** STARKs are built on collision-resistant hashing and information-theoretic proofs (like Reed–Solomon code based low-degree proofs), which are believed to be secure against quantum attacks. They do not rely on classical public-key cryptographic hardness (no discrete log, no RSA, no elliptic curve pairings). This means that even if large quantum computers become available, STARK proofs would remain sound (provided the hash functions used are resistant to quantum attacks, which only requires doubling the output length to counter Grover's algorithm). In contrast, many current digital identity systems use elliptic curve signatures (ECDSA, EdDSA) and potentially SNARKs, both of which could be broken by quantum computers. By using STARKs, DxID is **future-proofing** the identity verification process. As noted in one source, zk-STARKs leverage standard hash functions, making them more resilient to

future quantum computing threatthreeprotocol.ai]. This is a critical advantage for an identity system intended to have longevity and to be used in government or cross-border scenarios where long-term security is paramount.

Mathematical Background of STARKs: At a high level, a STARK works by converting a computational statement into a **polynomial verification problem** and then using an interactive proof protocol (made non-interactive via Fiat–Shamir) to verify that polynomial conditions hold.

- *Arithmetization:* The first step is to express the statement or computation in algebraic form. For example, suppose we want to prove “Given input data x , a program P computes output y .” The prover will execute program P with x to get y and also produce a trace of the execution (the state of the program at each step). This execution trace can be represented as a table of values, which we then interpret as evaluations of certain polynomials over a finite field. Constraints of the program (like correct state transitions) are encoded as polynomial equations that the trace values must satisfy. This transforms the problem of “ $P(x)=y$ ” into a problem of proving that a set of polynomials related to the trace have low degree and satisfy certain relationships. This approach is common to all polynomial-interactive proof systems, including SNARKs and STARKs.
- *Commitment via Merkle Trees:* The prover commits to the full set of polynomial evaluations (the execution trace and any auxiliary polynomial evaluations) by arranging them in Merkle trees and providing the Merkle root to the verifier. This large data commitment binds the prover to a specific computation without sending everything to the verifier. Merkle commitments are cryptographically secure and succinct to open at arbitrary positions. The verifier will later ask the prover to open (reveal) a small number of randomly chosen positions in these Merkle trees to spot-check the correctness.
- *Interactive Random Queries (FRI):* Using an interactive protocol called FRI (Fast Reed-Solomon Interactive Oracle Proof) or similar, the prover and verifier engage in a protocol where the verifier (or, in non-interactive mode, the Fiat–Shamir heuristic simulating the verifier) challenges the prover to demonstrate that the committed polynomials are all of degree below a certain bound and satisfy the constraints. The prover responds with values from the trace at certain indices and corresponding Merkle proofs. The verifier’s random challenges ensure that if the prover’s data were invalid (e.g., if the computation trace does not follow the program’s rules or the polynomial is not low-degree), there is a negligible probability that it would still pass all the random checks. Essentially, because a wrong computation would force the prover to cheat on a polynomial that can’t simultaneously satisfy low-degree conditions at all points, the random sampling will catch the inconsistency with high probabilitychain.linkchain.link].
- *Zero-Knowledge via Masking:* A STARK as described so far ensures computational integrity (soundness), but not zero-knowledge, because the verifier could potentially learn some intermediate values from the openings. To achieve zero-knowledge, the prover randomly blinds portions of the trace that correspond to private data. The protocol

is constructed such that these random masks do not affect the validity of the computation or the satisfaction of constraints (they typically add extra dummy steps or use random addends that cancel out in constraint equations). For example, before committing, the prover can extend the trace with a few extra steps filled with random values (noise dev.risczero.comdev.risczero.com]. The constraints are set up so those extra steps don't violate anything, but they hide the true end of the real trace. This way, any specific witness values are computationally hidden among random values from the verifier's perspective, achieving the zero-knowledge property.

- *Verification:* The verifier, after receiving the proof, will:
 1. Derive the same random challenge values (via hashing the proof transcript if non-interactive).
 2. Check the Merkle roots and opened Merkle branches to ensure consistency with committed data.
 3. Evaluate the provided polynomial values and check that they satisfy the low-degree checks (FRI proofs) and the computation constraints at the sampled points.
 4. Accept the proof if all checks pass. The probability that a false statement passes these checks can be made astronomically low (e.g., $2^{(-80)}$ or smaller) by appropriate parameter choices.

To illustrate with a simpler analogy: proving knowledge of a secret number that hashes to a given value. A STARK would represent “ $y = \text{Hash}(x)$ ” as a small computation, commit to x and y , and then the proof would essentially convince the verifier that the committed x when put through a known hash function yields y , without revealing x . The STARK ensures the hash computation constraints are satisfied through polynomial relations and random spot-checks. This is like a more complex form of the classical ZK proof of preimage, but STARK allows proving even very large computations in this way (like verifying a signature or running a program on private data) without revealing the data or the intermediate steps.

Comparison to SNARKs: It's useful to summarize the differences between zk-SNARKs and zk-STARKs in the context of DxID's requirements:

- *Trusted Setup:* zk-SNARKs (in many implementations) require a trusted setup; zk-STARKs do not threeprotocol.ai]. For an open identity system, avoiding a trusted setup removes a potential single point of failure or trust. DxID opts for STARKs partly for this reason – we do not want the validity of identity proofs to depend on the honesty of a one-time ceremony. Everyone using DxID can verify proofs without having to trust any secret parameters.

- *Cryptographic Assumptions:* SNARKs rely on algebraic assumptions (elliptic curve discrete log, pairing product equations), whereas STARKs rely on hash functions and coding theory. The latter are more conservative assumptions and widely believed to be secure (and as mentioned, more robust against quantum attacks). This makes STARKs more appealing for an identity system that may be used in the long term and needs to be immune to future advances in cryptanalysis.
- *Performance:* SNARK proofs are much smaller and quicker to verify than STARKs – which is a big reason SNARKs remain popular for on-chain verification. However, SNARK provers can be slower for large computations, whereas STARKs handle large computations more gracefully. In the identity context, most single proof statements (like proving one credential's property) are small enough that either system would be fast. But if we consider aggregating many proofs or doing more elaborate verifications (imagine proving a complex policy involving multiple credentials or proving properties of a dataset), STARKs' scalability becomes a significant advantage. Additionally, STARKs can be parallelized and implemented in GPU/FPGA to further accelerate proof generation. Verification of STARKs, while heavier, can often still be done in milliseconds to a few hundred milliseconds, which is acceptable for off-chain processes and even on-chain in some scalable chains. Notably, one source points out that while STARKs have larger proof sizes and can be more gas-intensive to verify on-chain, they avoid the “Achilles heel” of a trusted setup and are easier to audit [chain.linkchain.link](#)].
- *Auditability and Transparency:* Because STARKs are transparent and their security reduces to well-understood primitives (hashes and randomness), it's easier for a broad community to audit the system or implement it from scratch without relying on obscure parameters. This suits the open nature of a decentralized identity system: multiple vendors could implement the DxID proof verifier from the spec without needing any secret info. SNARKs, on the other hand, require consistent parameters and careful coordination if different parties implement them (to ensure they use the same trusted setup output).

In summary, zk-STARKs provide the cryptographic backbone for DxID, enabling **provable integrity** of identity claims with **zero knowledge** and at a **scale** appropriate for a decentralized ecosystem. As the inventors of STARKs noted, this technology offers a solution to the tension between privacy and trust: institutions can be forced to prove truthfulness without citizens having to surrender privacy [chain.link](#)]. With the cryptographic foundations laid out, we now proceed to the system architecture that makes use of these tools.

System Architecture

DxID's architecture is designed to realize a **self-sovereign, decentralized identity system** that incorporates zk-STARK proofs for privacy and integrity. The architecture covers the entire lifecycle of an identity: from creation and registration, through credential issuance and storage,

to presentation and verification of identity proofs. It also includes a specialized protocol for content authentication, where digital content is signed and verified via decentralized identities.

We will break down the system architecture into the following sub-sections:

- **Identity Creation and DID Management:** How a new decentralized identity is created, represented, and anchored on a blockchain.
- **Identity Attributes and Verifiable Credentials:** How identity attributes (like name, age, qualifications) are bound to the identity via credentials, and how these credentials are managed.
- **Off-Chain Data Storage and Privacy:** How sensitive identity data and credentials are stored off the blockchain, and how user privacy is preserved.
- **Verification Process and ZK Proof Generation:** The workflow for proving an identity attribute or claim to a verifier, including zk-STARK proof generation and verification steps.
- **Content Authentication Protocol:** How DxID is used to sign digital content and how verifiers can authenticate content's origin and integrity using the identity system.
- **System Components Overview:** The key components (identity wallet, issuers, verifiers, blockchain ledger, etc.) and their interactions in DxID.

Identity Creation and DID Management

Each participant in DxID (individual, organization, device, etc.) begins by creating a **Decentralized Identifier (DID)** to serve as their identity's reference. This DID is a globally unique identifier that does not depend on any central issuing authority. In DxID, we follow the W3C DID standard format, which looks like:

vbnet

CopyEdit

```
did:dxid:<unique-string>
```

For example, an identity could be represented as `did:dxid:123456789abcdef`. The `<unique-string>` could be, for instance, a self-generated public key fingerprint or a UUID; in our implementation we often derive it from a public key or a hash of a public key, ensuring it's unique and cryptographically bound to the user.

Key Generation: The user (or entity) generates a **public/private key pair** which will be used to control the DID. The private key is kept secret by the user (in their wallet or a secure enclave), and the public key (or a cryptographic digest of it) is associated with the DID. The security of the identity rests on this key pair: possession of the private key equates to control of the identity. For strong security, DxID can use modern elliptic-curve keys (such as Ed25519) and will evolve to support post-quantum signature schemes (like Dilithium) as they mature, in order to maintain the post-quantum integrity of the system end-to-end.

DID Document: In line with the W3C DID specification, each DID corresponds to a **DID Document** – a small JSON (or JSON-LD) document that contains metadata needed to use the DID:

- The **public key(s)** or verification methods associated with the DID (for example, an EdDSA key with its type and encoding, used for authenticating as this DID).
- The **authentication** section, indicating which keys can be used to prove control of the DID.
- **Assertion method** or **capability invocation** entries, which might list keys used for signing credentials or interacting with smart contracts.
- **Service endpoints** (optional): URLs or other identifiers for services related to this DID (such as a messaging service endpoint, profile data endpoint, or credential repository). In DxID, we might use service endpoints to point to off-chain storage for public profile info or to facilitate peer-to-peer connection establishment.
- Other metadata like timestamps or key rotation history if needed.

Blockchain Anchoring (DID Registry): The DID and its DID Document (or a hash/reference to it) are recorded on a blockchain or distributed ledger. This provides a globally accessible, tamper-evident record of the identity. In an implementation on Ethereum (as an example), we could have a **DxID Registry smart contract** where a user can register their DID by submitting the DID unique string and their public key (or the whole DID Document). The contract would store a mapping from DID to the latest DID Document data (or perhaps store an IPFS content hash if the document is large or uses JSON-LD contexts). On a purpose-built identity ledger (like Sovrin or Veres One), the DID and document would be written via that network's transactions. Veres One, for instance, is a public blockchain designed specifically for DIDs, where DIDs like `did:v1:<id>` can be created by anyone and updated on-chain [[w3c-ccg.github.io](https://github.com/w3c-ccg)]; similarly, `did:dxid` could be its own method on an Ethereum sidechain or any decentralized network optimized for identity. What's critical is that **any party can resolve a DxID DID to a DID Document via the blockchain**, ensuring they get the correct public keys to verify signatures from that identity.

Once registered, the identity is considered **self-sovereign** – only the identity owner (who holds the private key) can control it (e.g., update its associated keys or profile), and no central authority can revoke or impersonate that identity. The blockchain acts as a neutral ground that all verifiers and issuers can trust to fetch identities' public keys and revocation status.

Example Workflow – Registering a DID: Alice wants to create a DxD for herself:

1. Alice's DxD wallet generates a key pair (say `K_alice_private` / `K_alice_public`).
2. The wallet computes `DID = did:dxd: + <fingerprint of K_alice_public>` (for uniqueness).
3. Alice's wallet submits a transaction to the DxD Registry contract with her DID and her public key (and any additional DID Document info or a pointer to it). This might be a function call like `registerDID(did, publicKey, documentURI)`.
4. The transaction is confirmed on the blockchain. Now `did:dxd:xxxx...` is associated with Alice's public key on the ledger.
5. Whenever someone needs Alice's public key (to verify a signature from her, for example), they resolve `did:dxd:xxxx...` either by querying the blockchain or through a DID resolution infrastructure that knows how to query the DxD method. They receive the DID Document and extract `K_alice_public` from it.
6. Alice can now prove control of that DID by signing any challenge with `K_alice_private`. Verifiers will use `K_alice_public` (fetched via the DID Document) to check her signature and thereby authenticate her as the owner of the DID.

Key Update and Recovery: Over time, Alice might need to rotate her keys (if a key is compromised or as a routine security practice). DxD supports key rotation by allowing the DID Document on-chain to be updated. For example, Alice issues a DID update transaction that replaces the old public key with a new one (signed by her old key and/or using a recovery mechanism if the old key is lost). The ledger will then reflect the new public key from a certain timestamp/block onward. Entities resolving the DID will see the new key. Optionally, the system can maintain previous keys or a trust trail (to allow verification of signatures made in the past with old keys, if needed, or to support recovery processes involving trusted contacts or multi-sig as is common in SSI systems). Recovery policies (like social recovery or using a fiduciary DID) could be encoded either in the DID Document or handled at the application level if Alice loses her keys.

Privacy of DIDs: Simply creating a DID does not reveal personal information – it's just a random identifier plus a public key. In DxD, we encourage **pairwise unique DIDs** or at least context-specific DIDs to avoid correlation. This means Alice might use different DIDs for

different purposes so that observers cannot link all her activities to one identity. (For example, she could have `did:dxid:aliceFinance` for banking and `did:dxid:aliceSocial` for social media.) The wallet can manage multiple DIDs under the hood. This approach was also used by UPort, which created a new Ethereum address for each application to reduce linkability info.suresync.nl]. The downside can be complexity or needing to manage multiple keys, but privacy-conscious users and applications may find it worthwhile. DxID's protocol does not force correlation: verifiable credentials can be issued to different DIDs for the same person, and later combined in proofs if the person wants to prove something that involves multiple personas (though that's optional).

In summary, identity creation in DxID results in a **DID on a blockchain** linked to a **public key** (and associated metadata), controlled solely by the user's private key. This forms the root of trust for everything else: all credentials and proofs will reference this DID, and all signatures will be verifiable via this DID's public key.

Identity Attributes and Verifiable Credentials

An identity (DID) by itself is an empty container – it doesn't reveal *who* you are or *what* qualifications you have. In DxID, all personal attributes and claims (age, name, qualifications, affiliations, etc.) are conveyed through **Verifiable Credentials (VCs)**. A Verifiable Credential is a tamper-evident, digitally signed statement issued by some authority about a subject. The **subject** of a credential is typically identified by a DID (for self-sovereign identity scenarios), and the **issuer** is identified by their DID (or some trusted identifier). The credential contains a set of **claims** (attributes) and is digitally signed by the issuer to ensure authenticity and integrity.

Credential Structure: We adhere to the W3C Verifiable Credentials Data Model. In simplified terms, a credential might look like:

json

CopyEdit

```
{
  "id": "urn:uuid:123...456",
  "issuer": "did:dxid:GovAgency",
  "issued": "2025-01-15T00:00:00Z",
  "subject": "did:dxid:alice123...",
  "credentialType": ["ProofOfAgeCredential", "GovernmentID"],
  "claims": {
```

```

    "name": "Alice Smith",

    "birthDate": "2005-07-20",

    "nationality": "US"
  },

  "proof": {

    "type": "Ed25519Signature2018",

    "created": "2025-01-15T00:00:00Z",

    "verificationMethod": "did:dxid:GovAgency#key1",

    "signature": " base64-encoded-signature "

  }

}

```

In this example, a government agency (issuer) issues a credential to Alice's DID attesting her name, birth date, and nationality. The **proof** block is the issuer's digital signature over the credential content (usually a JSON-LD canonicalized and hashed). The verifier of this credential would need the issuer's public key (obtained via the issuer's DID Document) to check that signature.

Issuing Credentials: In practice, the issuance flow is:

1. **Issuer Preparation:** The issuer (e.g., Government) has its own DID and keys (e.g., `did:dxid:GovAgency`). This DID is known to verifiers as a trusted issuer (for example, verifiers might have a list of issuer DIDs they trust for certain credential types).
2. **Holder Request:** The holder (Alice) might request a credential from the issuer (e.g., apply for an ID). She provides her DID (`did:dxid:alice123...`) to the issuer as the subject identifier.
3. **Credential Creation:** The issuer creates the credential with the necessary claims about Alice. Importantly, the credential will reference Alice's DID as the subject, binding it to her identity.

4. **Issuer Signature:** The issuer digitally signs the credential. This could be done using a standard like JSON Web Token (JWT) or Linked Data Proofs. The signature ensures that the credential cannot be altered and that it indeed comes from the issuer.
5. **Delivery:** The credential is delivered to Alice. This could be via a secure API endpoint, a download link, or a direct peer-to-peer connection (like DIDComm messaging). The credential is often serialized as JSON or JWT which Alice's wallet can store. In systems like Sovrin/Indy, credentials are often handed off-line (or through an agent) and stored in the user's wallet app.

The **credential is not broadcast on the blockchain**. It's a private document held by Alice. Only Alice (or someone she presents it to) will see the content of the credential. This is a crucial privacy aspect: even though the credential is signed by an issuer, it lives with the user, not in a public repository.

Storing Credentials Off-Chain: Alice's wallet stores the credential securely (likely encrypted). The blockchain might only be involved to store a *schema* or *public key* related to the credential:

- *Credential Schema:* Sometimes an issuer will publish a schema or definition of the credential type (e.g., what fields "ProofOfAgeCredential" contains) on a ledger or public repository so that verifiers know how to interpret it.
- *Issuer Public Key:* The issuer's DID Document on-chain provides the key to verify the credential's signature.
- *Revocation Registry:* If the credential can be revoked, the issuer might maintain a revocation list on-chain (for instance, listing credential IDs or some revocation handle that can be marked revoked). In Hyperledger Indy, a cryptographic accumulator on the ledger allows checking revocation status. In DxDID, we can implement something similar: e.g., the issuer posts a hash of the credential to a revocation smart contract when issuing, and can later toggle a flag if revoked. This allows verifiers to check revocation without revealing the credential itself (they just check the status of its hash).

Importantly, the actual **claims (name, birthDate, etc.) are *not on-chain*** – they are only in the credential that Alice holds. This design follows privacy best practices, as seen in Sovrin (no personal data on ledgerinfo.suresync.nl) and uPort (claims stored off-chain, only attestations on-chaininfo.suresync.nl).

Selective Disclosure and ZK Proofs from Credentials: When Alice wants to prove something from her credential to a verifier, she does not simply hand over the whole credential (that would reveal all information). Instead, DxDID enables her to produce a **Verifiable Presentation** – a derived proof from the credential, possibly using zero-knowledge techniques:

- The simplest form is to just reveal certain fields. For example, Alice can present a VC and redact (not include) the **name** and **nationality** fields, only showing the **birthDate**. If the credential and signature format supports selective disclosure (e.g., BBS+ signatures or CL-signatures), this can be done in a way that the issuer's signature still verifies for the disclosed parts. However, a typical digital signature (like a JWT or basic LD Proof) can't be partially verified if you remove fields, so advanced cryptography is needed.
- This is where **zk-STARK proofs** become powerful. Alice's wallet can generate a zk-STARK proof of a predicate about the credential. For instance, to prove she is over 18, her wallet doesn't reveal her exact birthDate. Instead, it constructs a small program: *take birthDate from credential, compute age = today's date - birthDate, check if age >= 18*. It then proves *knowledge of a credential signed by Issuer, containing a birthDate that satisfies the condition*. The STARK proof would show:
 - It knows a set of data matching the credential schema.
 - That data includes a birth date which, when compared to the current date, implies age ≥ 18 .
 - The data and the issuer's public key yield a valid signature (meaning the issuer did sign that exact data).
 - The proof would not reveal the birth date itself, only the fact that the check passed (and perhaps some public info like the issuer's identity and the credential schema).
- The verifier receives this proof and verifies it. The verifier is convinced that a trusted issuer (e.g., the government DID) issued a credential to Alice that confirms she's over 18, without ever seeing her birth date or name curity.medium.com. The verifier also doesn't learn anything else from that credential (like nationality or ID number).
- As a result, Alice preserves privacy (only the needed fact is revealed), and the verifier gets exactly the assurance needed (and nothing more). This is privacy by design: sharing the *minimum* information required. As noted in an industry article, the user can share only a subset of identity data or even use zero-knowledge proofs to prove some information without revealing any underlying data curity.medium.com.

From a *verifier's perspective*, this not only protects user privacy but also reduces their own liability. They don't receive unnecessary personal data, so they don't have to worry about storing it or complying with data protection for that extra data curity.medium.com. For example, if a bar just gets a proof "Guest is 21+ (true)", it doesn't have to handle the person's name or exact

birthdate, freeing it from retaining or protecting that personal info (which is a benefit under regulations like GDPR).

Roles and Trust: In the credential ecosystem:

- **Issuer:** Trust in the credential comes from trusting the issuer. If a verifier trusts Government Agency's DID to issue age credentials, then a proof that "GovAgency says Alice is over 18" is as good as seeing her ID, from a trust standpoint. Therefore, establishing and managing trust in issuer DIDs (through governance frameworks, reputation, or a web-of-trust) is important but outside the technical scope of DxID (governance is usually handled by legal frameworks or trust registries).
- **Holder (Subject):** The holder's job is to store credentials and present proofs. The system ensures the holder cannot tamper with credentials (signatures prevent that) and cannot forge proofs without the actual credential. The holder also cannot meaningfully alter the age check, for instance, because the proof construction itself is verified – if she tried to present someone else's credential, she wouldn't be able to sign the presentation as that person, etc.
- **Verifier:** The verifier must check three things when receiving a proof/presentation: (1) The proof's cryptographic validity (e.g., STARK proof verifies, or disclosed signature verifies). (2) The issuer's signature or proof within it (ensuring it indeed comes from a trusted issuer). (3) The credential is not revoked (if applicable). All of these can be done without contacting the issuer directly. In decentralized identity, typically **verifier and issuer are decoupled** – the verifier doesn't need to ask the issuer each time, they rely on the credential and proof itself. This is a crucial privacy advantage: *"there is no direct link between the verifier and the issuer, only a trust relationship,"* meaning the issuer does not learn where or when the holder is using the credential [curia.medium.com](https://medium.com/curia/curia-identity-protocol-101-4d4d4d4d4d4d)].

Revocation Handling: Suppose the issuer revokes Alice's ID (maybe her ID expires or is reported lost). If the credential includes a revocation mechanism, Alice's future proofs should indicate whether the credential is still valid. There are a few methods:

- **Online check:** The verifier can check an online revocation registry (e.g., a blockchain list) for the credential's ID or revocation nonce. This might slightly compromise privacy if done naïvely (because it could allow correlation of where that credential ID is being checked). Techniques like using accumulators and proving non-revocation in zero-knowledge can avoid revealing which credential is being checked while still proving "this credential I'm using is not revoked as of latest ledger state." DxID could incorporate such a scheme, but initially, we might assume straightforward checks with the understanding that credential IDs themselves might be abstract or not easily linkable to the user.

- **Short-lived credentials:** Alternatively, credentials might be short-lived and frequently re-issued or refreshed to implicitly handle revocation. But that is less user-friendly.

Given the scope, we will assume that for each credential type, verifiers know how to check its revocation status (either via a direct registry lookup or via an embedded proof provided by the holder).

In summary, **verifiable credentials** empower users to carry certified statements about themselves, and **zero-knowledge proofs** allow them to reveal only what's necessary from those statements. DxID integrates these concepts so that, for example, a financial dApp can enforce “each user must have a government-issued ID and be over 18” by requesting a ZK proof from the user, rather than collecting a copy of their actual ID. The system thus significantly enhances privacy and security for all parties.

Off-Chain Data Storage and Privacy

DxID adheres to the principle that **personal data remains off-chain under user control**. Only minimal data (like DIDs, public keys, and perhaps hashed references) are recorded on-chain. This design minimizes the exposure of sensitive information on a public ledger and gives users full control over who gets to see their data. Let's outline how data is stored and accessed:

User Wallet (Identity Agent): Each user has an identity wallet (which could be an application on their phone or computer, or a hardware wallet device). This wallet is the cornerstone of user control and privacy. It typically performs the following functions:

- Stores the user's **private keys** for their DIDs securely.
- Stores **verifiable credentials** issued to the user, in encrypted form.
- Manages connections and communications with issuers and verifiers (possibly via DIDComm messages or similar secure channels).
- Creates presentations and ZK proofs as needed, on demand, with user consent.
- Optionally, provides a user interface for the user to approve or deny requests, and to view what information is being shared.

All personal data like names, addresses, photos, or certificate details are stored in the wallet's secure storage. Modern wallets may use the device's secure enclave or encryption with a passphrase to protect this data at rest.

Encrypted Backup & Cloud Storage: Users may opt to back up their encrypted wallet data (including credentials) to cloud storage or a personal server. Because the data is encrypted by

keys only the user knows, this does not inherently trust the cloud provider. Even if an attacker obtains the encrypted blob, they cannot read the credentials without the user's key. For example, Alice's wallet might encrypt all her credentials with a symmetric key derived from her passphrase and upload the ciphertext to a decentralized storage network (like IPFS or a cloud like Dropbox). In case she loses her device, she can recover her identity by re-downloading the backup and decrypting it with her passphrase (assuming her DID keys were also backed up or can be recovered via a seed phrase or social guardians). The key point is that any off-device storage of personal data is **encrypted** end-to-end.

Communication Channels: When an issuer issues a credential or a verifier requests a proof, these exchanges happen off-chain, typically over secure peer-to-peer channels:

- An issuer could have an API endpoint (HTTPS) where the user authenticates and then the issuer returns a signed credential. This happens within a secure session so only the user gets the credential.
- A verifier might communicate a proof request to the user's agent via a QR code scan or a push message. The user's agent then prepares the proof and sends it back directly to the verifier.
- Protocols like DIDComm (decentralized identity communication) define secure messaging between DID endpoints. For example, the verifier might send an encrypted message to `did:dxid:alice...` which gets routed to Alice's agent, asking for a certain credential proof. Alice's agent, after user consent, replies with the proof.

Throughout these interactions, **data minimization** is practiced: the user's agent only sends what is asked and approved. If a verifier requests more info than necessary, the user can refuse or the agent can negotiate to only provide a ZK proof instead of raw data. This aligns with privacy principles and also helps prevent oversharing inadvertently.

Preventing Correlation: A significant privacy concern in identity systems is that using the same identifier or credentials everywhere allows different verifiers to correlate and track a user's activities. DxID mitigates this in multiple ways:

- As discussed, users can use multiple DIDs. This means the credentials an issuer gives out could be tied to different DIDs for the same person in different contexts. If Alice has separate DIDs for healthcare and finance, then a bank and a hospital cannot as easily tell it's the same "Alice" just from the DID or credential metadata.
- The credentials themselves can be designed to not contain superfluous correlation identifiers. For instance, a credential could omit something like a social security number if it's not needed, or replace it with a token specific to that issuer and that subject.

- When presenting proofs, the user's agent might choose to not always present the same credential file. If an issuer gave two different credentials with overlapping info, the user could choose which one to use depending on context, to avoid the same credential ID showing up in multiple places.
- On the verifier's side, they see only what is revealed. For example, if Alice proves "over 18" to a nightclub and separately proves "over 18" to a lottery website, those two verifiers receive different zero-knowledge proofs that do not contain a common identifier (other than perhaps the issuer's public key reference). They would not be able to conclude it's the same person unless additional info was voluntarily revealed or the proofs were constructed to include something like Alice's DID (which is usually not necessary to include explicitly if the proof is tied to her DID's secret key).

It is acknowledged that managing multiple DIDs and credentials can be complex. Therefore, the user's wallet software automates as much as possible. For example, it could automatically select a new DID for each verifier if maximum privacy mode is desired. However, that might require the verifier to trust a new DID's credential without linking it to an existing reputation – which is fine if the trust comes solely from the issuer's attestation. Many SSI systems promote pairwise unique identifiers to prevent correlation, and DxID is compatible with that approach.

Example – Minimal Disclosure: Alice wants to prove to an online forum that she is a verified student to get a student discount, but she doesn't want to reveal her full identity. She has a university degree credential from State University. Using DxID, her wallet crafts a presentation that includes a ZK proof: *"I have a credential from State University asserting I am a student"* (perhaps the proof specifically shows that the credential's degree field is not empty or that enrollment status = "student"). The verifier (forum) learns only that Alice is indeed a student verified by State University. They don't learn her name, student ID, or graduation year. Moreover, if Alice later proves something else (like "I have a credential from State University asserting I have a degree" to some employer portal), that portal cannot link her to the forum usage, because the proofs don't contain a common identifier – the forum saw an anonymous proof, the employer saw another anonymous proof. Only State University could know both proofs pertain to Alice's credential if it somehow got logs of it, but State University isn't contacted during verification. This way, Alice's activities at the forum and employer remain unlinkable, preserving her privacy.

Regulatory Considerations: By keeping personal data off-chain and only sharing it in a controlled, consensual manner, DxID aligns well with privacy regulations. For example, under GDPR (EU data protection law), if Alice just presents a proof "over 18 = true" to a service, that service is not processing her personal data like birthdate or name – thus they have less regulatory burden and there's less risk in case of data breach [curity.medium.com](https://www.curity.medium.com)]. The issuer (like the government) of course had her info, but they are a regulated data controller in the physical world anyway. The verifier does not become an additional honeypot of personal data. This decentralized and minimal disclosure model is often highlighted as a way to improve compliance and reduce risk for businesses while enhancing user privacy [curity.medium.com](https://www.curity.medium.com)].

In summary, DxID's off-chain storage and privacy architecture ensures:

- **Users hold their data**, not a central server.
- **Data in transit is encrypted** (through secure protocols like HTTPS, DIDComm).
- **Data at rest is encrypted** (in the wallet or backups).
- **No unnecessary data on-chain** (only what's required for verification like DIDs and revocation status).
- **Minimal disclosure** through selective disclosure and ZKPs to avoid oversharing.
- **Anti-correlation techniques** (multiple DIDs, pairwise pseudos, and minimal consistent identifiers) to prevent different verifiers from tracking the same user without consent.

Verification Process and ZK Proof Generation

Now we detail how a typical verification occurs in DxID: how a verifier requests proof of some identity attributes, how the user's wallet generates the proof (often using zk-STARKs), and how the verifier checks it. This interactive flow is crucial to understanding how all pieces come together for an end-to-end use case.

1. Verifier Request: A verifier (e.g., a service or smart contract) has a certain requirement. For instance, *"User must be over 18 and have a valid government-issued ID"* or *"User must possess a diploma credential from an accredited university"*. The verifier communicates this requirement to the user in the form of a **proof request** or **challenge**. In a web scenario, this could be a QR code or a deep-link that triggers the user's DxID wallet, containing a JSON with the requested proof criteria (like a template describing what credential or issuer is needed and what condition to prove). It also includes a nonce or challenge string to prevent replay attacks (ensuring the proof is freshly generated for this verifier). In a smart contract scenario, the contract might emit an event or the user's wallet might fetch a challenge from the contract.

2. User Consent: The user's wallet displays the request to the user: e.g., "Verifier X is asking you to prove you are over 18 and have ID from Government Agency. Do you want to proceed?". The user can accept (and thereby proceed to generate the proof) or deny (refuse to share/prove that information). User consent is a vital step to maintain user agency in the process.

3. Proof Generation (ZK-STARK execution): If the user consents, the wallet searches the user's stored credentials for one(s) that satisfy the request. For example, it finds Alice's government-issued ID credential and her birthdate within it for the over-18 requirement. The wallet then prepares a **zero-knowledge proof**. This involves:

- Loading the relevant credential data (issuer DID, credential signature, and the attribute values needed).
- Formulating the computational statement to prove. For our example: *Check that `credential.subject == my DID`, `issuer == GovAgency DID`, and `birthDate <= (Today - 18 years)`. Also verify issuer's signature on the credential. And ensure credential is not revoked.* All these checks can be built into a single arithmetic circuit or program.
- Running a STARK prover on this statement. The prover takes as **private inputs**: Alice's credential (with all the claims) and perhaps Alice's DID private key (if we want to also prove she owns that DID by signing something internally). It takes as **public inputs**: the issuer's public key, the condition "age ≥ 18", and the verifier's provided nonce (so the proof is tied to the specific challenge).
- The result is a STARK proof attesting that these conditions hold true. The proof might also output some public values like "Issuer = GovAgency" and "Attribute proven = age_over_18" so the verifier knows what was proven and by whom.
- Optionally, the wallet could also simply sign a message with Alice's DID key to prove the presentation is coming from the DID owner. In many VC presentation protocols, there's a concept of a "proof of possession" of the credential by the subject. This can be done by having Alice sign the verifier's nonce with her DID private key. In our architecture, we can either incorporate that into the STARK proof (e.g., the STARK could include a step where it verifies a signature by the DID over the nonce, meaning the prover must have done that with the DID key), or we do it outside the ZK proof as a separate digital signature attached to the presentation. Either approach links the proof to the legitimate holder of the DID so that someone who stole just the credential file couldn't use it.

This proof generation is computational but well within modern devices' capability for typical credential checks (which are small circuits). If it's a larger proof (say proving something about a large dataset or a complex policy), the wallet might offload to a more powerful device or cloud service *that the user trusts* – but that is an edge case; most often a phone can produce the proof in a second or two for a simple check.

4. Presentation to Verifier: The wallet packages the proof into a **verifiable presentation** format. This could be a JSON containing:

- The zero-knowledge proof data (usually a blob of proof plus some metadata like which scheme and parameters were used).
- Disclosure of any necessary public information. For example, the verifier might need to know *which issuer* issued the credential to apply their trust policy. So the presentation may explicitly state "Issuer: did:dxid:GovAgency" (which the proof is tied to). If the verifier also needs to know the type of credential, it might state "Credential type:

GovernmentID”.

- A signature by the user’s DID on the whole presentation (to prevent tampering and to authenticate it was the user who sent this). This is like wrapping the ZK proof in an outer digital signature envelope for integrity and binding to the user.

The presentation is then sent to the verifier. This could happen via the web (POSTing to an endpoint, or the wallet app returning control to a web page with the proof), or scanning a QR code that contains the proof to a verifier app, etc., depending on context.

5. Verifier Validation: The verifier now must validate the proof and the presentation:

- They verify the user’s DID signature on the presentation (ensuring it indeed comes from the identity in question).
- They verify the **zk-STARK proof** using the public verification algorithm. This involves using the known verification key (which in STARKs is just a set of public parameters, no trusted setup key needed) to check the proof’s hashes and polynomial commitments. If the proof is valid, the verifier now knows with cryptographic certainty that the statement is true – in our example, that *GovAgency issued a credential to this user with birthdate meeting the age condition*.
- They check that the issuer is one they trust for that attribute. For instance, they see “Issuer: did:dxid:GovAgency” came out in the proof (or is implied because the proof is only valid if signed by GovAgency’s key). The verifier cross-checks that DID against their trust list (perhaps they have an internal list or fetch a trust registry that says GovAgency is indeed a recognized government issuer).
- They may check revocation: perhaps the proof was constructed such that it included a check against the latest revocation status (some advanced proofs can incorporate a Merkle proof of non-revocation). If not, the verifier could query the issuer’s revocation registry with the credential ID or a revocation identifier that the user provided in the presentation (if the user had to reveal a non-identifying revocation handle).
- Optionally, the verifier might log the fact that “User with DID X proved attribute Y at time Z”. But they still don’t know the user’s name or any extraneous info unless that was part of the proof. They might just log an internal user account number that was associated with this DID from when the user registered with the service.

If all checks pass, the verifier concludes the verification is successful. The user has met the requirement.

6. Outcome: The verifier grants access or service to the user. For example, the website unlocks age-restricted content for Alice's session. Or the smart contract allows Alice's blockchain transaction to execute, knowing she provided a proof of KYC off-chain that was referenced on-chain via a validity hash (if we extend this concept to on-chain verification, one might post a proof or a proof digest on-chain – more on that in the blockchain integration section).

Throughout this process:

- The **issuer** was never involved in real-time. They did their job at issuance. They are not tracking Alice's usage of the credential; as an earlier quote described, "the issuer does not know how or when the holder uses the credentials"[curity.medium.com](#)]. This preserves privacy and prevents something akin to the federated identity tracking where your IdP (identity provider) sees every login (as happens with OAuth logins).
- The **user** is in control and aware of what is being shared/proven. They had to consent and their wallet only gave exactly what was asked (in ZK form).
- The **verifier** gets a high-assurance result (better than just visually checking an ID). Cryptographically, a DxID proof cannot be forged by the user (since they'd need to break ZK soundness or the issuer's signature). It's also tamper-evident – if something was off (say the credential was revoked or fake), the proof wouldn't verify. This is stronger than, say, just looking at an image of an ID card or trusting a third-party API without cryptographic verification.

Example – Multi-Attribute Proof: Consider a more complex case: applying for a loan might require “prove your name, and that your income is above X, and that a known credit agency vouches that you have no bankruptcies”. In a single proof, the user could prove:

- They have a credential from their employer or tax authority with an income field $> X$.
- They have a credential from the credit agency with a “noBankruptcy = true” claim.
- Their name is the same on both credentials and the verifier can see the name. (This might require revealing the name attribute in plaintext, or proving that the name in both credentials matches each other and then disclosing it.)
- These credentials are both bound to the same DID (the user’s DID).
- All signatures (employer’s, agency’s) are valid and issuers are trusted. This can be done by combining multiple credentials in one STARK proof or by doing multiple proofs and combining results. DxID’s flexibility with STARKs means we can encode quite complex logic (AND/OR conditions, arithmetic comparisons, etc.) in the proofs, far beyond just revealing fields.

On-Chain Verification Scenario: If the verifier is a smart contract (for instance, a decentralized application that wants to enforce an age check without centralized servers), the flow is slightly different:

- The user generates a proof off-chain (because doing it on-chain is impossible computationally).
- The user then submits the proof (or a digest of it) to the smart contract. Because STARK proofs are transparent and relatively larger, one approach is to have an on-chain STARK verifier. StarkWare has demonstrated on-chain STARK verification (e.g., in their StarkEx validity proofs on Ethereum). It's heavy but possible for certain sizes. Alternatively, the contract could accept a hash of the proof and rely on some off-chain trusted service to validate (less ideal decentralization-wise).
- The contract would run the verification algorithm in Solidity (or WASM, etc.) to check the proof given the public inputs (like it would have the issuer's known public key hardcoded or accessible).
- If valid, the contract marks the user as verified (maybe by storing a Boolean under the user's on-chain address or DID).
- Future interactions can then rely on that on-chain flag rather than repeating the proof each time.
- This hybrid approach means the expensive cryptography is done once per user on-chain, amortized across sessions.

However, on-chain integration details we will explore in a later section. For most user-centric applications, verification will happen off-chain as described, with only the end result (like granting access) reflected on-chain if needed.

To close this sub-section, the verification process in DxID is secure and privacy-preserving:

- The **verifier learns only what was in the proof** and nothing else (thanks to zero-knowledge).
- The **user cannot cheat** the proof without detection (soundness).
- The **issuer's authority is correctly utilized** without involving them in each transaction (the issuer's signature is embedded in the credential and proof, so their trust carries through).

- The protocol ensures **freshness** (nonce prevents reusing old proofs) and **authenticity** (binding to user's DID).
- All parties can have high confidence in the outcome with minimal data exchange.

Content Authentication Protocol

Beyond personal attributes, DxID is designed to authenticate *content*. In an age of misinformation and deepfakes, being able to verify who created a piece of content and that it hasn't been tampered with is extremely valuable. This section describes how an identity (DID) in DxID can be used to sign and verify content, and how zero-knowledge or credentials can be attached to content to provide context (e.g., the author is certified or the content is unaltered).

The **content authentication protocol** works roughly as follows:

1. Content Signing by Author: Suppose Alice creates a digital content – it could be a document, an image, a video, or even a block of text (like a social media post). Alice wants to ensure that anyone later can verify it came from her and hasn't been modified. She uses her DxID identity to sign the content. There are a couple of approaches:

- *Direct Signature:* Alice computes a cryptographic hash of the content (e.g., SHA-256 of the file or message). Then she uses her DID's private key to create a digital signature on that hash. She packages the signature along with metadata such as her DID, and possibly a timestamp. This package can be attached to the content (as a separate file or embedded in metadata fields, e.g., EXIF in images or as a JSON Web Signature around a piece of data).
- *Content Credential:* Alternatively, Alice could create a verifiable credential about the content. For example, a credential that says "This image with hash H was created by Alice at time T and is of type photograph." She would then sign this credential as the issuer (since she is effectively issuing a credential to the content). However, usually we don't need a full VC for content, a simple signature suffices.

The simplest model is akin to how PGP signing a document works or how code is signed. The identity's private key produces a signature that can be verified with the identity's public key (from the DID Document). This provides **authenticity** (knowing who authored it) and **integrity** (detecting any changes to the content via hash mismatch).

If Alice is a journalist or a government entity, she might also include additional information in the signature metadata:

- A human-readable claim: e.g., "Signed by Alice, Reporter at XYZ News, on 2025-04-24".

- A reference to a credential: e.g., Alice might include a reference to her “Press ID credential” and even attach a ZK proof of her press credentials. This could be done by generating a proof and embedding it in the content’s metadata or a sidecar file. For instance, a news article PDF could contain an embedded proof that *“the author holds a valid Press ID from PressGuild”* without revealing the author’s personal identity if they wanted to remain pseudonymous but vetted.

2. Publishing Content: Alice then publishes or sends the content along with its signature metadata. For example, she uploads the signed article to a website, or if it’s a video, she includes the signature in the video’s metadata and uploads it to a platform that preserves it, or uses a content authenticity registry. In a decentralized context, she might store it on IPFS or another decentralized storage with the signature. The key is that the signature (and any proofs) travel with the content or are easily retrievable by anyone who gets the content.

3. Verification by Consumers: Now suppose Bob comes across this content (the document or video). Bob wants to verify its authenticity:

- Bob obtains the content and the attached signature data. He sees it claims to be signed by `did:dxid:alice...` (or maybe it gives the full DID URL to a public key like `did:dxid:alice#key-2025`).
- Bob uses a DID resolver to fetch the DID Document for `did:dxid:alice...`. He obtains Alice’s public key.
- Bob computes the hash of the content himself and checks the digital signature using Alice’s public key. If the signature verifies, Bob now knows this content was indeed signed by the holder of that DID (presumably Alice). He also checks the signature’s timestamp if provided (to know when it was signed) and perhaps whether the DID was valid at that time (not revoked or rotated).
- Bob might also check if Alice’s DID has any credentials or endorsements. For example, Bob might know “Alice is a reporter” because either she disclosed that or Bob has a trust framework that says DID Alice is associated with a known entity. In practice, Alice might have her DID tied to a public profile or even a Verifiable Credential that is publicly discoverable (like an attestation from her news organization). If so, Bob can further verify that the content came from a credible source.

What if Bob doesn’t know who Alice is? At minimum, Bob knows the content is signed by *someone* who controls that DID. If that DID corresponds to an organization or known person, that can be conveyed in the DID Document or via public credentials (the DID Document could include a service endpoint or claim like “companyName”: “XYZ News”). If not, Bob might rely on

a web of trust or a reputation system: e.g., if content is frequently signed by a DID that's been accurate in the past or endorsed by others, it gains credibility.

Tamper Detection: If any bit of the content was changed after Alice signed it (for example, a deepfaker altered the video), the signature verification will fail because the hash won't match. This is a strong guarantee of integrity. As the UNSW deepfake article noted, *"anyone will be able to tell if even a pixel in a digital video has been altered"* with such a solution businessthink.unsw.edu.au]. This is because the slightest change in content produces a completely different cryptographic hash, invalidating the original signature. Thus, content signatures provide a robust way to detect manipulation.

Content Timestamping: The blockchain can be used to anchor content authenticity in time. For instance, Alice could record the hash of her content on the blockchain (or her signature of it) at the time of publication. This creates a public timestamp. Later, if someone sees a conflicting version of the content or claims it was created later by someone else, the blockchain timestamp serves as evidence of the original. Some systems like OpenTimestamps or Chronicle use this principle. In DxID's context, Alice's identity could periodically batch timestamp her signed content (e.g., by including it in a daily Merkle tree whose root is on-chain). This is optional but adds additional trust (especially if one worries about key revocations or disputes).

Combining with Credentials (Misinformation Resistance): DxID can integrate content signing with identity credentials to combat misinformation:

- A government could require that official announcements (content) be signed by an official DID, and that DID is known to belong to, say, the "Ministry of Health" with a credential attesting that. Then any press release can be verified by citizens: *signed by did:gov:healthministry, which we know is legit (because that DID has a credential from the Government DID itself or is listed on an official site)*. This prevents fake press releases.
- Journalists could have DIDs with credentials from a Press Association. When they sign content, they might attach a ZK proof "I am a certified journalist". This way, anonymous whistleblower content could still carry a proof of authenticity ("I am a verified employee of company X") without revealing the author's name cheqd.io]. That builds trust in the content while preserving anonymity where needed.
- Social media posts could be signed by user DIDs; platforms could then show a badge if a post is verified. Or in a more decentralized scenario, people could choose to trust posts that come with signatures from known identities and down-rank anonymous or unsigned content.

As another reference, a cheqd whitepaper on verifiable AI content describes how content (like deepfake videos or AI-generated articles) can carry *"a cryptographic proof linking it to its original dataset and creation process,"* and be *"timestamped and stored on decentralized ledgers,*

ensuring it wasn't altered post-publication.”cheqd.io]. This mirrors our approach: the signature links to original author and time, and storing either the signature or hash on a ledger can provide a decentralized audit trail.

Verification by Platforms: Not only end-users, but also services (like a video hosting site or news aggregator) can automatically verify incoming content. They could reject or flag content that isn't properly signed by a trusted identity. For example, an email provider could integrate DxID to check if an email purportedly from “Alice” actually has a DxID signature from Alice's DID – if not, it might warn that the sender is unverified (similar to DKIM but at the identity level).

Diagram (Conceptual): Imagine a diagram with three parts: Content Creator (with DID and credentials) -> signs content -> Content (with signature) -> Consumer verifies signature -> Consumer trusts content. The core idea is that under a decentralized identity regime, “every public figure or noteworthy source would have their official public key registered on a public blockchain. All content released by them is signed with their private key. Everyone in the world can instantly verify if the content is correctly signed by checking it against the public key, and also verify that the content's hash matches (detecting any tampering)”[businessthink.unsw.edu.au](https://businessthink.unsw.edu.au/businessthink.unsw.edu.au)]. This approach has been proposed as a solution to deepfakes: for instance, a professor at UNSW argued that a global DID framework could allow verification of videos from leaders to prevent fake speeches[businessthink.unsw.edu.au](https://businessthink.unsw.edu.au/businessthink.unsw.edu.au)]. DxID essentially provides the infrastructure to do exactly that.

In summary, the Content Authentication Protocol in DxID enables:

- **Provenance:** Recipients can trace content back to a DID identity (and then to a real entity or pseudonymous reputation).
- **Integrity:** Any alteration breaks the signature, so false or edited content is detectable.
- **Attribution & Accountability:** Authors are accountable for what they sign; they cannot later repudiate it, and forgers cannot impersonate them without stealing their keys (which, if keys are well protected, is much harder than, say, copy-pasting a logo onto a fake document).
- **Misinformation Deterrence:** It raises the bar for misinformation actors. If the ecosystem expects signed content, unsigned content might be viewed suspiciously by users or automatically filtered. Over time this creates a norm where authentic information carries cryptographic credentials, making it easier to spot fake, unsigned information.

This closes the architecture section by tying identities to content as well as to personal data. Next, we proceed to detailed **use cases** demonstrating DxID in action in various sectors, which will make the above concepts more concrete.

Use Cases of DxID

DxID's capabilities can be applied across numerous domains to improve security, privacy, and efficiency. In this section, we explore several full use cases in different sectors:

- **Financial Services (KYC and Customer Onboarding)**
- **Healthcare (Patient and Provider Credentials)**
- **Education (Degree and Certificate Verification)**
- **Supply Chain (Product Provenance and Compliance)**
- **Misinformation Prevention (Authentic Media and News)**

For each use case, we will describe the current challenges and show how DxID provides a solution, often referencing the architectural components described above. These scenarios will illustrate the real-world value of decentralized identity with privacy-preserving proofs.

Financial Services: Privacy-Preserving KYC and Compliance

Current Challenges: In banking and finance, regulations like KYC (Know Your Customer), AML (Anti-Money Laundering), and CFT (Counter Financing of Terrorism) require institutions to verify identity information about customers. Traditionally, this involves collecting copies of government IDs, proofs of address, tax identification numbers, etc. Customers have to repeatedly submit the same documents to different banks and exchanges, risking exposure of personal data. Centralized databases of KYC information are high-value targets for hackers (many exchanges and banks have suffered data breaches leaking customer IDs). Also, KYC processes are time-consuming and costly for financial institutions.

DxID Solution: With DxID, a user can carry verifiable credentials for KYC info – e.g., a government-issued ID credential, a proof-of-address credential from a utility company, a tax ID credential – all stored privately. When onboarding to a new financial service, the user can selectively prove what the institution needs:

- **Proof of Identity:** The bank needs to know your legal name and that your government ID is valid. Instead of uploading your ID card, you present a verifiable credential (or proof derived from it) that contains your name, DOB, and a photo, issued by the government. The bank's system verifies the credential's signature (from the government DID) and possibly checks revocation (e.g., ensure the ID isn't reported lost). The bank sees the necessary details (name, DOB, maybe the photo if needed for face match), but nothing else like your ID serial number or personal document numbers need to be exposed if not required.

- **Proof of Address:** Perhaps the bank needs to confirm your address for regulatory reasons. You have, say, a credential from the postal service or a previous bank statement credential confirming your address. You generate a proof that *“My address on credential X matches the address I provided and credential X is from a trusted issuer (e.g., a utility company)”*. Or you might simply share the address field from a verifiable credential signed by a utility company, which the bank trusts more than an editable PDF bill. The bank gets confidence the address is valid without you sending a paper bill.
- **Sanctions/AML Check:** Instead of the bank screening your name against a sanctions list themselves (which they still may do if they see your name), a future scenario could be that you carry a credential from a regulated identity provider that states *“Not on Sanctions List as of Jan 2025”*. You could present a zero-knowledge proof that *“I have a credential from KYC-Provider Co. stating I am not on any watchlist”* without revealing your name or other details to the verifying service (this is more speculative, as banks typically have to know your name anyway – but you could imagine a cryptocurrency exchange that just needs to know you’re not blacklisted, without storing your identity, which would be revolutionary for privacy in compliance).

In all these, the **user only shares what’s needed**. The bank doesn’t accumulate piles of documents; it gets cryptographic attestation of the facts it needs, which are harder to forge than scanned documents. The user’s personal data isn’t sitting in as many databases (just remains with user and maybe some primary credential issuers like the government). The risk of identity theft is reduced since fewer entities are holding copies of your documents.

No-KYC Scenario: There’s a concept of “No-KYC” or “Zero-Knowledge KYC” emerging in DeFi, where a user can prove eligibility without revealing identity to the protocol (to maintain decentralization). For example, a DeFi platform might require *“user is not from a restricted jurisdiction and is over 18”*. Using DxlID, the user could prove those conditions via zk-STARK proofs from credentials (like country = not on blacklist, age > 18threeprotocol.ai). The platform’s smart contract (or an off-chain verification service) checks the proof, and then the user’s address is allowed to interact. Throughout, the user remains pseudonymous to the platform, achieving compliance without formal KYC. Three Protocol, for instance, has explored *pseudonymous on-chain IDs with zk-STARK proofs to enable “No-KYC” marketplacesthreeprotocol.ai). They found it enhanced privacy and reduced the risk of data breaches by eliminating centralized KYC databasethreeprotocol.ai).

Benefits:

- **Privacy:** The user isn’t handing over a full dossier each time. Only necessary info is disclosed. For instance, if a certain investment is only open to accredited investors, the user can prove “I am an accredited investor” via a credential from a regulator, without exposing net worth or all financial details.

- *Security*: The financial institution gets cryptographically signed data directly from issuers (government, etc.), reducing reliance on human inspection of documents that could be photoshopped. Also, if their database is hacked, the data they stored can be minimal (they might just store “KYC verified = yes” and reference to credential IDs, instead of actual passport scans).
- *Efficiency*: Onboarding can be faster – automated verification of digital credentials can be done in seconds, vs. waiting days for manual review of documents. Also, the user doesn’t have to repetitively fill forms; their wallet can provide the necessary data in one click.
- *User control*: Users can revoke consent or limit what’s shared. For example, if a user stops using a service, they could request deletion knowing the service didn’t have much data in the first place beyond a yes/no verification outcome.
- *Regulatory compliance*: Paradoxically, giving less data to verifiers can make compliance easier. As noted earlier, if verifiers receive only minimal data (e.g., a “above 18” flag), they have less personal info to guard, which helps them comply with privacy law [curiositymedium.com](https://www.curiositymedium.com) while still meeting the regulatory requirement through a trusted attestation.

A concrete implementation: **Digital Identity Wallets for Banks**. There are already national digital ID schemes (like eID in Europe or Aadhaar in India) that can be used for banking KYC. DxID would generalize this in a decentralized way: instead of a centralized ID, multiple credentials from various issuers (government, employer, etc.) in the user’s control could satisfy KYC. Banks in different countries could accept local government-issued credentials (through an interoperable DID system). This reduces duplication – e.g., instead of each bank doing KYC, a one-time verification credential could be re-used.

Some projects like zkPass and others are trying to do KYC with zero knowledge. The approach described fits right into that category: using ZK proofs for identity verification in finance. As an example, Three Protocol’s *no-KYC digital IDs on-chain* allows pseudonymous profiles to engage in commerce with trust ensured by zk-STARK proof threeprotocol.ai.

Healthcare: Patient and Provider Identity, Health Credentials

Current Challenges: Healthcare involves sensitive personal data and a need for strong identity verification. Patients often have to fill out repetitive forms with the same data at each clinic. Proving you have insurance coverage, or that you’re eligible for a certain treatment (e.g., age or condition), or vaccinated for a disease currently involves sharing documents or records that may reveal more info than necessary. Likewise, medical professionals need to prove their licenses and qualifications across hospitals – typically via certificate copies and lengthy credentialing processes. Privacy of health data is critical due to regulations like HIPAA, and breaches of health records are highly concerning.

DxID Solution: DxID can empower both patients and providers with cryptographic credentials:

- **Patient Credentials:** Patients could have credentials for things like vaccination records, prescriptions, test results, or insurance membership. For instance, a COVID vaccination credential issued by a pharmacy, or an insurance card credential issued by an insurer. When visiting a new doctor, instead of filling a form with their vaccination history, the patient lets the clinic scan a QR code that presents verifiable proof of required immunizations. This proof could be selective – if the clinic only needs to know if you have the COVID vaccine, the proof can just say “Yes, 2 doses, issuer: PharmacyX” without disclosing other vaccines. During a pandemic, to enter a venue requiring vaccination, you show a QR that is verifiably signed by a health authority (many COVID pass systems were actually early examples of verifiable credentials, though not always decentralized).

Insurance verification is another example: the patient presents a credential from their insurance company proving coverage (maybe just a policy ID signed by the insurer). The clinic verifies it and knows the patient is covered, without needing to make phone calls or see the full policy details. Moreover, any personal data (like what plan details) can be kept minimal.

- **Age or Condition Proof:** Certain medications or procedures require the patient to be above a certain age or to have a certain diagnosis. Instead of exposing birthdates or entire medical files, the patient could obtain a credential from a doctor that simply asserts the relevant fact (e.g., “Patient X has condition Y”, or “Patient is over 50 and qualifies for screening Z”). Then the patient proves that to whatever service needs it. For instance, to get a vaccine prioritized for seniors, you prove “age > 65” via your ID credential, without giving exact birthdate to the pharmacy’s system (which might only need to log yes/no).
- **Provider Credentials:** Doctors, nurses, and other licensed professionals can have their license as a verifiable credential issued by a medical board. When they join a new hospital or need access to electronic health record systems, they can present that credential. The hospital verifies it (signatures from the board and no revocation), possibly checks additional attributes (specialty, expiration date of license) via ZK proof if needed, and can onboard the doctor much faster. This is essentially a decentralized version of the credentialing process. If integrated into medical device access, a doctor’s smart badge could cryptographically prove to a medication dispenser “I am Dr. Alice, licensed MD with authority to prescribe narcotics” before it dispenses a controlled drug, increasing security.
- **Medical Records Access:** A patient could control access to their records by managing decryption keys, which is a bit different from verifiable credentials. However, a related usage: a patient’s DxID could be linked to their records, and a doctor wanting to access them needs to present a credential “I am currently treating this patient” or get the

patient's consent signature via DID. While complex, the DID framework can secure logins to health records portals more strongly (no more weak passwords or sharing). The patient's consent for data sharing can be a verifiable transaction recorded (like a credential "Patient X consented to share record Y with Dr. Z on date D").

Benefits:

- **Interoperability:** Because credentials are standard-based, different healthcare providers and insurers can accept each other's credentials (assuming trust relationships or common standards). This improves interoperability of health information – something that is notoriously difficult with current siloed systems.
- **Privacy Compliance:** Using ZK proofs and minimal disclosure helps comply with HIPAA and other privacy laws. For example, if a clinical trial needs to verify participants have a certain condition without knowing their identity, participants could present an anonymized proof "I have condition X confirmed by Hospital Y [chain.link](#)". The trial organizers get the needed assurance but don't collect personally identifying health info.
- **Fraud Reduction:** Fraudulent insurance claims or fake medical licenses can be curbed because credentials are harder to forge than paper documents. For instance, a fake doctor couldn't present a verifiable license because they wouldn't have the private key of the medical board. Similarly, someone trying to misuse another's insurance would fail if they can't produce the digital credential (which ideally is bound to the person's DID).
- **Patient Convenience:** No need to repeatedly fill forms or carry a thick folder of records. A digital wallet could auto-fill forms with verifiable claims or just directly send required info to a doctor's system with patient approval. It can also prevent errors that come from manual data entry.

Example: Vaccination Passport: During international travel, instead of carrying a yellow fever card, a traveler holds a DID-based vaccination credential (some countries are already exploring this via the ICAO Visible Digital Seal or EU's Digital COVID Certificate, which are conceptually similar to W3C credentials). Border control scans the QR, the credential's issuer (e.g., country's health ministry) signature is verified [businessstink.unsw.edu.au](#)], and entry is granted. The traveler's other health data remains private. This could extend to negative test results or recovery certificates in outbreaks.

Another example: *Prescription ZK Proof*. Say a telemedicine app needs to ensure a user indeed has a prescription for a certain medication. A user could have a credential from their doctor or pharmacy for that prescription. The app can verify it without seeing the actual prescription (just that it's valid for that medication and hasn't expired). This prevents abuse and ensures regulatory compliance in online medicine sales.

Education: Academic Credentials and Skill Certifications

Current Challenges: Verification of degrees, transcripts, and certifications is often manual and slow. Alumni have to request transcripts or diploma copies to be sent to employers or other institutions. Employers spend significant time and money verifying that job candidates actually have the degrees or skills they claim. Fake degrees and certificates are a known problem. Additionally, learners often have multiple micro-credentials from online courses, but no unified way to present them.

DxID Solution: Verifiable Academic Credentials:

- Universities and schools can issue diplomas, degrees, transcripts, and other certificates as verifiable credentials to students' DIDs upon graduation. For example, Alice's DID receives a credential from State University: "B.Sc in Computer Science, awarded 2025". This credential is signed by the university's DID (which is trusted because it's listed in a trusted registry of educational institutions).
- Students can also get other credentials like course completion certificates, honors, or extracurricular achievements similarly.
- **Presenting to Employers:** When Alice applies for a job, she doesn't need to have her university send a sealed transcript. Instead, she presents the verifiable degree credential to the employer. The employer's HR system verifies the digital signature (ensuring it's from State University and hasn't been tampered). They see the degree name, field, and date. They could also receive a proof of GPA or specific courses completed via transcripts credential, possibly with selective disclosure (e.g., prove "took courses A, B, C" without listing all grades if not needed).
- Because the credential is cryptographically signed, the employer can be confident it's authentic. This streamlines background checks significantly. It can even be automated: an application tracking system could automatically verify credentials in seconds, only flagging if a credential doesn't verify or the institution isn't recognized.
- **Skill Credentials:** In the modern workforce, people also have certifications (like Cisco network engineer, Project Management Professional, etc.) and MOOC (Massive Open Online Course) certificates (Coursera, edX, etc.). Each of these can be a verifiable credential issued by the respective organization. The learner's DxID wallet accumulates all these over time.
- When needed, the learner can compile proofs. For a freelance platform, the user might prove "I have these 5 professional certificates" to earn a trust badge, without exposing serial numbers or full certificates to the platform (the platform just sees the titles and issuers, or maybe just a "certified yes/no" result).

- **Student Mobility:** Students transferring schools could present credentials of completed coursework to get credits at the new school, and the new school can trust the data (the credential might detail the course syllabus and grade, signed by the old school). This could eventually eliminate the need to send official transcripts between registrars by mail.

Digital Diploma Example: Several universities (like MIT) have experimented with issuing blockchain-based diplomas [nature.com](https://www.nature.com)]. Those are essentially early forms of verifiable credentials. DxD provides the standard way: the university is the issuer DID, the diploma is the credential, and the graduate is the subject DID. The MIT example showed that graduates could directly share a verifiable digital diploma with employers, who could validate it through a mobile app in seconds, rather than contacting the university.

Open Badges Integration: There's an existing standard called Open Badges for digital certificates. Those could be adapted or encapsulated as W3C Verifiable Credentials. DxD can unify these with more formal credentials.

Benefits:

- **Fraud Prevention:** Fake degrees would be extremely difficult since employers would expect a verifiable credential. If an applicant presents a degree that cannot be cryptographically verified (signature doesn't check out or issuer DID not recognized), it will be flagged instantly. Diploma mills would lose value if their credentials aren't in the trusted registry.
- **Efficiency:** Verifications that took days or weeks (and sometimes fees for transcript services) can be done instantly for free (barring minimal blockchain lookup costs). This reduces administrative overhead for both schools and employers.
- **Learner Ownership:** Students have lifelong ownership of their achievements. Even if an institution closes or its records are lost, students who have their credential signed and in their wallet still possess verifiable proof of their education. (There is a dependency on being able to still resolve the issuer DID and key – ideally, the keys of the institution would be maintained by some custodian if the institution ceases to exist, or anchored in a long-lived ledger).
- **Selective Disclosure:** A student might not want to share their entire transcript for a job that only cares about degree and major. With ZK, they could prove “has degree X” without revealing GPA, or prove “completed course Y” without revealing the grade. This protects students from potential bias. For example, sharing that you graduated is enough; you don't have to share that you got a C in one course if it's not relevant.
- **Stackable Credentials:** In the modern economy, people accumulate many small credentials (bootcamps, workshops, nano-degrees). DxD allows a person to present a

consolidated profile of verifiable skills. An online professional profile (like LinkedIn of the future) could be underpinned by actual verifiable credentials instead of just self-assertions or endorsements.

Comparison to Existing Systems: Currently, we see things like the National Student Clearinghouse in the US verifying degrees, or platforms like LinkedIn just trusting user input (with occasional background check by third parties). DxID would integrate the clearing function into the credential itself. Organizations like Learning Machine and Blockcerts (MIT's project) have demonstrated the feasibility info.acreditta.com]. Our approach is aligned but using the latest interoperable standards and adding ZK for privacy.

Use Case Narrative: Alice obtains her B.Sc credential in 2025. In 2030, she applies for a Master's program; she sends her B.Sc credential to the university as part of application – they verify it instantly and accept it as proof of prerequisite degree. In 2035, she applies for a job; her application includes a bundle of credentials: B.Sc from X, M.Sc from Y, and a Professional Engineer license (also a credential) – the employer's system verifies all three in an automated way and fast-tracks her application because her qualifications are confirmed. All along, Alice didn't have to go back to each institution for letters and transcripts – she already had the signed credentials. This demonstrates portability of trust.

Supply Chain: Product Provenance and Compliance

Current Challenges: Supply chains involve many parties and handoffs, and ensuring the authenticity and provenance of goods is difficult. Customers and regulators increasingly want proof that products are sourced ethically (e.g., conflict-free minerals, organic produce) and that they meet safety standards. Currently, this is done with paper certificates, barcodes, and siloed databases. Counterfeit products and fraudulent origin claims are rampant issues (e.g., counterfeit pharmaceuticals, mislabeled origin of food). Recalls are also complicated by lack of traceability.

DxID Solution: Decentralized identity can assign DIDs to organizations, factories, even to batches of products or IoT devices, and use credentials to certify steps in the supply chain:

- **Entity DIDs:** Every manufacturer, supplier, distributor in the chain has a DID. These could be anchored in a consortium ledger or public chain. They might have credentials attached (e.g., a government agency issues a credential "Licensed Organic Farm" to a farm's DID).
- **Product DIDs or Identifiers:** For high-value items or batches, a DID (or some decentralized identifier like an EPC DID) can represent a product or lot. Alternatively, we use the concept of verifiable *events* in the supply chain: each transfer or transformation of a product is logged as a credential or event signed by the actor's DID.

- **Proof of Origin and Chain-of-Custody:** As a product moves, each handler issues a credential or signed statement about it. For example, a mine produces a diamond and issues a credential “Diamond #123 of batch XYZ, mined on 2025-04-01, weight 1.0 carat” signed by MineCo’s DID. It passes to an exporter who signs a transfer declaration. A gem certifier (third party) issues a credential “Diamond #123 is verified natural, VS1 clarity” signed by their DID. These credentials accompany the diamond through the supply chain (possibly referenced via a digital twin or a physical tag linking to the credentials).
 - When a retailer or consumer wants to verify the diamond’s provenance, they compile all these credentials, verify each signature (MineCo’s, exporter’s, certifier’s, etc.), and thus have a secure chain from mine to them. If any link was fake or missing, it breaks.
- **Compliance Credentials:** Consider electronics manufacturing: a factory could have an ISO certification credential, each component might have a RoHS compliance certificate (for hazardous substances), etc. With DxID, when the final product (say a smartphone) is assembled, the manufacturer can present to regulators or importers a set of proofs: *all components in this phone have RoHS certificates from their makers* (this could even be one aggregated ZK proof “All components of type X used in product Y have property RoHS-compliant = true” drawn from multiple credentials without revealing all supplier details). Similarly, for food: a fishing vessel could have a credential for its catch quota usage, a

Supply Chain: Product Provenance and Compliance

Current Challenges: Supply chains involve many parties and handoffs, making it difficult to trace the origin of products and ensure they meet standards. Consumers and regulators increasingly demand proof of provenance (e.g., fair trade coffee, conflict-free minerals) and compliance (safety certifications, environmental standards). Today this relies on paper certificates and siloed databases that are prone to fraud and error. Counterfeit goods and false origin claims are rampant, and audits for compliance often require revealing sensitive supply chain data (like supplier identities or formulas), which companies are reluctant to do.

DxID Solution: In a DxID-enabled supply chain, every entity (supplier, manufacturer, distributor, retailer) has a DID and relevant verifiable credentials. We create a *web of trust for products*: at each stage, the responsible party issues a signed attestation about the product, forming a chain-of-custody that can be independently verified.

- **Organization Credentials:** Companies can have credentials attesting to their certifications or practices. For example, a factory might have an ISO 9001 quality certification credential issued by a standards body, or a mine might have a “Conflict-Free Certified” credential from a watchdog NGO. These credentials (digitally signed by the certifier’s DID) can be presented down the supply chain to assure partners of

compliance. When sharing these, they might do so via zero-knowledge proofs to protect sensitive data if needed (e.g., proving “We are certified conflict-free” without revealing which auditor gave the certificate if that’s confidential).

- **Product Batch Credentials:** Goods or batches of goods can be represented by identifiers (like a lot number or a digital token) associated with credentials. For instance, a cocoa farm could issue a credential for a shipment: *“Batch #123, 1000kg cocoa beans, organic, Grade A”* signed by the farm’s DID. This credential travels with the shipment (perhaps referenced by a QR code on the container). At the port, a customs official can scan and instantly verify origin and organic status via the farm’s signature (and the farm’s organic certification credential issued by an agricultural agency). No paper docs needed, and any attempt to tamper with the data (like change origin) would break the signature.
- **Chain-of-Custody Proofs:** As materials move, each handoff can be logged with a credential or event signed by the giver and receiver. For example, a diamond mine issues a origin credential for a rough diamond. When it’s sold to a cutter, the mine’s DID and the cutter’s DID co-sign a transfer record (or the cutter gets a credential from the mine: “I, MineCo, transferred Diamond #XYZ to CutterCo on date D”). CutterCo can then prove the diamond came from MineCo. If CutterCo then sells to a jeweler, a similar signed transfer occurs. The final jeweler can provide the retail customer with a **provenance dossier**: a series of verifiable credentials from mine to jewelry store, each signed by the relevant party. The customer (or an app) verifies each signature and thus trusts that the diamond is authentic and conflict-free, since each party in the chain was verified. If any link were fake (say a fraudulent middleman), their lack of a valid DID or signature would be apparent.
- **Ethical and Sustainable Sourcing:** With ZK proofs, companies can make verifiable claims about their supply chain ethics without exposing all details. For example, a fashion brand can prove “No child labor was used in manufacturing this garment” by leveraging credentials from its suppliers and perhaps auditing firms, but without revealing the supplier names or factory locations (which might be competitive secrets). This is done by a zero-knowledge proof that each supplier in the chain has a credential from a labor auditor stating compliance with labor law cryptoslate.com6]. The proof might simply yield a true/false to the consumer or regulator query, protecting specific identities of suppliers. In this way, **ZKPs allow verification of ethical claims while maintaining supplier confidentiality cryptoslate.com3].
- **Regulatory Compliance and Audits:** Similarly, in highly regulated industries (pharma, aerospace), companies can demonstrate compliance with standards via credentials instead of lengthy audits. A drug manufacturer can present proofs that every ingredient in a medicine was sourced from GMP (Good Manufacturing Practice) certified suppliers, and that the final product passed required tests, all without revealing proprietary formulations. For instance, an auditor could get a zero-knowledge proof *“All components*

in Product X comply with safety standard Y” which the company generates from the component certificates it holds cryptoslate.com8]. The auditor is satisfied that standard Y is met, yet confidential business information (like exact suppliers or composition) isn’t exposed. This greatly streamlines audits: regulators can quickly verify compliance digital cryptoslate.com4].

- **Anti-Counterfeiting and Product Authenticity:** For luxury goods or electronics, DxID enables strong product authentication. A luxury handbag might come with a credential issued by the brand’s DID: *“Product serial #ABC123, model, date of manufacture, authentic”*. When a customer later scans the bag’s NFC tag, they retrieve this credential and verify the brand’s signature to confirm authenticity. If someone duplicates the tag, they wouldn’t have the brand’s private key to sign a valid credential. Moreover, the brand can prove authenticity through the chain without revealing internal supply chain data: *“This item was produced in our owned factory and transported via our distributor”* can be encapsulated in signed data that consumers or customs can verify without the brand disclosing factory details public cryptoslate.com7]. As a result, counterfeit items can be easily distinguished (no valid credential), protecting both consumers and brand reputation.
- **IoT and Automation:** IoT devices can also participate. For example, a sensor on a storage container could sign a record *“temperature maintained between 2-8°C throughout transport”* (for vaccines or perishable food). This record is a verifiable claim (the device’s DID signs it, and the device might have a credential from the manufacturer to attest its calibration). At delivery, the recipient can verify the container’s log hasn’t been tampered with and meets the required conditions. This ensures product quality with trustless verification.

Benefits:

- **Traceability and Trust:** Each step’s credential provides an immutable, verifiable trail from origin to consumer. If a problem is found (e.g., contamination), one can quickly pinpoint where in the chain it occurred by examining credentials. This traceability is tamper-proof because of the blockchain backing – events are recorded as signed credentials/transactions on a ledger that stakeholders share (perhaps a consortium ledger).
- **Reduced Fraud and Counterfeits:** It becomes extremely difficult for a fake product to slip in. A counterfeit part would lack the chain of credentials from authorized entities. For someone to insert a fake, they would need to compromise private keys of legitimate DIDs (highly unlikely if those are secure). Thus, the system fosters a ****tamper-evident supply chain** cryptoslate.com7].
- **Privacy for Businesses:** Unlike putting all supply chain data on a public blockchain, DxID with ZK proofs allows companies to maintain privacy. Proprietary info (supplier

identities, recipes, etc.) can be kept confidential, yet the claims about them can be verified. As one source notes, ZKPs let companies prove things like ethical sourcing or quality without disclosing the “underlying detail cryptoslate.com0】. This encourages participation, since companies can be transparent *and* protect trade secrets.

- **Streamlined Compliance:** Rather than spend weeks on audits or paperwork, companies and regulators can rely on continuous cryptographic assurance. Compliance checks become as simple as verifying a set of digital signatures and proofs, which is much faster and less error-prone. A process that used to require trust in documents now becomes **trust in math and digital signatures**, which are more reliable.
- **Consumer Empowerment:** Consumers with a smartphone can scan a product and immediately see verified information: origin, certifications, authenticity. They don’t have to rely solely on the brand’s marketing; they get cryptographic proof. This builds consumer trust and allows informed choices (e.g., verifying a fish is indeed from a sustainable fishery with proof from a fishery DID and an environmental agency’s credential).
- **Efficiency and Cost Savings:** Automation of verification reduces manual interventions and intermediary certification costs. Smart contracts could automatically release payment when a shipment’s credentials are verified, for example, speeding up supply chain finance. The Project Provenance (a pilot in supply chains) found that blockchain verification can reduce paperwork and increase transparenprovenance.org3】 – DxID would amplify this with privacy and ZK features.

Example: Coffee Supply Chain. A coffee farmer cooperative issues a credential for each lot of coffee beans stating origin and organic status (signed by cooperative + local certifier). A shipper signs a receipt of that lot. The roaster, upon receiving it, signs a roasting record and quantity. Finally, the coffee bag for retail gets a QR linking to these records. A consumer in a grocery store scans it: an app verifies the coop’s signature (and sees the coop had an “FairTrade Certified” credential from FairTrade International), verifies the shipping and roasting records, and displays to the consumer: “*Origin: Ethiopia, Sidamo region – verified. Organic: Yes – certified by ECOCERT. Fair Trade: Yes – certified by FairTrade International. Roasted by RoastCo on 2025-03-01. Supply chain carbon footprint: <computed>.*” All with green checkmarks indicating cryptographically verified claims, not just printed labels. The consumer can trust the story behind the coffcryptoslate.com4】, increasing brand credibility.

Misinformation Prevention: Verified Media and News

Current Challenges: Misinformation, fake news, and deepfakes are proliferating, undermining trust in media. It’s often hard for people to discern authentic content from false or manipulated content. Social media platforms struggle to moderate or fact-check the vast amount of user-generated content. There is a need for mechanisms to verify content provenance (who

actually created it) and integrity (that it hasn't been altered), as well as to provide credentials for content creators (is this source accredited or reputable?). However, solutions must also balance anonymity and free speech – whistleblowers or activists might need to stay anonymous yet still prove their credibility.

DxID Solution: Building on the **Content Authentication Protocol** described earlier, DxID can help create an ecosystem of **verifiable content**:

- **Verified News Articles:** Journalists and news organizations have DIDs. When publishing an article or video report, they sign it with their DID. The content carries a verifiable signature and possibly a credential reference. For example, a news video could include a signature by `did:bbc:newsroom` and an attached credential from a press association stating that this DID belongs to a bona fide news organization. Consumers (or their news apps) can verify that signature. A green badge “Verified Source: BBC News” could be displayed on platforms next to the content. If someone attempts a fake BBC video, they wouldn't have the signing key, so the content would show as unverified or a bad signature. This addresses deepfakes of known figures: e.g., a deepfake video of a president can be exposed by the fact it's not signed by that president's official DID (assuming public figures adopt signing of official communication businessthink.unsw.edu.au]).
- **Social Media Posts and Profiles:** DxID can integrate with social platforms to verify accounts and posts. A user (or bot) could have a DID with certain credentials (like “Verified identity: John Doe, Government Official” or “Verified domain: owner of example.com”). Tweets or posts can be signed by the user's DID key. Twitter already has a verification system, but cryptographic signing would decentralize and strengthen it. It could even allow multiple verification levels: for instance, a pseudonymous whistleblower might not reveal their name but could present a credential “I am an employee of Company X” in zero-knowledge. They could then post anonymously on a forum a statement about Company X, attaching a ZK proof of that credential. Readers could verify “*the author is confirmed as an employee of Company X (by Company X's HR system or a third-party verifier) but we don't know which employee*”. This lends credibility to the information without outing the person. It creates a **misinformation resistance** mechanism: people are more likely to trust posts that come with verifiable attestation of the poster's relevance or identity, rather than an random anonymous claim.
- **Content Platforms and Fact-Checking:** Platforms like YouTube or Facebook could integrate DxID verification into their UI. For example, videos from certain DIDs (news orgs, celebrities) could get a “verified authentic” tag if the signature checks out. Conversely, videos that purport to be from someone but aren't signed by that person's DID would get a warning label. This approach was suggested by researchers: “under a decentralized identity regime, every public figure or media outlet would register their public key, and all content released by them is signed. Anyone can instantaneously verify if a video or digital content is correctly signed by the official public key and if the

content has been tampered with by checking its hash [businessstink.unsw.edu.au8](https://businessstink.unsw.edu.au/businessstink.unsw.edu.au8)]. DxID provides exactly this capability. Even a single-pixel change in a video would break the original signature, alerting to possible tampering businessstink.unsw.edu.au8].

- **Provenance of Images and Videos:** The Content Authenticity Initiative (led by Adobe) is introducing content credentials that travel with images to record their origin and edits. DxID can complement this by anchoring those content credentials in DIDs and blockchain. For instance, a camera device could sign a photo at capture time (camera's DID, or photographer's DID via the camera). Any subsequent edit in Photoshop could attach an edit credential (with cryptographic link to the original). This way, an image circulating on the internet can be traced: you can verify the original camera signature (to know source and time) and see an audit trail of edits. If an image lacks any verifiable origin, it might be treated with suspicion. This helps combat things like fake viral images, because if a shocking image has no provenance credentials, fact-checkers will be alerted. Some systems already attempt this (Adobe's CAI uses a form of signed metadata, though not fully decentralized). With DxID, the photographer's DID might have a credential ("Registered Journalist") which adds trust, and every edit can be signed by an editor's DID, maintaining accountability.
- **Verified Credentials for Content Creators:** Content creators (bloggers, influencers) can have verifiable reputations. For example, a science blogger might obtain a credential from a science journalism society vouching for their expertise. When they publish an article about, say, COVID-19, they can attach a proof "I am accredited by ScienceJournalismOr [cheqd.io](https://sciencejournalism.org)7]. Readers or platforms can verify that without revealing the blogger's real identity if it's pseudonymous – it just gives a trust signal. On the flip side, state-sponsored trolls would not be able to obtain legitimate credentials, so their content would lack verification and could be algorithmically down-ranked or flagged by platforms.
- **Reducing Malicious Bots:** If social platforms required content to be signed by a DID (even an anonymous one, but rate-limited to what a person could manage), it raises the cost for bots to flood misinformation, because each fake account would need a unique DID and possibly some credentials to appear trustworthy. Platforms could choose to heavily prioritize content with certain verifications (e.g., human-verified). Users could opt to only see content that is from verified DIDs or that carries certain proof of source. This could significantly reduce the reach of misinformation bots that currently exploit the lack of verification.

Benefits:

- **Authenticity and Integrity:** It becomes much harder to pass off fake content as real. For instance, a fake press release purportedly from a government will not have the government agency's digital signature, so journalists and automated systems can quickly

invalidate it. The **tension between anonymity and credibility** can be managed by selective proof of attributes (like whistleblowers proving their insider status without revealing identity [chain.link1](#)).

- **Accountability:** Public officials or organizations can no longer easily deny statements they actually made – if it's signed with their DID, it's undeniably theirs (non-repudiation). Conversely, they are protected from false attributions (someone can't easily forge a statement in their name). This could elevate public discourse, as genuine statements are verifiable and fakes are exposed.
- **User Choice and Filtering:** End-users can get assistance from their software to filter information. For example, an email client might warn "This email claims to be from YourBank but is not digitally signed by YourBank's official DID – likely phishing." Or a messaging app might show "Forwarded many times – source unverified" vs "Directly from known source – here's their verified profile." This allows people to weigh information accordingly, reinforcing trust in verified information [businessthink.unsw.edu.au8](#)].
- **Resilience Against Deepfakes:** As deepfakes become more convincing, having an infrastructure of verification is crucial. If, for example, all official communications from leaders are expected to be signed (or broadcast on authenticated channels), then any video that isn't signed should be met with skepticism. The UNSW Business School opined that a decentralized identity framework could be a strong contender to combat deepfakes [businessthink.unsw.edu.au4](#)], because it provides a universal way to check if a piece of media comes from an authoritative source or not. DxD's approach of linking content to DIDs and signatures directly implements this idea.
- **Misinformation Deterrence:** If consumers and platforms start favoring content that comes with verifiable authenticity, producers of fake news will find their content has less impact. It shifts the information ecosystem from one of "anyone can claim anything" to "claims with proof get amplified." Over time, this could create a norm where serious information (news, important announcements) is expected to be signed/verifiable. Unsigned rumors would then be less trusted by default.

Example: Deepfake Scenario. A deepfake video of a CEO making a false announcement is circulated to manipulate stock prices. In a DxD world, the company has a DID and it routinely signs all its official video communications. Viewers (and trading algorithms) check the video's signature and find none – red flag. Simultaneously, the company's real announcement (or denial) is released, signed by their DID, which everyone can verify. The market thus trusts the signed statement and disregards the unsigned deepfake, minimizing impact. The false video is quickly flagged on social media as "Unverified source." This scenario shows how verified content can outcompete fake content in credibility and speed.

Another example: *Web of Trust for Fact-Checking*. A fact-checking organization could issue credentials to experts (say climate scientists have a credential as domain experts). When a piece of content about climate circulates widely, those with credentials could attach a quick fact-check verdict signed by their DID. Platforms could display “This statement about climate change has been evaluated by verified climate scientists – verdict: misleading cryptoslate.com4].” Because the scientists’ expertise is verifiable (via their credential), the audience has reason to accept their assessment. This adds a layer of trusted meta-information on top of content, enabled by the decentralized credential infrastructure.

In summary, DxID can play a pivotal role in strengthening the authenticity and reliability of information in the public sphere, thereby helping to **resist misinformation**. By attaching verifiable identity and credentials to content, we create an environment where truth can be cryptographically distinguished from falsehood, without relying on centralized gatekeepers, and while preserving necessary privacy.

zk-STARK Implementation Details in DxID

Under the hood, DxID’s privacy-preserving proofs are powered by zk-STARK technology. This section provides technical details on how these proofs are constructed and verified for the use cases above. We will outline:

- How identity and credential data are encoded into a STARK-friendly format.
- The typical **statements** that we prove (e.g., “age > 18” or “credential signature is valid and contains X attribute”) and how those are turned into arithmetic circuits or AIR (Algebraic Intermediate Representation).
- The proof generation process, including any optimizations or cryptographic libraries used.
- The verification process, including on-chain verification considerations.

While a full deep dive into STARK proving systems is complex, we focus on the aspects relevant to DxID’s application and any customizations made for identity use cases.

Representing Credentials and Identities in Circuits

Each verifiable credential in DxID has certain fields (attributes) and a digital signature (e.g., an RSA, ECDSA, or BLS signature by the issuer). To create a ZK proof about a credential’s content, we need to verify the issuer’s signature inside the proof and then apply constraints on the content.

Encoding Data:

- We choose a prime field (for example, a 256-bit prime field) that is large enough to represent the cryptographic elements we need (hashes, signature components). Commonly, STARKs work over fields like $\mathbb{F}_p$ where $p \sim 2^{61}$ or similar; but to simplify, we might use a binary representation or split big numbers into field elements.
- A credential's critical data (issuer DID or public key, subject DID or subject binding, attributes, signature) must be inputs to the circuit. We convert strings (like DIDs or names) to numeric values via hashing (e.g., using a collision-resistant hash like SHA-256). For example, the issuer's DID Document public key can be hashed to 256 bits and fed as a public input (so the verifier knows which issuer key is being used).
- An important consideration: verifying standard digital signatures (like ECDSA on secp256k1 or RSA) directly in a STARK circuit can be costly because those algorithms aren't naturally arithmetic-friendly in prime fields used by STARKs. There are a few approaches:
 1. Use STARK-friendly signature schemes for credentials. For instance, we could use EdDSA over a field that aligns with our STARK field. StarkWare's systems often use Pedersen hashes and ECDSA in a Stark-friendly manner by using twisted Edwards curves that map into their field. If issuers issue credentials using a STARK-friendly signature (like one based on Poseidon hash and EdDSA on BabyJubjub curve), it simplifies proof generation. However, issuers might be external who use standard PKI (X.509, etc.), so we need to handle standard algorithms too.
 2. Use hash-based verification: If the verifier of the ZK proof is also going to separately verify the issuer's signature (for example, on-chain one might simply require the user provide the issuer signature as part of public inputs and the contract checks it normally), then the STARK doesn't need to redo it. But that leaks information we might want to keep private.
 3. For SNARKs, there are known circuits for verifying RSA or BLS signatures. For STARKs, the proofs are larger, but it's still doable albeit heavy. Alternatively, one could incorporate a SNARK as a subprotocol for signature verification (a bit complex).

For our context, a practical approach is: **use hash commitments and external signature checks where acceptable**. For instance, the user could reveal the issuer's signature and the issuer's public key in plaintext to the verifier outside the ZK proof (since showing the signature doesn't necessarily reveal personal data, it just proves authenticity). Then the ZK proof focuses on proving "I possess a credential with attributes X that hashes to H". The verifier can check that H corresponds to a credential signed by Issuer. This splits the work: a normal signature check (fast, on real curve) plus a STARK for the secret bits. This approach was used in systems like

Hyperledger Indy, where the ZK proof proves knowledge of signature but the actual signature verification is done by the verifier's code with known public key – but that required a special signature scheme (CL signatures). We can adapt similarly: perhaps require issuers to use BLS or something where multi-credentials can be combined.

In summary, inside the STARK, we likely will:

- Take as a private input the credential data (including perhaps the signature or parts of it).
- If we avoid verifying the actual signature inside, we still need to ensure the credential wasn't tampered: we can include as a public input the hash of the credential's body and require the prover to show that hash corresponds to the private data they have. If the issuer's signature is separately provided on that hash and valid, we've tied it together. This sacrifices a bit of self-contained proof (the STARK proof alone doesn't prove authenticity, you need the actual signature to verify outside), but it simplifies things greatly.

Circuit for Age Example: As an example, to prove “age ≥ 18 ”:

- Private inputs: `birthYear` (from credential), and perhaps a signature or credential hash.
- Public inputs: `issuer_hash` (hash of issuer's public key or DID, as an identifier), and `currentYear` (or current date).
- The circuit can subtract `birthYear` from `currentYear` to get `age`. Then it checks a boolean constraint that `age  $\geq 18$` . This can be done by checking `age - 18  $\geq 0$` . In an arithmetic circuit, we'd likely represent `age - 18` as a number and use bit decomposition constraints to ensure it's non-negative (one way is to require it to equal some square, since in mod arithmetic a square can't be negative, but we have to be careful with field wrap-around—often a range proof technique is used, like ensuring no overflow on subtract).
- Alternatively, we convert birthdate to a timestamp and just check timestamp difference ≥ 18 years in seconds.
- The circuit would output a bit (1 if ≥ 18 , 0 if not) and we enforce it is 1 (or directly enforce the condition as a constraint).
- We also need to ensure that the `birthYear` we used truly comes from a credential issued by `issuer_hash`. If we have the credential's hash (including birthYear) as a private input, we can also have as public input a commitment that the issuer signed that hash. If signature verification is external, the verifier would only accept the proof if they

also see a valid signature from issuer on that same hash. So the trust flows: STARK proves “Given a credential with birthYear B, B yields age ≥ 18 and credential hash = H and issuer = X”, and outside the STARK the verifier knows “Issuer X did sign H”.

- Thus, the total proof to verifier is composite: {issuer’s signature on credential, issuer’s public key, STARK proof of age condition and consistency with credential hash}. This composite approach is somewhat a hybrid of ZK and standard crypto but is practical. It is similar to how ZKPs like zk-SNARKs have been used in some systems (the SNARK proves something about the plaintext, but the shielded data’s authenticity might still rely on an external signature).

Proof Generation and Performance

DxID’s proof generation will typically run on the user’s device (for selective disclosure proofs) or on a server in case of batch processing (like generating many content signatures). We likely utilize an existing STARK proving library:

- **Libraries:** Projects like StarkWare’s *Cairo* (a STARK VM) or *Winterfell* (an open-source STARK prover in Rust) can be leveraged. For example, Winterfell could be used to construct AIR for our identity statements. There is also ongoing work in proving systems like *ZK-STARK (libSTARK)* by SCIPR lab, etc. We choose one that is well-supported.
- **Example:** We might implement our circuits in a high-level language (like a custom AIR or using a RISC-V program to be proven). Suppose we use Cairo: we can write a Cairo program that takes as input the credential fields and outputs 1 if conditions hold. The Cairo program can also implement hash functions like Poseidon or Pedersen to handle commitments. StarkWare’s platform could then generate a proof (though StarkWare’s provers are not public for custom use without their platform).
- For independence, we might use Winterfell: it allows writing constraints and transitions directly. For example, verifying a hash preimage or doing a comparison can be set up in the AIR. We might not need many steps for a simple check (a few dozens steps).
- **Optimizations:** Since many credential proofs might share similar structure, we could reuse circuits. E.g., a generic “prove one attribute meets comparison” circuit, or “prove attribute equals X” can be parameterized. The heavy part might be signature verification if included; to optimize, maybe use BLS signatures because pairing checks can be done in a finite field, albeit at cost. Alternatively, the external signature check trick offloads that heavy part.
- **Performance considerations:** STARK proofs are larger (tens of KB). For an age check or similar small logic, a STARK proof might be maybe 50-100 KB. That’s still fine to transmit to a verifier (especially off-chain). Generation time might be on the order of

hundreds of milliseconds to a few seconds depending on the machine and optimization. This is acceptable for user interactions (e.g., scanning an ID for 2 seconds is okay). If it needed to run on a phone, we might need a mobile-optimized library or to restrict to simpler proofs. Perhaps some use cases (like content signing by a camera) wouldn't use ZK at all because privacy is not needed there; they'd just sign normally.

- **Batch proofs:** If a user needs to prove multiple things at once (like our earlier example of multiple credentials for a job application), it might be possible to aggregate into one proof to avoid overhead. STARKs can prove a batch of statements together with slightly increased size, but not linearly (some overhead per statement but shared setup cost). There's also possibility of using one proof to cover multiple credentials by treating them as multiple tape inputs in the AIR.
- **Post-Quantum Aspect:** The proofs themselves (the STARKs) use only hashes, which are quantum-resistant. But if our issuers use classical signatures (RSA/ECDSA), the trust in those is not quantum-resistant. However, that's a separate upgrade path: in the future issuers can shift to PQ signatures (like Dilithium). Our STARK circuits can adapt to verify those as needed. The important part is the identification proofs remain valid even in a post-quantum scenario as long as the credential signatures hold up or are replaced. Meanwhile, the STARK portion – including any commitments or random queries – is already PQ secure by design [[threeprotocol.ai2](#)].

Verification and On-Chain Verification

Verifiers, if they are software like a mobile app or website, will run the STARK verifier algorithm which involves a series of hash checks and polynomial evaluations. This is very fast (milliseconds) for typical proof sizes. We assume verifiers will use a library to do this. Because our proofs will likely be in a known format, we can standardize a verification API.

If the verifier is a smart contract on a blockchain, verifying a STARK proof on-chain is more challenging due to proof size and computation. However, some blockchains are integrating zk verification precompiles or custom VMs (e.g., StarkNet uses STARKs off-chain and verifies a succinct commitment on-chain). On Ethereum mainnet, verifying a large STARK directly is expensive (though projects like Herodotus have experimented with it). We have a few strategies:

- **Semi-off-chain verification:** The user provides the proof to an off-chain server or a layer-2 that verifies it and logs a result on-chain (like “DID X provided a valid proof of Y at time Z”). This result could be a simple boolean or a credential minted as an on-chain token/NFT. Then on-chain contracts can trust that logged result (perhaps because it's signed by a known verifier or because the L2 is secured). This is more of a centralized approach (trusting an oracle or committee).

- **SNARKing the STARK:** One could wrap a STARK proof in a SNARK to compress it for Ethereum verification. This is an advanced technique (use a SNARK to verify the STARK's Merkle tree in a succinct proof). Some projects do recursive proofs. This would allow on-chain verification at low cost but adds complexity and trusted setup if using SNARK. Given this is a deep research area, it might be future work.
- **Use of specialized chains:** If deployed on StarkNet or another ZK-rollup, verifying STARKs might be native or simpler. For instance, in StarkNet, contracts can call a verifier built into the system perhaps. Alternatively, using a different proof system (like zkSNARK) for on-chain might be chosen for practicality.

DxID's design is chain-agnostic; where on-chain interaction is required (like stamping a verified credential or gating a smart contract based on proof), one practical solution is to issue the user an on-chain verifiable credential (like an NFT badge) once they show the proof off-chain to a trusted issuer. For example, a DAO might require "prove you have a degree" – the user shows the zk-STARK proof to a DAO-run server, and the server mints an NFT to the user's address saying "Degree Verified". Then the user's on-chain address holds that token and can directly interact with DAO smart contracts. This avoids needing to verify the heavy proof on every contract call. It does introduce an off-chain step, but with a decentralized or multi-party oracle this can be trust-minimized. Some identity protocols (like iden3/Polygon ID) do similar: generate a proof off-chain, then register the fact on-chain via a trusted issuer contract.

In summary, for now:

- Off-chain verifiers (which covers most web and device use cases) will do normal STARK verification with the proof and known public inputs.
- On-chain enforcement will often rely on *derived credentials or attestations* to avoid heavy on-chain computation. For truly trustless on-chain verification, SNARKs might be more suitable until STARK verification costs drop or special support exists.

Code Snippet Example

To illustrate, here's pseudocode for generating and verifying a simple attribute proof (age example) using a hypothetical STARK library:

python

CopyEdit

```
# Pseudocode outline (not actual library calls)

from starkware import StarkProver, StarkVerifier
```

```

from did_credential import Credential, IssuerPublicKey

# Holder side (proof generation):

cred = Credential.load("alice_idcredential.json") # contains
birthdate and issuer signature

issuer_pk = IssuerPublicKey.load(cred.issuer)      # load issuer
public key (from DID resolution)

assert issuer_pk.verify_signature(cred.hash, cred.signature) # check
credential is valid


# Set up statement: Prove birth_year <= (current_year - 18)

birth_year = cred.attributes["birthYear"]

current_year = 2025

threshold = current_year - 18


# Construct trace or witness for STARK: in a simple scenario, one step
with birth_year and threshold

trace = [birth_year, threshold]

# Program: compute diff = threshold - birth_year; output bit = 1 if
diff >= 0 else 0.

# We need to prove bit == 1.

prover = StarkProver(program="diff_check_program") # program
pre-written to check non-negativity

proof = prover.generate_proof(trace, public_inputs=[threshold]) #
threshold as public input

```

```
# Send to verifier: (proof, issuer DID, cred.signature)
```

```
data_to_verifier = {
```

```
    "issuer": cred.issuer,
```

```
    "credential_signature": cred.signature,
```

```
    "credential_hash": cred.hash,
```

```
    "public_inputs": [threshold],
```

```
    "stark_proof": proof
```

```
}
```

```
send_to_verifier(data_to_verifier)
```

python

CopyEdit

```
# Verifier side (proof verification):
```

```
data = receive_from_holder()
```

```
issuer = data["issuer"]
```

```
issuer_pk = resolve_issuer_pubkey(issuer)
```

```
cred_hash = data["credential_hash"]
```

```
sig = data["credential_signature"]
```

```
# Verify issuer signature on credential hash
```

```
if not issuer_pk.verify_signature(cred_hash, sig):
```

```
    raise Exception("Credential signature invalid") # authenticity
fails
```

```

# Verify STARK proof for the age condition

proof = data["stark_proof"]

public_inputs = data["public_inputs"] # should be [threshold_year]

verifier = StarkVerifier(program="diff_check_program")

if not verifier.verify_proof(proof, public_inputs):

    raise Exception("Zero-knowledge proof failed")


print("Proof verified: Subject from issuer", issuer, "is over 18.")

```

In this pseudocode:

- The holder first independently verifies their own credential (this step isn't strictly necessary for the protocol, but the wallet might do it to ensure it doesn't generate proofs on a tampered credential).
- The holder then generates a STARK proof that `threshold_year - birth_year >= 0`.
- The holder provides the issuer's signature so that the verifier can check it normally.
- The verifier checks the issuer's signature (ensuring the credential was real) and then checks the STARK proof (ensuring the birth year in that credential meets the condition).
- The combination yields high assurance: the credential came from the trusted issuer and the attribute meets the policy, without the verifier learning the actual birth year.

A real implementation would involve more details (like including the credential hash inside the STARK proof to bind it, etc.), but this conveys the flow.

Trust Considerations in Implementation

It's important to note what the STARK proof covers and what it doesn't:

- The STARK proof can ensure the *integrity of revealed computations or comparisons on hidden data*, but if we rely on external signature verification, the verifier still needs to trust that external step. In the above example, if issuer's signature verification is done outside the ZK, we have to trust the verifier to do it (which they will, since it's in their interest).
- In a fully trustless ZK (where even issuer's signature is checked inside), the proof would directly state "There exists a credential signed by issuer with birth_year = X and $X \leq 2005$ " and the verifier would only know issuer and the truth of the statement. This is ideal from a privacy perspective (verifier learns nothing except the statement true), but as discussed, is heavier.
- Our hybrid approach leaks the credential hash and requires sending the signature, which might allow the verifier to deduce some info (like if they have a database of hashes corresponding to different birthdates or names, theoretically). To mitigate this, the credential hash could be salted or the credential could be of a type that doesn't allow easy reverse-engineering (if it's mostly personal data, a hash might be unique enough to identify the person if the verifier had seen the credential before). One solution: use a one-time nonce in the credential that the proof also accounts for, so that the hash is not deterministic across presentations. (Indy AnonCreds do something similar with link secrets).
- For many use cases, the small privacy leak of revealing a credential hash or even the signature is acceptable, given the alternative is handing over the entire credential. We drastically reduced data shared.

In conclusion, the zk-STARK implementation in DxID is a careful blend of:

- Efficient use of ZK proofs to hide personal data while proving required facts.
- Clever off-chain and on-chain integration to handle trust in issuer signatures.
- Use of existing cryptographic libraries and potential specialization (like choosing certain signature schemes) to optimize performance.
- A focus on user devices being able to generate proofs in a reasonable time (which is feasible for the scales of data involved in identity credentials).

This implementation approach ensures that the strong security properties of STARKs (transparency, post-quantum security, scalability) are harnessed in the identity context, delivering practical privacy to users.

Blockchain Integration and Smart Contracts

DxID operates at the intersection of off-chain credential management and on-chain verification. Here we describe how the system integrates with blockchains and smart contracts to achieve decentralization and persistence of identity data:

- **DID Registration on Blockchain:** how DIDs are anchored (e.g., on Ethereum or a consortium ledger).
- **Credential Revocation and Status Lists:** maintaining revocation registries or status checks using smart contracts.
- **On-Chain DID Documents and Resolvers:** storing or retrieving DID Documents and how this works in a decentralized way.
- **Smart Contract Verification:** how a smart contract might consume identity proofs or credentials (for access control, token distribution, etc.).
- **Example Integration Patterns:** e.g., issuing a token/NFT as a result of a successful off-chain proof (making the result usable on-chain), or using identity metadata in DeFi.

DID Registration and Resolution

DxID assumes a **DID Method “dxid”** that is implemented via a blockchain. This could be an Ethereum smart contract (for a public permissionless approach) or a dedicated DID ledger (like SideTree or Veres One for high throughput among consortium participants). For concreteness, let's consider an Ethereum implementation using a smart contract:

- There is a **DxID Registry Contract** deployed at a known address.
- To register a DID, a user calls `registerDID(did, publicKey, metadataURI)` (the contract might derive `did` from `msg.sender` or other unique info to avoid collisions). Typically, the DID unique string could be a hash of the initial public key to ensure uniqueness without central coordination.
- The contract emits an event and stores the mapping: `DID -> (publicKey, metadataURI, lastUpdatedBlock)`. The `metadataURI` could point to an off-chain DID Document if storing everything on-chain is expensive. For example, it could be an IPFS CID where the full DID Document JSON is stored (including service endpoints, multiple keys, etc.).

- Users can update their DID entry by calling `updateDID(did, newPublicKey/metadata)` with proper authentication (only the current public key owner can authorize changes, enforced by requiring a valid signature in the transaction or the tx coming from the key's address if using account-based identity).
- Resolution: Anyone (including off-chain resolvers) can call or query this contract to get the current public key and metadata for a DID. This feeds into the DID resolution process. For convenience, a standard DID Resolver library would be configured with the method `dxid` pointing to this contract's logic.

If using a ledger like Veres One or Hyperledger Indy, the concept is similar but with transactions on those networks. The W3C DID standard ensures that no matter the underlying ledger, a DID can be resolved to its document given the method's specified process.

Decentralization aspects:

- The DID Registry smart contract could be governed by no one (purely user-driven registration), or by a DAO for oversight in case of disputes over names, etc. For “dxid:” we can keep it permissionless: anyone can create a DID for themselves.
- For trust, others relying on a DID need to know the method is secure (e.g., on Ethereum, the security is tied to Ethereum's security). We might publish method specs.
- The use of blockchain means DIDs and their public keys are tamper-evident and censorship-resistant (assuming the chain's properties). If the contract is truly permissionless, no central authority can revoke or reassign your DID – only you can via your key.

Gas and cost considerations: Writing to Ethereum costs gas (perhaps a few dollars worth to register). This is acceptable for relatively static identities (you don't register often). Reading (resolution) can be done off-chain for free via Infura or similar, or via events.

Alternate models: Some DID methods avoid smart contracts by using blockchain addresses themselves as DIDs (e.g., `did:ethr:0xabc...` uses an Ethereum address as the identifier and a known contract to store a Document allowing rotations). We could also adopt that approach. For example, `did:dxid:ropsten:0x1234...` could mean an Ethereum address on Ropsten testnet. The contract known as Ethr-DID-Registry would store its doc (this is exactly the `did:ethr` method pattern). Using that existing standard might be wise, as it's already implemented. However, `ethr:` dids equate identity with a specific key (the address). DxDID might want to abstract so keys can change (`ethr` allows that via its registry contract where the address can update keys). So yes, `did:ethr` is a viable method we could adopt or mirror. Alternatively,

a newer emerging approach is using Sidetree (like ION on Bitcoin) which allows batching many DID operations in a single chain transaction.

For simplicity, we'll assume a straightforward contract-based registry.

Credential Revocation via Smart Contracts

When an issuer (like a university or DMV) issues credentials, they might need the ability to revoke them (e.g., a license is suspended). To handle this:

- The issuer can maintain a list on-chain. E.g., a mapping from `CredentialID` -> `revokedStatus`. The credential ID could be a hash or a UUID that's also inside the credential (so it's known to holder and verifier).
- The issuer, via their DID keys, can update that mapping (setting `revoked` = true).
- This smart contract list can be public for verifiers to check. Alternatively, for privacy, the issuer could publish a hash of revoked credentials in an accumulator and update that (Hyperledger Indy does something akin to this with accumulators).
- In DxDID, we can use a simpler approach where each credential might include the ID of a revocation registry and an index, and the verifier either queries the contract or expects the holder to provide a proof of non-revocation (like a Merkle proof if the registry is published as a Merkle tree).
- Concretely: If a credential has ID "XYZ", the verifier could call `RevocationRegistryContract.isRevoked(issuer, XYZ)` which returns true/false. If true, reject the proof. The STARK proof can be crafted to reveal the credential ID and issuer DID to allow this check without revealing more about the credential.

This does leak that "credential with ID XYZ was checked". Depending on how identifiers are structured, that might be fine or might allow correlation. One mitigation is to use ZK to prove non-revocation too: for instance, there are known ZK schemes where the prover shows "I have a credential that is not in the revoked set as of now" by using accumulator witness. Systems like Idemix/AnonCreds incorporate that in the ZK proof so the verifier doesn't see the actual credential ID at all. That's more complex but achievable. In a first iteration, a straightforward check might suffice given revocation events are not too frequent.

- The revocation registry itself can be simply a list of hashes. Issuer adds "XYZ" to list by a transaction. Verifier checks membership. This is transparent but could expose who got credentials (if IDs are guessable). To mitigate that, credential IDs could be random UUIDs so outsiders can't easily enumerate who's revoked (though the issuer obviously

knows which they revoked).

- On-chain revocation is beneficial because it's globally visible and doesn't depend on querying issuer's servers (which might be down or biased).

On-Chain Credential Storage (optional)

We generally avoid storing full credentials on-chain for privacy and cost. However, there are cases where a user might want to make a credential public (like a business could publish its license certificate for anyone to see). They could simply do that by putting the certificate hash or data on-chain. For example, a company might register a verifiable credential of compliance to a public contract (essentially notarizing it). Then anyone can just fetch it from the chain. But for personal credentials, we definitely keep them off-chain.

Smart Contracts Consuming Identity Data

Some smart contracts might directly integrate identity requirements:

- **Access Control:** A contract (like a token sale contract) might restrict participation to verified users. It could require the user's address to have a certain NFT or credential token (meaning they completed off-chain KYC). This is a pattern where after off-chain verification, an on-chain token is granted (Soulbound token or similar) which the contract checks (`require(balanceOf(msg.sender) > 0)`). This way, the heavy identity checks are done off-chain but the result is enforced on-chain with a lightweight check. For instance, a "accredited investor" NFT after the user proves net worth off-chain can gate a private investment contract.
- **Attribute-Based Conditions:** More granular, a contract could use zero-knowledge proofs directly. Some projects like Dark Forest (a game) have used zkp for proof-of-knowledge of secrets. We could imagine a contract that accepts a proof via a transaction (e.g., user posts a proof as transaction data, contract verifies it with a precompiled or via a SNARK). For example, a governance contract might allow one vote per human: it could require a proof that "this address corresponds to a human with unique ID" using some ZK scheme without revealing identity. There's research in "Proof of Personhood" (BrightID etc) that might be integrated.
- At present, directly verifying complex identity proofs on mainnet is not efficient. But on certain ZK-rollup chains, it could be. For example, on StarkNet, one could foresee verifying an attestation proof since that environment is designed for STARKs. In general Ethereum, one might lean on SNARK-friendly techniques due to available precompiles (like `alt_bn128` pairing for zkSNARK verify). So a practical route is: convert an identity STARK proof to a Groth16 SNARK (via a circuit aggregator or at proof time do SNARK

of STARK). This is advanced, likely beyond initial scope.

Therefore, a more straightforward pattern for blockchain integration is **Verifiable Credentials -> Verifiable Claims Tokens**:

1. User proves something off-chain to an authority or the contract owner (like a DAO committee).
2. The authority mints an ERC-721 or ERC-1155 token to the user's address encoding the claim (could be in token metadata or by using different token IDs for different claims).
3. Any contract that needs that claim checks for the token. Because token transfers are transparent, we ensure they are non-transferable if it's personal (use Soulbound tokens or simply only allow mint and burn, no transfer).
4. If the claim should expire, the token can have an expiration or the authority can revoke (burn) it if needed.

This approach is being explored (e.g., POAP tokens for attendance, or SBTs for credentials as proposed by Ethereum researchers). It shifts trust slightly (you trust the issuer of the token similarly to trusting a credential issuer), but it leverages smart contract enforceability easily.

Example Integration: A lending dApp wants to give better rates to users who can prove a high credit score. A credit bureau DID can issue a credential "CreditScore > 700" to the user via DxDID. The user goes to the dApp's website, proves this to the bureau (or via an interactive proof that the bureau verifies), and the bureau (or the dApp itself if authorized to see the proof) mints a "Gold Borrower" soulbound token to the user's Ethereum address. The lending smart contract, when user borrows, checks `require(GoldBorrowers.balanceOf(msg.sender) >= 1)` to apply a better rate. If the user's score falls later, the bureau could theoretically revoke by instructing the contract to burn the token (this needs the contract to allow that function for the issuer's address). This achieves dynamic on-chain behavior from off-chain private data.

Cross-Chain and Interoperability

DxDID's blockchain integration can be multi-chain. DIDs are globally unique, so even if anchored on one chain, they can be used on another. For example, `did:dxid:...` might be primarily on Ethereum, but a Polkadot or Solana app could still resolve it by consulting Ethereum (via a bridge or oracle). There are emerging decentralized DID resolution networks to handle that. Alternatively, separate DID methods per chain can be used, but that complicates identity portability.

Verifiable credentials themselves are chain-agnostic data objects. They can be stored in IPFS, exchanged peer-to-peer, or even embedded in QR codes. The blockchain is mostly used for

discovery (DID to key) and revocation status. It's possible to use DxID without *any* blockchain if, say, all trust is out-of-band (like scanning QR codes offline and trusting offline exchanges), but the blockchain components add robustness (e.g., if Alice just shows Bob a signed credential and Bob trusts the issuer's printed public key, that works one-to-one without chain; but to do it globally at scale, a ledger of public keys and revocations is needed).

Example: Integration with IoT and AI (Preview)

As a segue into the next integration section: suppose IoT devices have DIDs and sign data (as discussed in content authenticity and supply chain). A smart contract that manages a decentralized IoT network could require device signatures on telemetry. Using DxID, those device DIDs are in a registry (so the contract knows which device IDs are valid), and devices might even hold on-chain permission tokens representing manufacturer approval. This ties identity into IoT device management.

For AI: consider AI agents on-chain (DAO-controlled bots) that need identities to prove certain training data or compliance (like an AI that can prove via a zk proof it was trained on licensed data, and a contract will only pay it if such proof is present). These are speculative but give a sense that identity constructs and ZK proofs can extend beyond people to entities like AI bots, and smart contracts may gate interactions with them based on those proofs.

In summary, blockchain integration provides the trust substrate for DxID:

- DIDs give a decentralized identity anchor.
- Smart contracts allow embedding identity verification into on-chain logic, albeit often via creative architectures (like issuing tokens for verified users).
- Revocation and credential status can be managed collectively on-chain for transparency.
- Over time, as ZK verification on-chain improves, we might directly verify more proofs on-chain, reducing reliance on off-chain attestations.

DxID remains flexible: it can work entirely off-chain for human-to-human verification, and optionally leverage on-chain components when decentralized apps or public verifiability are required.

Comparison to Existing DID Standards and Systems

Decentralized identity is an active field, and several standards and networks exist. Here, we compare DxID's approach (zk-STARK-powered, privacy-centric DID system) with some notable existing decentralized identity systems:

- **Sovrin / Hyperledger Indy (AnonCreds)**
- **uPort / ERC-1056 / Ethr-DID (Ethereum-based identity)**
- **Veres One (Independent ledger for DIDs)**
- *(plus mention other relevant systems like DID on Bitcoin (ION), and emerging standards like W3C DID & VC themselves, and how DxID aligns with them.)*

We focus on differences in architecture, privacy features, trust model, and maturity.

Sovrin / Hyperledger Indy (AnonCreds)

Overview: Sovrin is a public-permissioned decentralized identity network based on Hyperledger Indy. It introduced the concept of **AnonCreds** (Anonymous Credentials) which allows selective disclosure and proof of credentials using CL-Signatures (Camenisch-Lysyanskaya signatures) – a form of zero-knowledge proof purpose-built for credentials. In Sovrin/Indy:

- DIDs are recorded on a ledger (Indy ledger) run by known stewards (permissioned validators).
- Credentials are issued off-chain and stored by the user. They use a specific credential schema and signature (CL signature) that supports selective disclosure and a proof algorithm (similar to how DxID uses zk-STARKs, Indy uses a ZK proof protocol optimized for those sigs).
- Indy emphasizes pairwise DIDs: every relationship can use a unique DID to avoid correlation [info.suresync.nl9].
- Revocation is handled via accumulators on the ledger for each credential schema (users present a proof their credential's revocation registry index is still valid, using ZK).
- Privacy: Indy was built with privacy in mind; the verifier learns only the disclosed attributes or predicate proofs (like >18) and nothing else. This is very much in line with DxID's goals. However, Indy's cryptography is older and specialized (not general-purpose like STARKs). It uses large RSA moduli and the proofs have certain limitations (e.g., all credentials must come from Indy network issuers and follow Indy format).

Comparison to DxID:

- *Network Model:* Sovrin is **public-permissioned** (trusted stewards operate node [info.suresync.nl1]). DxID aims to be **public-permissionless** (anyone can run a DID

registry on Ethereum or a similar public chain, no central governance needed except maybe ensuring uniqueness). This means Sovrin has some governance and potentially censorship ability by stewards, whereas DxID inherits the underlying chain's decentralization (Ethereum, etc., with proof-of-stake or work).

- *Credential Privacy*: Both systems support selective disclosure. Sovrin's AnonCreds is proven tech for that: e.g., a user can prove attributes without revealing the credential info.suresync.nl/3. DxID's use of zk-STARKs achieves a similar outcome but with a different crypto backend. One advantage of STARKs: no trusted setup and post-quantum security, whereas CL-signatures (based on RSA and discrete log) are not PQ-safe.
- *Scalability*: Indy's anoncreds proofs are relatively efficient for the sizes of credentials (proofs are on the order of a few kilobytes). STARK proofs are larger (tens of KB) but still quite manageable in most contexts, and STARKs can handle proving more complex logic than CL can. Sovrin's ledger itself is optimized for high throughput of DIDs and revocations, but being permissioned, it doesn't reach Ethereum-level global decentralization.
- *Revocation*: Both handle it via cryptographic accumulators. Indy's approach is quite sophisticated; DxID could implement similar or use simpler on-chain flags at cost of privacy.
- *Ease of integration*: Sovrin/Indy has its own stack and isn't EVM-compatible or widely used in mainstream apps now (used in specific SSI wallets). DxID building on W3C standard credentials and existing blockchains might integrate more easily with web services and smart contracts.

Notably, Sovrin and Hyperledger Aries have produced a rich set of protocols (like did-exchange, presentation exchange) that DxID can reuse at a conceptual level. The main difference is the crypto: Sovrin uses **Idemix/AnonCreds** ZK, DxID uses **STARKs**. Functionally, they aim to provide the same privacy guarantees (Sovrin proved that 5+ years ago, at production scale for some pilot projects), but DxID extends this by:

- Removing the need to trust a specific ledger's governance (we can deploy on Ethereum or any chain).
- Being potentially more flexible (any credential format, not just the specific schema Indy requires).
- Being future-proof in security (transparent and hash-based vs. Indy's needing trust in credential schema parameters generation – though Indy's “issuance by default” involves trust in the issuer not colluding with others to correlate, etc).

So, **DxID vs Sovrin**: Both champion privacy; Sovrin was pioneering but is a somewhat closed ecosystem; DxID strives to be open and modular, using modern ZK that can prove arbitrary statements and integrate with modern blockchains. One source notes Sovrin's strong data minimization focus via agents and unique DIDs info.suresync.nl/1], which DxID similarly achieves but perhaps with less complexity (Sovrin's use of multiple agents is powerful but heavy).

uPort / Ethereum DID (Ethr-DID)

Overview: uPort (now mostly evolved into the ERC-1056 Ethr-DID standard and the Veramo SDK) was one of the first Ethereum-based self-sovereign identity solutions. It allowed Ethereum addresses to act as DIDs and store identity data on Ethereum or IPFS.

- A user's Ethereum address served as their identifier; a smart contract (ERC-1056 registry) could store a DID document (keys and service endpoints) so they could rotate keys or add authentication methods.
- uPort initially had a mobile wallet that could send verifiable credential-like JSON Web Tokens. These were simpler (often just "claims" signed by one's private key or an issuer's key, using JWT standards).
- Privacy: uPort did not originally focus on zero-knowledge or selective disclosure. If you got an attestation (say "Alice is over 18" as a JWT from an issuer), you could present it, but it was an all-or-nothing disclosure (though the claim might itself be minimal, like just the over-18 flag). Ethereum DIDs are often one per context to avoid correlation, but in practice many used a single DID which can be correlatable across uses if reused.
- There was no built-in ZKP support; however, one could incorporate zk-SNARK proofs as part of a claim if desired. The approach was more straightforward: rely on the fact that not everything is on-chain, but correlation could happen if the same DID is used everywhere (uPort recommended unique DIDs per application later to mitigate this, similar to Sovrin's approach).

Comparison:

- *Decentralization:* uPort/Ethr-DID is permissionless on Ethereum – good. It relies on Ethereum addresses as root identities (so ultimately backed by Ethereum's decentralization).
- *Privacy:* Ethr-DIDs themselves don't provide anonymity – an Ethereum address can be tracked. uPort's approach to privacy was basically to use pairwise addresses (create a new address/DID for each connection info.suresync.nl/1], and to not publish any unnecessary data on-chain (attestations were usually shared off-chain via direct communication, not posted on chain). But there was no cryptographic hiding of

attributes: if an issuer gives you a JWT with your birthdate, showing it reveals that birthdate.

- *Use of ZK:* None inherent. In contrast, DxID bakes ZK into the core, enabling attributes to be proven without revealing them. uPort couldn't do things like predicate proofs without custom protocols or turning to ZK circuits externally.
- *Credential Format:* uPort used JWT (JSON Web Token) or JSON-LD with secp256k1 signatures for credentials. These are easy to generate and verify (just signature checks), but they lack the selective disclosure of CL or ZK proofs. DxID uses W3C Verifiable Credentials too, but extends with ZK for presentations.
- *Network and adoption:* uPort was an early product; it influenced standards but is not widely used now. Ethr-DID (ERC1056) is used behind the scenes in some identity wallets (e.g., SelfKey, Bloom, etc.). Many Ethereum-based identity solutions ended up focusing on attestations bound to addresses and didn't solve privacy strongly (some rely on trusted third parties to attest things and user reveals attestation when needed, similar to showing a digital certificate).
- The **Veramo** framework and Ceramic network (for DIDs) are newer evolutions in Ethereum community, but again, privacy often means "don't put it on a public chain, exchange it P2P" rather than "use ZK to hide it".

DxID vs uPort/Ethereum-DID: DxID is essentially adding the missing privacy layer on top of what uPort started. It's like combining the Ethereum DID approach (open, on public chain) with the privacy of Sovrin. One could say DxID = Ethr-DID + ZK proofs (and using STARK for those proofs). The advantage: no reliance on a special-purpose ledger (use Ethereum's security), and no reliance on JWTs that reveal full info (use ZK to minimize info). However, one trade-off: verifying STARK proofs is heavier than verifying a simple signature on a JWT. But as discussed, that's manageable off-chain, and on-chain can be handled by issuing an ERC-721 token or something as a result.

A cited comparison: "Uport's relatively simple way of signing claims makes them easy to look up and verify. However, Uport did have shortcomings in data minimization... requires Ethereum transactions that can be correlated via funding accounts^{info.suresync.nl¹}]. This highlights that uPort had the ease-of-use but at the cost of potential correlation and not completely meeting privacy ideals. Sovrin had better privacy but in a permissioned blockchain. DxID aims to achieve the privacy of Sovrin with the openness of uPort.

The SureSync blog explicitly noted: Uport is more decentralized (permissionless Ethereum) but had privacy issues (correlation through funding, etc^{info.suresync.nl¹}], whereas Sovrin is more privacy-preserving but runs on a permissioned network^{info.suresync.nl³}]. DxID tries to take the best of both.

Veres One

Overview: Veres One is a public-permissionless blockchain launched by an identity community (including Digital Bazaar). It is specifically built for DIDs.

- Veres One DIDs ([did:v1](#)) are created on this network. The network doesn't use crypto tokens; it's a pure identity utility.
- It focuses on being **globally interoperable**, **cost-effective** (low fees), and **governed by stakeholders** for identity.
- Privacy: Veres One's DID method supports all W3C DID features, but it doesn't inherently provide ZK proofs – it's more about the DID resolution and key management. Actual credentials and presentations can use any W3C VC mechanism (so one could use ZK or not, up to the implementers).
- They emphasize *privacy by design* (minimize data on chain, give control to user over sharing [veres.one9](#)). But they don't have a built-in Anonymous Credentials scheme like Sovrin's (Veres One expects you to possibly use ZK or just selective disclosure JSON-LD proofs).
- In terms of decentralization, Veres One is permissionless (anyone can run a node) and doesn't rely on any coin (to avoid volatility [veres.one0](#)). It's funded by nominal fees that go down as network scales, governed by a non-profit consortium.

Comparison:

- *Permissionless ledger:* Both Veres and an Ethereum-based DxD are open networks. Veres One is purpose-built for identity so it might have performance benefits (no gas wars with DeFi etc., stable costs). Ethereum is more widely adopted but has unpredictable fees.
- *Token vs No-Token:* Veres One avoids a cryptocurrency, which some enterprises prefer. Ethereum-based DID uses Ether for gas, which introduces regulatory complexity sometimes and volatility. However, L2 solutions could mitigate cost issues for Ethereum identities.
- *Privacy and ZK:* Veres One by itself doesn't implement a specific ZK scheme; it relies on the VC layer. DxD explicitly integrates STARKs for privacy. One could certainly implement STARK-based proofs on top of Veres One as well – they're not mutually exclusive. So a difference is more in emphasis: Veres One solves decentralized key management and discovery, leaving credential privacy choices to users (they can use ZK if they want, but no default). DxD is packaging an opinionated approach to use ZK

by default for presentations.

- *Adoption and Standard Alignment:* Veres One folks are heavily involved in W3C DID and VC standards. DxID aligns with those standards too (we assume W3C DID/VC format compliance). So conceptually, an identity on Veres One and on DxID should both be resolvable as DIDs and use similar credential formats. The main difference might come in how proofs are generated/verified (Veres might typically see LD-Proofs or BBS+ signatures for selective disclosure, etc., whereas DxID uses STARKs).
- *Performance:* Since Veres One is optimized for identity, it might handle DID operations more efficiently than a general chain. But performance of verifying proofs or exchanging credentials mostly happens off-chain in both.

One could integrate the two: for instance, use Veres One as the DID ledger for DxID participants, and still use STARKs for proofs. That's feasible if one wants the advantages of both (Veres's stable identity infra and DxID's advanced ZK). So these aren't strictly competitors but alternative choices in architecture.

Sovrin vs uPort vs Veres vs DxID Summary:

Let's tabulate some key features for clarity:

Feature	Sovrin / Indy	uPort / Ethr-DID	Veres One	DxID (Our approach)
Ledger Type	Public permissioned (trustees operate node info.suresync.nl 1]	Uses Ethereum (public permissionless) for DI info.suresync.nl 1]	Public permissionless dedicated ledger (no token w3c-ccg.github.io/veres.one 2]	Uses existing public chains (e.g. Ethereum) permissionless
Decentralization	Federated governance by Sovrin Foundation stewards	Fully open (any Ethereum user can have DI info.suresync.nl 3]	Open participation (nodes run by anyone; governed via community	Fully open (inherits decentralization of chosen blockchain)

rule[veres.one](#)
[es.one](#)7】

DID Method	did:sov / did:indy (ledger-specific DIDs)	did:ethr (Ethereum addresses) or did:uport (now deprecated)	did:v1 (Veres One metho w3c-ccg.git hub.io 1】	e.g. did:dxid (method on Ethereum or others, similar to ethr)
Credential format	AnonCreds (CL-signature based credentials)	JSON Web Tokens or JSON-LD with secp256k1 sigs	JSON-LD Verifiable Credentials (typically LD-Proofs)	W3C Verifiable Credentials (JSON-LD or JWT)
Selective Disclosure	Yes – built-in via ZK proofs of CL signature info.sures ync.nl 3】	No inherent (JWTs share all claims; need multiple tokens for selective info)	Partial – can use BBS+ signature for LD-Proofs to selectively disclose, but not default	Yes – via zk-STARK proofs of disclosed conditions or attributes
Zero-Knowled ge Tech	Uses ZK for proofs (CL-signature protocol) (trusted setup for CL params)	None by default (could add SNARK externally)	None by default (depends on credential format chosen, can integrate ZK)	Native use of zk-STARK for all proofs (no trusted setup)
Post-Quantu m	No (uses RSA / classical crypto)	No (uses secp256k1 ECDSA)	No (currently standard crypto, but can adopt new ones)	Yes for proof part (hash-based STARK); underlying ledger (Ethereum) not

PQ yet, but identity proofs are PQ
[securitythreeprotocol.ai7】](#)

Privacy & Anti-Correlation

High – unique DIDs per relationship, agents to proxy interaction[info.suresync.nl9】](#)

Medium – recommended use of multiple addresses, but users often reused one (correlation risk)[info.suresync.nl1】](#)

Medium/High – DIDs can be created freely, no global correlation unless same DID reused; credentials can be exchanged off-ledger

High – users can use unique DIDs, plus ZK proofs mean even if same DID used, minimal data revealed

Revocation

Supported via accumulator on ledger (AnonCreds Revocation)

Could put revocation status on Ethereum (e.g., smart contract list or just revoke token) – not standardized in uPort but possible

Supported – Veres One DID Document can list status endpoints, and they propose credential status lists (e.g., via DID or [URdiddirectory.com3】](#)

Supported – via on-chain registry or ZK proof of non-revocation, similar models (e.g., check in proof or via contract)

Integration

Custom stack (Indy SDK, Aries protocols) – less mainstream dev tooling

Uses standard web tech (JWT, Ethereum) – easy for blockchain devs, mobile support via uPort app (now evolved to other wallets)

Standard DID/VC – integration via libraries (Digital Bazaar’s libraries). Good for web because JSON-LD, but JSON-LD can be heavy for some devs

Standard DID/VC + ZK – needs specialized ZK support (our libraries), which is newer for devs. But conceptually

aligns with W3C standards.

Maturity & Usage	Production networks (Sovrin MainNet/BuilderNet), used in niche SSI deployments (e.g., government pilots)	uPort is deprecated; Ethr-DID widely implemented in libraries like Veramo; some enterprise use for simple identity, no major public adoption as unified system	Veres One launched (~2019), used in some pilots for universal DID (e.g., by govt of British Columbia in experiments). Not as famous as Ethereum or Sovrin but active in standards.	DxID is a new combination – leverages mature components (DIDs, VCs, STARKs) but as a whole, not deployed yet. This document is effectively proposing it.
-----------------------------	--	--	--	---

(Table: Comparison of Sovrin, uPort/Ethereum DID, Veres One, and DxID.)

Looking at the table:

- Sovrin shines in privacy but is a closed network for writing (open for reading).
- uPort was user-friendly but weaker in privacy (one reference said “Uport does not completely meet data minimization criteria” due to correlation issueinfo.suresync.nl1]).
- Veres One aims to be a general-purpose public utility for DIDs (with privacy by not sharing data unless neededveres.one3]).
- DxID’s unique value is the integration of a general-purpose *zero-knowledge proof layer* (STARKs) with the identity framework, on a widely-used public chain, aiming for the best of both worlds (privacy + decentralization).

Security and Trust Model Differences

- **Trusted Setup:** Sovrin’s CL-signature system doesn’t require a trusted setup for each credential issuance, but the credential definition includes a public key from issuer which if issuer cheats with correlating attributes across credentials, privacy can break (that’s an issuer trust issue, not system). zk-STARK (DxID) requires no trusted setup at all for the proof system. uPort/Ethr and Veres rely on standard signatures (no ZK).

- **Network Attacks:** Sovrin being permissioned could be more resilient to Sybil (since stewards are known orgs) but also could be a target for collusion. Ethereum (DxID) and Veres being permissionless face potential Sybil, but Ethereum's economic security is high, and Veres uses its own consensus (tolerating presumably a certain % of malicious nodes). But for DID resolution, even if a network is attacked, worst case an attacker could prevent resolution or spoof a DID Document if they had majority (Veres) or 51% (Ethereum) – which is quite difficult on Ethereum. Sovrin's trust in stewards might be considered more centralized by some, though it's decentralized in governance.
- **Usage Scope:** Sovrin/Indy was very SSI-specific, with custom wallets. Ethereum solutions integrate with general blockchain wallets (e.g., MetaMask can sign a message that becomes an "attestation"). DxID aims to allow normal crypto wallets plus a ZK module.
- **Interoperability:** All are converging on W3C standards, so credentials from one might be theoretically translatable to another. Indeed, an Indy AnonCred could be transformed into a W3C VC (some work is happening on that), and Ethereum-based credentials can be expressed as W3C too. Our DxID uses W3C VC, so it should interoperate at that layer with others (except the proof format might differ – e.g., an Indy proof is not understandable to a Veres-focused verifier unless they specifically support Indy's protocol, and vice versa).

In conclusion, **DxID distinguishes itself by its strong emphasis on built-in zero-knowledge privacy and use of a transparent proof system, while aligning with open standards and infrastructure.** It addresses the gaps identified in earlier systems:

- As noted by comparisons: uPort lacked complete data minimization info.suresync.nl/1 – DxID fills that with ZK.
- Sovrin had great privacy but is not permissionless – DxID uses permissionless networks.
- Veres One is neutral on ZK but heavy on DID utility – DxID can complement that with a ZK layer.
- And unlike earlier systems that often either ignored the on-chain world (Sovrin largely separate from smart contracts) or ignored privacy (many Ethereum identity projects), DxID bridges off-chain privacy with on-chain integration.

By learning from these systems (Sovrin's crypto, uPort's usability, Veres' standardization), DxID aims to push the state of the art forward, combining their strengths and avoiding their weaknesses.

Security Considerations and Post-Quantum Resilience

Security is paramount for an identity system. We analyze DxID's security properties and benefits:

- **Privacy Security:** how well it protects user data (ensuring zero-knowledge really leaks nothing identifiable).
- **Authenticity and Integrity:** ensuring that credentials and proofs cannot be forged or altered without detection.
- **Resistance to Attackers:** including malicious issuers, verifiers, or blockchain attackers.
- **Post-Quantum Security:** why DxID is robust against future quantum adversaries, ensuring longevity of the identities.
- **Comparison of ZK-STARK vs alternative crypto (like SNARKs) in terms of security.**

Privacy and Confidentiality

DxID is designed so that users disclose the minimum possible information:

- Using zk-STARK proofs, a user can convince a verifier of a statement without revealing the underlying personal data. This provides **cryptographic confidentiality**. As long as the STARK protocol's zero-knowledge property holds (which it does, given proper use of randomness and blinding in the protocol), the verifier learns nothing beyond the truth of the statement [curiy.medium.com7](https://www.curiy.medium.com/7)].
- The system avoids publishing personal data on any public ledger. DIDs and perhaps credential hashes might be on-chain, but those by themselves are not sensitive (though we consider correlation below).
- **Correlation resistance:** The use of pairwise DIDs and not reusing them broadly limits the ability of observers to correlate activities across contexts. Even if a single DID is reused, because the user can prove statements without revealing which credential (only that one exists), verifiers cannot easily link the user across different proof presentations. For instance, if Alice uses the same DID to prove over-18 to a game site and to a wine shop, and if both just get a "true" proof and see her DID, they might know the same DID did both. If that is a concern, she should use different DIDs. But even if not, the actual personal data (like name or birthdate) was not exposed for correlation, only an abstract identifier. Still, correlation via DIDs is something to manage (by rotating DIDs or including a mechanism in proofs to hide the DID – perhaps by proving knowledge of a secret tied

to the DID rather than showing the DID).

- The **Sovrin vs uPort** analysis in the SureSync blog indicated that permissionless networks can lead to correlation through public activity (e.g., funding multiple uPort identities from one account could correlate the info.suresync.nl ¹). DxID mitigates that by encouraging not linking funding or usage across identities. Also, since proofs don't require on-chain transactions (the presentation is off-chain), using multiple personas doesn't incur extra on-chain cost except perhaps an initial DID registration which is minor.

In summary, if best practices are followed (unique DIDs per relationship or at least per credential type, using ZK proofs for all presentations, and careful handling of any public pointers like credential IDs), DxID provides a very high level of privacy – comparable to or exceeding Sovrin's approach, and far better than typical web login or certificate systems.

Integrity and Authentication

DxID builds on tried-and-true digital signature principles for authentication:

- Every credential is digitally signed by its issuer. This ensures **credential integrity** – if anyone tries to alter the content (like changing an expiration date or attribute), the signature check will fail. As a verifier, you only trust credentials from issuers you consider trustworthy (e.g., government, bank, etc.), similar to how you trust certain certificate authorities in TLS.
- All DID Documents (and thus public keys) are anchored in a tamper-resistant ledger. This prevents an attacker from impersonating someone by publishing a fake key: any such attempt would either be impossible (if DID already taken) or detectable (if they create a new DID, it won't match the trusted one the verifier expects for that entity).
- The zk-STARK proof itself has cryptographic integrity: it includes commitments to the secret data and random challenges that bind the prover. It's computationally infeasible for a prover to create a proof of a false statement [chain.linkchain.link9](#) ⁹ (soundness property). For instance, if Alice is 17, she should not be able to produce a *(continued)*

If Alice is 17, she cannot forge a proof that she is 18 – the STARK soundness guarantees the verifier won't be fooled by a false proof [chain.link79](#) ⁹. Thus, **no one can fake or tamper with a proof or credential without detection**. This holds under the usual cryptographic assumptions (hash functions are collision-resistant, etc.).

- **Authentication of Presentations:** When a user presents a proof, we include a signature or DID binding (like the user signs the verifier's challenge with their DID key) to prevent a malicious party from reusing someone else's proof. This ensures **proofs are**

tied to the session and cannot be replayed by an eavesdropper. Essentially, the verifier knows the proof came from the legitimate holder of the credential (since only they could produce it and also sign the session challenge).

- **Issuer Trust:** The system doesn't magically create trust in issuers – one must still trust that an issuer's credential content is accurate (e.g., the DMV truly verifies eyesight before putting it on a driver's license). What DxID does is ensure that if you *do* trust an issuer, you can reliably accept credentials from them without fear of forgery. And through selective disclosure, you trust that even though you aren't seeing the whole credential, the parts you care about are proven. This is analogous to trusting a passport – you trust the government to have checked the person's identity; the difference here is you might only see “over 18 = yes” rather than the full birthdate, but since it's cryptographically derived from a government credential, the trust is the same.

Threats:

- *Malicious Holder:* A user cannot lie beyond what their legitimate credentials allow. They also cannot use someone else's credentials because they won't have the corresponding keys or secret values (e.g., CL-signature credentials have a secret “link secret”; similarly, we can enforce user is holder by requiring knowledge of a private DID key or a random nonce in the credential known only to holder).
- *Malicious Issuer:* An issuer could collude with a verifier to try to track a user by encoding correlation bits in multiple credentials (this is a known SSI issue – e.g., issuing two credentials with the same “master secret” could let colluding verifiers correlate). DxID can mitigate this by user-generated secrets: in many systems, the user contributes a random nonce that the issuer includes in the credential (Indy does this) so that even the issuer cannot correlate that user's credential across different contexts. We would similarly ensure that if a user has multiple credentials from one issuer, they are unlinkable unless the user deliberately links them in a proof. This maintains **unlinkability**, a key privacy info.suresync.nl39].
- *Malicious Verifier:* A verifier might try to extract more info by, say, requiring the user reveal more than necessary or by mounting traffic analysis. With DxID, the user is in control of what they prove. They should not present full credentials if not needed. A verifier could refuse service if a user doesn't give up data – that's more of a policy issue than a security flaw. But technologically, DxID gives the user the tools to minimize data given. Also, proofs are usually done directly between user and verifier (or via an encrypted channel), so eavesdroppers cannot glean info. If a verifier tries to share a user's proof with someone else, that proof typically can't be verified by a third party (it might be tied to the verifier's challenge, and also reveals very limited info anyway).
- *Ledger Attacks:* If the blockchain/DID registry is attacked (51% attack or corrupted validators), an adversary might try to rewrite a DID Document (e.g., replace someone's

public key to impersonate them). On a robust network like Ethereum, this is extremely difficult. On smaller networks, one mitigant is that participants can pin or cross-verify DID Documents (for example, maintain caches of trusted contacts' DIDs or require proofs of DID integrity from multiple sources). Additionally, most verifiers will have some out-of-band trust for major issuers (like they might have the government's DID or public key on file, so even if the ledger is momentarily compromised, a verifier would catch that the government's key suddenly changed without proper authorization). This is a general risk for any blockchain-based PKI. Using a widely-secured chain with decentralized governance mitigates much of this.

- *Quantum Attacks:* We address below how DxID remains secure against quantum adversaries who could break traditional public-key crypto.

Post-Quantum Resilience

One of DxID's strong points is its use of **zk-STARKs, which rely only on hash functions and information-theoretic security**, avoiding classical cryptographic hardness assumptions that quantum computers threaten. Let's break down the components:

- **STARK Proofs:** They use collision-resistant hash functions (like SHA-256 or similar). Quantum attacks (Grover's algorithm) can at best quadratically speed up finding preimages, meaning if we double hash output size, we counter that. So by choosing sufficiently large hash outputs (e.g., 256-bit), STARKs are considered **quantum-resistant* [threeprotocol.ai](#)82]. There are no required trapdoors or structures that Shor's algorithm can exploit (unlike factoring or discrete log in SNARKs).
- **Digital Signatures:** Currently, many issuers might use ECDSA or RSA to sign credentials. Those are *not* post-quantum. In a fully quantum-safe DxID, issuers would instead use post-quantum signature schemes (like CRYSTALS-Dilithium or Falcon which are lattice-based, or stateful hash-based signatures like XMSS). The DID Documents could list PQ keys. The nice thing is, DxID's proof system wouldn't have to change – it can prove statements about PQ signatures just as well as classical ones (with perhaps some circuit adjustments). Until issuers move to PQ algorithms, there is a vulnerability that a sufficiently powerful quantum adversary could forge issuer signatures (e.g., fake a credential by breaking the issuer's public key). This is a broader problem beyond DxID. However, migrating to PQ signatures is a known path (NIST has standardized some). DxID is flexible to accommodate those keys when available.
- **DID Registry/Blockchain:** Ethereum's current cryptography (ECDSA, etc.) is also quantum-vulnerable. A quantum attacker could potentially steal funds or alter transactions by breaking private keys. The community plans to migrate to PQ crypto in the future (though it's a huge undertaking). If DxID is on a chain that doesn't upgrade in time, an attacker with a quantum computer could, say, forge a DID update transaction or

impersonate someone's on-chain account. This risk is common to all blockchain applications, not unique to DxID. One mitigating factor: identity data could be anchored on multiple ledgers or use quantum-secure blockchains (there are some or one could timestamp data in Bitcoin's blockchain via hash, and Bitcoin addresses can use PQ schemes in future).

- **Interim solution:** In anticipation, DxID could encourage using hybrid schemes (e.g., requiring two signatures – one ECDSA, one PQ – for critical issuers' transactions). Already, the W3C DID spec allows multiple verification methods. An issuer could publish both a classical and a PQ public key in their DID Document; verifiers could check that credentials are signed by at least one of them. As PQ adoption grows, they can phase out the classical key. This way, the system remains secure even if a quantum attacker breaks one scheme, as they'd still need to break the other.

In summary, **the zero-knowledge layer of DxID is intrinsically post-quantum secure**[threeprotocol.ai77](#)], which is a major advantage over systems that rely on legacy zk-SNARKs (which use elliptic curves that quantum computers could break the security of, by solving discrete log). For example, if a system used Groth16 SNARKs, a quantum attacker who obtained the toxic waste from setup or could break pairing-based assumptions might forge proofs; STARKs have no such trapdoor.

DxID's only remaining quantum vulnerability is in the credential signatures and DID ledger, which are being addressed industry-wide. We can say DxID is **“quantum-resistant by design, and quantum-robust with prudent cryptographic agility.”** This is important for an identity system, because identities ideally last a lifetime – we want credentials issued today (e.g., a birth certificate) to still be verifiable 30 years from now in a (possibly) post-quantum world. With DxID, as long as issuers rotate to PQ keys, the previously issued credentials can be resigned or reissued in PQ form, and the proofs remain valid. Many current identity systems (including Sovrin, which uses CL-signatures over RSA) would require a major overhaul to be quantum-safe; DxID is ahead by choosing STARKs.

Trusted Setup vs Transparency

A significant security benefit of DxID using STARKs is the **elimination of trusted setup ceremonies**. Some competing ZK systems (like zk-SNARK frameworks) require a one-time setup where secret randomness (toxic waste) must be destroyed, because if an attacker got it, they could forge proofs at [wchain.linkchain.link11](#)]. This was often cited as a potential Achilles' heel for SNARK-based identity schemes – if someone compromised the setup, they might issue themselves fake credentials that still produce “valid” proofs. STARKs avoid this entirely[threeprotocol.ai77](#)]. The security rests only on standard assumptions (hash security). Thus, DxID has **no dependency on a trusted third party for its cryptographic soundness**. This makes it easier to audit (the algorithms are public and don't hide secrets) and removes any concern of backdoors in the proving system.

Security Benefits Summary

- **Data Minimization:** Complies with privacy by design – only necessary info is shared, reducing exposure and breach impact. As one source noted, reducing shared personal data means verifiers have less to guard and less liability [curemagazine.com](https://www.curemagazine.com/2020/07/20/identity-privacy-by-design/) ⁸⁷].
- **Strong Authentication:** Every claim is digitally signed by an authority and/or proven in ZK – far more reliable than scanned documents or self-assertions. This reduces identity fraud (e.g., hard to fake a diploma or ID card).
- **Encryption and Transport Security:** While not a core focus of DxID (transport encryption is handled by protocols like HTTPS or DIDComm), the system assumes credentials and proofs in transit are sent over secure channels. The wallet likely encrypts any stored credentials. These are standard practices to complement the cryptographic privacy. If an attacker intercepted an encrypted credential, they can't read it. If they intercepted a ZK proof, it's one-time and reveals nothing sensitive anyway.
- **Resilience:** No single point of failure – identities are self-sovereign (user holds keys), ledger is decentralized (no central CA that if hacked breaks entire system, unlike X.509 PKI where compromising one CA can cause issues). Even if one issuer's keys are compromised, only that issuer's credentials are affected, not others. And if detected, that issuer's DID can be marked compromised on-chain and relying parties will reject its credentials.
- **Account Recovery:** One must consider security of key management (outside scope of cryptography). DxID like other SSI systems must have recovery methods if user loses keys (e.g., social recovery, multi-sig guardians). Those are operational security issues. But at least, losing a key doesn't leak data – it just means others can't verify new claims from you until you recover. We can implement DID rotation so user can recover by proving identity to issuers and getting new credentials tied to a new DID, etc. That process can involve out-of-band verification (like going to DMV to re-establish a DID if you lost everything – similar to getting a new ID if you lost your wallet). This is not unique to DxID but worth noting in deployment.

Finally, **security vs. usability:** DxID strives to maintain strong security without sacrificing too much usability. By automating cryptographic checks and using friendly DID references, the complexity is under the hood. Users just see a “Verified” badge or get to click “Share proof” in their wallet. This is similar to how TLS is extremely complex but a user just sees a lock icon. We emphasize that all these security features should operate seamlessly from the user's perspective.

Integration with IoT, AI, and Cross-Border Identities

The principles of DxID can extend beyond human identities to devices, organizations, and even AI agents, enabling a unified and interoperable identity layer for the emerging digital ecosystem. Additionally, because DIDs are global, DxID can facilitate cross-border recognition of identities and credentials, easing international interactions.

IoT Integration (Identity of Devices and Secure Data)

In the Internet of Things, billions of devices (sensors, appliances, vehicles) need secure identities and the ability to trust each other's data. Traditional device identity often relies on device certificates issued by manufacturers – which is centralized and sometimes hard to manage (e.g., recalls for certificate updates). DxID offers:

- **Device DIDs:** Each device can be provisioned with a DID and key pair at manufacturing. The DID could be recorded on a public ledger or a local edge ledger. The manufacturer issues a verifiable credential to the device's DID, such as *"Device type: Thermostat, Model ABC, Serial 123, manufactured by Company X on 2025-06-01."* This credential, signed by the manufacturer's DID, acts as the device's identity "passport."
- **Secure Boot & Component Attestation:** A device can store credentials for its components or software (e.g., a TPM in a computer could have credentials for BIOS version, OS patches). When the device joins a network, it can prove via ZK or selective disclosure that it's running approved firmware (without revealing the firmware binary itself). For instance, *"I am Device X and I attest my software hash is one from the approved list"* could be proved using a ZK proof to a hub, enhancing supply chain security for IoT updates.
- **Mutual Authentication:** Devices can authenticate each other by exchanging and verifying credentials. For example, a smart home hub (with its own DID) might require that any device connecting presents a credential signed by an authorized manufacturer and perhaps a proof it hasn't been reported as stolen or compromised (via a revocation check credential). This prevents rogue or counterfeit devices from infiltrating. Standard protocols like MQTT could incorporate this identity verification step (with DxID credentials instead of just passwords or insecure IDs).
- **Privacy for Sensor Data:** IoT sensors often collect personal or sensitive data. Using DxID, a sensor can tag its data with its DID and even sign the data. Consumers of the data can verify it came from a genuine sensor (preventing spoofing). If the data itself is sensitive, zero-knowledge proofs can be used in aggregate. For example, a smart electric meter might prove to the grid *"total monthly usage > 100 kWh"* for billing tiers without revealing detailed daily readings (which could invade privacy). The meter has a credential with its calibration; it signs its encrypted readings and also provides a ZK proof of the computed total. The utility smart contract or system verifies the proof and the meter's device credential. This ensures data integrity and user privacy.

- **Edge and Fog Computing Authentication:** As IoT devices delegate tasks to local servers or AI (fog computing), those servers (AI agents) can have DIDs and credentials too (like “Certified processing node by Company Y”). Devices verify the fog node’s credential before sending data for processing, mitigating man-in-the-middle risks. And the fog node can prove to the device that it will handle data according to certain policies (e.g., *“I have a credential that I comply with GDPR storage rules”*).
- This all builds an **IoT Web of Trust**, where decisions (like whether to trust a sensor reading or allow a device command) can be made based on cryptographic credentials and proofs rather than hardcoded serial number lists or proprietary trust anchors. It’s decentralized – no single vendor lock-in – and scalable as devices roam (a car could authenticate to a smart city infrastructure in any city because its credentials are globally verifiable, not just recognized in its home domain).

AI Integration (Identities for AI Agents and Data Provenance)

As AI systems become autonomous agents (chatbots, personal assistant AIs, decision-making systems), giving them verifiable identities and ensuring the integrity of their training and outputs becomes important:

- **AI Agent DIDs:** An AI (for example, a chatbot or an IoT AI agent that monitors systems) can have a DID just like a person or devcheqd.io [9]. Its DID Document might list its public key (for signing messages) and perhaps a controller (the organization or person who owns the AI). Attaching a DID to AI outputs means you can verify an output came from a specific AI, not from an impostor or an unknown source.
- **AI Credentials (Model Cards):** There is a concept of “model cards” in AI – documentation of an AI model’s training data, accuracy, ethical considerations. These could be turned into verifiable credentials. E.g., an AI developer issues a credential: *“AI Model Z, version 1.2, trained on dataset XYZ (which is 80% public data and 20% licensed data), achieves accuracy 95% on benchmark Q.”* This credential is signed by the developer or an auditing firm. When the AI interacts or makes a claim, it can present this credential to establish trust or context. For instance, a medical diagnostic AI could present proof of its FDA certification credential before giving advice.
- **Zero-Knowledge Proofs for AI Ethics:** Some future applications might require an AI to prove it didn’t use certain data or biases. Using advanced ZK, an AI could prove it was trained on a dataset that meets certain properties (like no personal data, or balanced classes) without revealing the proprietary dataset. This is an active research area (ZK-SNARKs on neural network integrity), beyond current practical use, but DxD’s framework could incorporate it when ready. An AI could carry a proof *“I did not train on any personal user data”* provided by an auditor who uses ZK to check the weights of the model cheqd.io [87]. This helps address AI transparency and fairness in a privacy-preserving

way.

- **AI Agents with Wallets:** If an AI agent operates on blockchain (like a trading bot), giving it a DID and credentials could allow regulation: e.g., only AI agents that have a credential *“Registered Algorithmic Trader with SEC”* can execute certain contracts. The AI would present that credential to the contract (perhaps via an on-chain token as proxy) to be allowed. This ensures accountability; if the AI misbehaves, its credential can be revoked. It also gives humans insight – the DID could be looked up to see who controls that AI (the DID Document’s controller field might list an organization’s DID).
- **AI-generated Content Attribution:** Building on the content authentication earlier: as AI systems generate content (text, images, deepfakes), if they sign their output with their DID, consumers can trace it. E.g., a news article might say “This video was generated by XGPT model; here’s its verifiable model ID.” If the model is known to be a parody generator, you take it less seriously; if it’s a verified factual AI, you trust it more. Also, if a malicious actor tries to pass off AI content as human, the lack of a credible DID signature or credential (like a journalist license) could raise suspicion. Conversely, AI could embed watermarks or proofs in output that a verifying service (maybe an AI itself) can detect to say “this has known AI provenance cheqd.io”].
- **Human-AI Collaboration Logging:** In scenarios where AI assists humans (like a doctor’s AI assistant making suggestions), having the AI’s actions signed by its DID and recorded (with timestamps) can create an audit trail. If something goes wrong, one can see what the AI suggested versus what the human did, with cryptographic certainty. This can be crucial for liability and learning.

Cross-Border and Global Identities

DIDs are URL-like and not tied to any nation, which makes them inherently global. DxID can help in:

- **Travel and Digital Passports:** One could have a digital passport credential (issued by their government’s immigration authority as a verifiable credential). When crossing a border, instead of handing a booklet, one could present the credential (likely via a secure device-to-device link) plus a proof of relevant info (citizenship, name, etc.). The border agent (or automated gate) verifies the issuing government’s signature (with its DID, which could be universally known via something like DID:gov:us etc.) and checks any visas or health credentials similarly. Some efforts like ICAO’s Digital Travel Credential are moving this direction (not using DIDs yet, but similar concepts). DxID could unify these – a traveler’s wallet could hold passport, visa, vaccination certs from multiple countries, and present them in one go. The receiving country’s systems verify each credential’s issuer signature (which can be automated through DID resolution – e.g.,

did:gov:ca for a Canadian vaccine certificate, did:icao:us-visa for a US visa).

- **Refugee and Stateless Identities:** People who lack recognized ID could bootstrap a trusted identity through NGOs or coalitions that act as issuers. Because DIDs are not under a single government's control, a coalition of organizations could issue a "global identity credential" to a refugee's DID, which could then be accepted by services. For example, a UN agency's DID could sign a credential "Name: A.B., DOB, photo, status: refugee, country of origin X" for a person who lost all documents. Border officials or banks could choose to trust that credential because it's signed by UNHCR's known DID, even if the person's home government is unreachable or untrusted. This empowers individuals in cross-border situations to identify themselves with credentials widely recognized, not just paper letters.
- **Education and Jobs Across Borders:** A student from Country A with a DxID degree credential can apply for a job in Country B and instantly prove their degree; the employer in B verifies Country A's university DID signature (the trust may come via a registry of accredited universities DIDs). Similarly professional licenses can be recognized internationally by just accepting the foreign credential if one trusts the issuer. This could reduce bureaucracy (no need for "notarized translations" or contacting foreign institutions manually – the digital signature suffices).
- **International Compliance:** Cross-border trade could see DxID use: e.g., a product exported from Country X to Y comes with credentials (as discussed in supply chain) that Y's customs verifies. Right now, often exporters get a certificate stamped by Chamber of Commerce etc. – in future a digital equivalent (verifiable credential) could be instantly verified by the importing country's customs by looking up the Chamber's DID. This speeds up customs clearance and reduces fraud (hard to forge those certs).
- **Global Directory and Interop:** Because all these DIDs and credentials follow global standards, an ecosystem emerges where any verifier can theoretically verify any credential if they trust the issuer's DID. Compare this to today: verifying a foreign document often requires expert knowledge or third-party services. DxID automates cryptographic verification and shifts the problem to "*do we trust this issuer?*", which can be managed via international agreements or networks of trust (maybe a web-of-trust where organizations vouch for others by issuing meta-credentials like "*Government of X is recognized by Government of Y*" – one can imagine foreign ministry DIDs endorsing each other, etc., to help verifiers decide trust).

In summary, the integration potential of DxID is broad:

- **IoT:** Provides secure identities and trusted data for devices, improving security and automation in smart systems.

- **AI:** Ensures AI agents and outputs carry verifiable identity and compliance info, mitigating risks of rogue AI actions and allowing integration of AI in regulated environments.
- **Cross-Border:** Creates a more fluid world where identity and credentials don't stop at borders – verifiability and trust become portable, which can enhance global commerce, travel, and human mobility.

DxID's flexible, decentralized, and interoperable framework means it can be the backbone identity layer whether it's a person logging into a website, a drone reporting its flight path to a city's traffic system, or an algorithm proving its training pedigree. By unifying these under one identity metasytem, we reduce silos and increase trust across domains.

README: Getting Started with DxID (Developer Guide)

This section serves as a developer-oriented guide (like a README) for implementing and using the DxID system. It provides instructions for installation, building, and running a reference implementation (CLI or application), and examples of common operations. The goal is to help developers or technical users quickly try out DxID.

Installation and Setup

DxID is provided as an open-source toolkit (for example, a combination of smart contracts, a client library in TypeScript/Python, and reference wallet code). To get started:

1. **Prerequisites:** You should have:
 - A blockchain client or access to a node (e.g., an Ethereum node or use a testnet via Infura).
 - Python 3.8+ or Node.js (if using the TypeScript library).
 - For zk-STARK proving, the reference implementation uses **[py-stark]** (a hypothetical Python STARK library) or WebAssembly builds for browser usage.
2. **Install the DxID library:**
 - **Python:** `pip install dxid` (installs the DxID SDK and CLI).
 - **Node.js:** `npm install @dxid/sdk` (installs the JS/TS SDK).

- This will include modules for DIDs, credential handling, and proof generation/verification.
- Optionally, deploy or obtain addresses for the DxID Registry smart contract on your target blockchain (for testing, we provide one on Ethereum Ropsten network).

Configuration: If using a local blockchain (e.g., Ganache or Hardhat for testing), configure the SDK with the RPC endpoint and deploy the registry contract:

```
bash
CopyEdit
export DXID_RPC_URL=http://localhost:8545

dxid deploy-registry  # CLI command to deploy the DID Registry
contract
```

3. This outputs the contract address which the SDK will use for DID operations. (On testnets/mainnet, the address is known/preset in the SDK.)

Creating an Identity (DID) and Managing Keys

Using the DxID CLI:

```
bash
CopyEdit
# 1. Create a new DID identity (generates a key pair and DID Document)
dxid did:create --network ropsten
```

This will output something like:

```
vbnet
CopyEdit
🎉 DID created: did:dxid:ropsten:0x5A1F...D1C
🔑 Private key saved to wallet (didstore.json) with alias "my-did-1"
```

By default, the CLI stores keys in an encrypted file (you'll be prompted to set a password). The DID is derived from the Ethereum address of the public key (here 0x5A1F...D1C).

Register the DID on-chain:

bash

CopyEdit

```
dxid did:register did:dxid:ropsten:0x5A1F...D1C --from my-did-1
```

This sends a transaction to the DxD Registry. `--from my-did-1` uses the private key for signing. After confirmation, your DID Document (with your public key) is live on Ropsten. Anyone can now resolve your DID and get your public key.

Developer note: The CLI can also update or rotate keys:

bash

CopyEdit

```
dxid did:rotate did:dxid:ropsten:0x5A1F...D1C --newkey newKeyFile.json
```

This would update the on-chain DID Document to a new public key (requires proving control of the old key, which the CLI handles if you have the old key in your wallet).

Issuing a Verifiable Credential

Let's simulate an issuer (e.g., a university) and a holder (a user):

- Ensure the issuer also has a DID (create one as above, or use an existing DID).
- As issuer, prepare a credential JSON (following W3C format). You can use the CLI to template this:

bash

CopyEdit

```
dxid vc:issue --issuer did:dxid:ropsten:0xUNiv...123 --subject
did:dxid:ropsten:0x5A1F...D1C \

    --claims '{"degree": "Bachelor of Science", "major": "Computer
Science", "year": 2025}' \

    --output alice_degree_cred.json
```

This command takes the issuer DID, subject DID, and a JSON of claims, and produces a signed credential:

pgsql

CopyEdit

Credential issued and saved to `alice_degree_cred.json`

The content of `alice_degree_cred.json` will be something like:

json

CopyEdit

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "id": "urn:uuid:...some-uuid...",
  "type": ["VerifiableCredential", "DegreeCredential"],
  "issuer": "did:dxid:ropsten:0xUNiv...123",
  "issuanceDate": "2025-06-01T....Z",
  "credentialSubject": {
    "id": "did:dxid:ropsten:0x5A1F...D1C",
    "degree": "Bachelor of Science",
```

```
{
  "major": "Computer Science",
  "year": 2025
},
"proof": {
  "type": "EcdsaSecp256k1Signature2019",
  "created": "...",
  "verificationMethod": "did:dxid:ropsten:0xUNiv...123#key-1",
  "proofPurpose": "assertionMethod",
  "jws": "eyJ...signature..."
}
}
```

This is a standard VC with a JWS (JSON Web Signature) proof, signed by the issuer's DID key (Secp256k1 in this case). The holder (Alice) now has this credential file.

(In a real scenario, the issuer would convey this credential to Alice's wallet via an API or QR code, rather than saving to a local file. But for testing, a file suffices.)

Presenting a Proof (Selective Disclosure)

Now Alice wants to prove something from that credential – say her major – to an employer, without revealing other details like year or degree level (this is a bit contrived because year and degree aren't too sensitive, but it's an example of selective disclosure).

Alice uses the CLI or SDK to generate a zero-knowledge presentation:

bash

CopyEdit

```
dxid vc:prove --credential alice_degree_cred.json \
  --reveal "major" \
```



```
--holder did:dxid:ropsten:0x5A1F...D1C \  
  
--challenge "random-nonce-from-employer" \  
  
--output alice_presentation.json
```

Options:

- `--reveal "major"` indicates she chooses to reveal the `major` claim openly.
- By omission, `degree` and `year` will be kept hidden, but she could still prove something about them if needed using `--prove-statement` (e.g., `--prove-statement 'year >= 2020'` if that was required).
- `--challenge` is a nonce provided by the verifier (to avoid replay; here we assume verifier gave her “random-nonce-from-employer”).

The CLI will:

1. Verify the credential’s signature (to ensure it’s valid and not tampered).
2. Construct a STARK proof where:
 - Public inputs: the nonce, her subject DID (if needed), the revealed claim “major: Computer Science”.
 - Private inputs: the hidden claims (`degree`, `year`).
 - It then proves that the credential was issued by the stated issuer, to her DID, and that all the claims in it (revealed and hidden) are exactly as in the credential, and the signature was valid.
 - It packages the proof and the revealed data into a Verifiable Presentation structure.

After completion:

pgsql

CopyEdit

Presentation generated: `alice_presentation.json`

The `alice_presentation.json` might look like:

json

CopyEdit

```
{
  "type": ["VerifiablePresentation"],
  "holder": "did:dxid:ropsten:0x5A1F...D1C",
  "verifiableCredential": [],
  "proof": {
    "type": "DxIDZKProof2025",
    "challenge": "random-nonce-from-employer",
    "issuer": "did:dxid:ropsten:0xUNiv...123",
    "revealedAttributes": {
      "major": "Computer Science"
    },
    "proofValue": "<huge base64 blob representing STARK proof>"
  }
}
```

We leave `verifiableCredential` array empty or omitted because we are not sending the full credential (to preserve privacy). Instead, the proof contains `issuer` and `revealedAttributes` and a `proofValue` which encodes the fact that all attributes (including "degree" and "year") were verified against a valid credential signature but not revealed.

Verifier Checking a Presentation

The verifier (employer) uses the DxID SDK (or CLI) to verify what they received from Alice. If they got `alice_presentation.json`, they do:

bash

CopyEdit

```
dxid vp:verify alice_presentation.json
```

The tool will:

- Parse the presentation, see it's a ZK proof type.
- Fetch the issuer's DID Document (from chain) to get issuer public key.
- Verify the issuer's signature portion (often the proof will include a signature of the credential hash or similar inside the STARK, or as part of `proofValue`).
- Run the STARK verification on `proofValue` with the given public inputs (nonce, issuer key, revealed "major"). This yields true/false.
- Check the holder's DID signature if the presentation was also signed by holder (some presentations might include an outer holder signature to ensure it came from the holder's key; in our case, we might rely on the ZK proof itself binding the holder's DID).
- Output the result.

Sample output:

yaml

CopyEdit

Verification succeeded:

- Issuer: did:dxid:ropsten:0xUNiv...123 (University X) ✓
- Holder: did:dxid:ropsten:0x5A1F...D1C ✓
- Revealed attributes: { "major": "Computer Science" } ✓

- All proofs are valid. Unrevealed attributes were verified against issuer's credential.

The employer thus learns that:

- The credential was indeed issued by University X to this holder.
- The holder's major is Computer Science (and this is authentic).
- They do not learn the holder's degree level or graduation year (those were not revealed).

If something was off (invalid proof or mismatch), the tool would report failure and likely an error message about what didn't verify.

Building from Source and Running Tests

For developers wanting to modify DxID:

- The source code is on GitHub (e.g., github.com/dxid/dxid). Clone it.
- It includes smart contract code (Solidity) under `/contracts`, core libraries under `/sdk` (for various languages), and a `/circuits` or `/prover` directory for the STARK prover definitions.
- You can run `npm run test` or `pytest` depending on the language, to execute unit tests. The repository includes test vectors (like a sample credential and a corresponding proof) to ensure all components interoperate.
- For example, a test might issue a credential, then generate a proof of age, then simulate a verifier check – asserting that the result is true for the correct proof and false if any bit is changed.

Using DxID in Applications

- **Web App Integration:** You can use the JS SDK in a web app to request a proof from a user's wallet. For instance, using a QR code or a deeplink:
 - The verifier web app generates a random challenge and displays a QR code containing a `didcomm` message or deep link like `dxid://prove?issuer=did:dxid:...&request=age_over_18&challen`

ge=12345.

- The user scans it with their DxID-compatible mobile wallet. The wallet finds the relevant credential, creates the proof, and either returns a QR for the verifier to scan or sends it directly (if there's an established communication channel like a relay).
- The verifier app receives the proof and calls `dxid.verifyProof(proof)` in the JS SDK, which performs as above and returns true/false plus the revealed data.
- **Mobile Wallet Reference:** The DxID project provides a reference mobile wallet (perhaps as an extension of an existing wallet like MetaMask or a standalone). This wallet manages DIDs and credentials. From a UX perspective, it would show a user:
 - Their identities (DIDs) and which credentials are associated.
 - Options to share/prove attributes with consent (with clear UI showing what will be revealed).
 - The underlying crypto (like STARK proving) is done in the app (potentially using a WebAssembly module for performance).
- **Smart Contract Integration:** If writing a Solidity contract that uses DxID, you might import an interface of the Registry to check a DID's key or use an oracle pattern as discussed. The documentation in the SDK guides how to design such contracts. For now, no solidity verifier for STARK is provided (due to gas cost), so the pattern is to rely on an off-chain process or issued token as described. The README in `/contracts` covers deploying the registry and an example "KYCGuard" contract that requires a token to unlock a function.

Licensing and Contributions

DxID is released under the **MIT License**, making it free to use and integrate in commercial and open-source projects. (The MIT License text is included in the repository – see the `LICENSE` file.)

We welcome contributions via pull requests on GitHub. All contributors are expected to sign a Developer Certificate of Origin (DCO) for IP clarity.

Further Documentation

- **Specification:** For detailed protocol specs, see the [docs/](#) folder which includes the technical whitepaper (this document) and JSON schemas for credentials and presentations.
- **Examples:** The [examples/](#) folder contains sample code: e.g., a Python notebook demonstrating a full flow (issuing a health certificate and verifying it with a proof of a condition).
- **Community & Support:** Join our Discord or Slack (link available in README online) for help, or check the GitHub discussions for common Q&A. We also maintain a Wiki for tips (like gas optimization techniques, or how to persuade an issuer to adopt DxID).

By following this README guide, developers can start experimenting with decentralized identities and zero-knowledge proofs using DxID, and integrate strong privacy-preserving identity verification in their applications.

Licensing and References

Licensing: The DxID project is open source, released under the MIT License. This permissive license allows you to use, modify, and distribute DxID in your own solutions, provided you include the license notice. (For full text, see the [LICENSE](#) file in the repository.) Components such as the zk-STARK proving system may include contributions from other libraries which are MIT or Apache-2.0 licensed – we maintain attribution for those in our NOTICE file. There are no patent claims or royalties; DxID is intended as an open standard implementation.

References: This document has cited various sources (white papers, articles, standards) to ground its statements and to credit prior work. Below is a list of references used, in the order they appeared, with cross-references to the in-text citation markers:

1. Three Protocol Blog – *"How ZK-STARKs Are Shaping the Future of Digital Identities without KYC"* – on zk-STARK features: no trusted setup, quantum resilience, [ethreeprotocol.ai](#)⁶³】
2. Chainlink Education – *"zk-SNARK vs zk-STARK – Explained Simple"* – introduction of STARKs by Ben-Sasson et al. 2018 and a quote about the tension between privacy and integr[chain.linkchain.link](#)⁴¹】.
3. Chainlink Education (cont'd) – details on STARKs moving computations off-chain and transparency (no trusted set[chain.linkchain.link](#)⁶³】.
4. Chainlink Education (cont'd) – comparison of SNARKs and STARKs, highlighting SNARK trusted setup issues and STARK proof size trade-o[chain.linkchain.link](#)⁸²】.

5. Sovrin vs UPort comparison (SureSync blog) – noted UPort on Ethereum vs Sovrin on permissioned ledger; UPort’s decentralization vs Sovrin’s governance info.suresync.nl info.suresync.nl43】 and data minimization issues in UPort (correlation via funding account info.suresync.nl51】.
6. Sovrin privacy (SureSync blog) – Sovrin’s use of agents and zero-knowledge proof for selective disclosure info.suresync.nl info.suresync.nl43】.
7. W3C Verifiable Credentials Data Model – description of roles (Issuer, Holder, Verifier) and how selective disclosure and privacy are handled in the standard info.suresync.nl curity.medium.com57】 (implicitly referenced regarding W3C compliance).
8. Curity/TheNewStack article – stressed no direct link between verifier and issuer in decentralized identity, enhancing privacy curity.medium.com57】, and noted sharing subset of data or using ZKPs to avoid oversharing curity.medium.com58】.
9. Curity Medium – example of use case (proving age > 18) and benefit to verifier (less data to manage under GDPR curity.medium.com87】.
10. Implementing SSI (SureSync) – UPort’s approach and shortcomings in data minimization, and Sovrin’s contrasting approach info.suresync.nl info.suresync.nl51】.
11. Veres One Primer – described as a public utility for identity, emphasizing privacy by design (minimize shared data, GDPR compliant veres.one63】.
12. Cryptoslate article on ZK in supply chains – outlines how ZKPs allow proving claims like ethical sourcing without exposing details cryptoslate.com cryptoslate.com16】, and verifying compliance to regulators privately cryptoslate.com28】, and anti-counterfeit for luxury goods cryptoslate.com37】.
13. Cryptoslate (cont’d) – use case scenarios: ethical sourcing privacy cryptoslate.com16】, regulatory compliance privacy cryptoslate.com28】, authenticity in luxury goods cryptoslate.com37】 – all aligning with our supply chain discussion.
14. Forbes (cheqd) – mention of establishing AI agent trust through D cheqd.ioL9】 (used in our AI section conceptually).
15. Cheqd blog – describing a framework (Verifiable AI) where each AI agent has a DID and credentials (training data compliance, etc cheqd.io cheqd.io87】, and linking content authenticity to cryptographic proof and tamper-evident ledger cheqd.io03】.
16. UNSW Business School article – argument that decentralized identity can combat deepfakes, with scenario of every public figure having a public key on blockchain and

signing content, enabling instant verification of any video's authentic[businessthink.unsw.edu.au](https://businessthink.unsw.edu.au/businessthink.unsw.edu.au)⁹⁸].

17. Sovrin AnonCreds paper or Hyperledger Indy documentation – implied reference for CL-signature ZKP and revocation via accumulators (not explicitly cited in text, but the details we gave align with known Indy docs).
18. W3C DID Core spec – implied reference for DID Document structure and methods.
19. Our own knowledge base/training data was used where citations weren't explicitly found (for general statements about systems like uPort or Veres One), but we anchored critical claims with references above.

These references underpin our design rationale and show DxID is built on concepts proven in prior work, now combined and advanced with more powerful cryptography.

Concluding Note: DxID brings together the strengths of decentralized identity standards and zero-knowledge proofs to create a secure, private, and scalable identification system. By following this technical specification and utilizing the provided tools, developers and organizations can implement DxID to enhance privacy and trust in digital interactions. The future of identity is decentralized and privacy-first – and DxID is a concrete step toward that future.